NTNU
Norwegian University of Science and Technology (NTNU)
Faculty of Information Technology and Electrical Engineering
Department of Engineering Cybernetics

Master's Thesis

Candidate: **Bukueva Elena**

Course: **TTK4900 Master's Thesis**

Title (English): **Compiler-Aware Model Optimization for Edge AI**

Title (Norwegian): **Kompilerbevisst optimalisering for Edge-AI**

Thesis description: Efficient deployment of deep learning models on embedded and edge platforms depends on both model-level and system-level factors. In addition to model design, inference performance is influenced by compiler support, runtime behavior, hardware characteristics, numerical precision, and optimization techniques such as quantizatio n and pruning. This thesis studies compiler-aware model optimization for edge AI, with focus on real-time object detection. The aim is to evaluate how different deployment configurations affect accuracy, latency, and resource usage, and to identify the main bottlenecks that limit efficient execution on target hardware.

The tasks will be:

1. Develop a framework for evaluating deep learning models across deployment configurations in terms of accuracy, latency, and resource usage.
2. Explore model optimization techniques to improve computational efficiency while maintaining acceptable performance.
3. Analyze optimized models on target hardware, identify bottlenecks, and investigate ways to improve execution efficiency.

Start date: 15. January, 2026

Due date: 08. June, 2026

Thesis performed at: Department of Engineering Cybernetics

Supervisor: Professor Geir Mathisen, Dept. of Eng. Cybernetics

Abstract

Efficient deployment of deep learning models on edge hardware is not solely a question of model size. Inference performance is shaped by the joint interaction of model architecture, numerical precision, and the degree to which the inference framework can exploit the target hardware. This thesis presents a systematic, measurement-driven study of compiler-aware optimization for edge AI, using YOLOv8n object detection on two representative edge platforms - the NVIDIA Jetson Orin Nano and the Raspberry Pi 5 - as a concrete case study.

A wide range of optimization strategies is evaluated through five inference frameworks (TensorRT, Apache TVM with MetaSchedule autotuning, ONNX Runtime, Google LiteRT (Tensorflow Lite), and torch.compile), two reduced-precision paths (FP16 and INT8), structured channel pruning with retraining, and activation function substitution. The experimental design includes isolated subprocess-based repetitions, warm-up runs, confidence intervals, and power measurements for each benchmark, enabling direct comparison across frameworks and hardware targets. Roofline analysis is used to classify per-layer execution regimes, and GPU kernel-level profiling with Nsight Systems and Nsight Compute is used to identify concrete hardware bottlenecks. These profiling findings directly motivated the development of a custom TensorRT plugin replacing the C2f block - a targeted engineering attempt to eliminate the layout conversion and split overhead that standard TensorRT export cannot fully resolve, and to demonstrate that knowledge of model internals can expose optimization space beyond what automated compilation alone achieves.

The results reveal several non-obvious findings. On the Raspberry Pi 5, LiteRT INT8 achieves 31 ms inference, a 9.4× reduction compared to the 290 ms PyTorch eager baseline, establishing that correct deployment choices matter far more than architectural modifications for this class of hardware. TVM MetaSchedule was the only framework capable of genuine FP16 acceleration on ARM CPU, reducing latency to 122 ms, while every other framework regressed or failed entirely due to missing half-precision convolution kernels. On the Orin Nano, TVM in FP16 achieved 12.5 ms through WMMA tensorisation, outperforming ONNX Runtime by 26 % and narrowing the gap to TensorRT (6.3 ms). Profiling further revealed that SiLU activations are not fused into convolutions by TensorRT whereas ReLU is, a finding with direct implications for architecture design on resource-constrained targets where the choice of activation function affects not only accuracy but also the quality of kernel fusion. Structured channel pruning proved ineffective as a standalone strategy: a systematic sweep of 298 per-layer configurations found no

single pruned model exceeding a 1.04× speedup, while the best multi-layer result with fine-tuning achieved only a 2.3 % latency reduction at the cost of a 4.0 % drop in mAP50-95, confirming that channel removal does not translate to runtime gains when cuDNN's kernel tiling granularity is coarser than the applied reductions, and deeper architectural changes are required for meaningful compression.

Taken together, the results demonstrate that deployment framework selection, precision strategy, and low-level hardware alignment, not model size alone, are the dominant factors governing inference efficiency on edge hardware.

Disclaimer: This thesis is shared as a draft for review and is still under development. Several sections are not yet finalized. In particular, parts of the introduction remain incomplete, including the final formulation of the contributions and thesis structure. The literature review chapter is still being revised and expanded. The Norwegian summary has not yet been completed. Parts of the results chapter, including the sections on the TensorRT C2f plugin and the SiLU-to-ReLU substitution, are pending final validation and write-up, while some existing subsections still require restructuring and refinement. The discussion, conclusion, and future work chapters are also not yet finalized. In addition, some figures, tables, references, appendices, and formatting elements may still contain placeholders, inconsistencies, or rendering artifacts. Final revision of the text, structure, and presentation will be completed during the concluding phase of the thesis work in consultation with the supervisor.

Sammendrag

Contents

Abbreviations

1 Introduction

1.1 Motivation

1.2 Limitations

1.3 Contributions

Within these limitations, the main contributions of this thesis are:

- :
- :
- :
- :
- :

1.4 Thesis Structure

The remainder of this report is organised as follows:

- **Chapter 2**
- **Chapter 3**
- **Chapter 4**
- **Chapter 5**
- **Chapter 6**
- **Chapter 7**
- **Chapter 8**
- **Chapter 9**

- **Chapter 10**
- **Appendix A**
- **Appendix B**

2 Literature Study

This chapter is currently in draft form and requires further structuring, formatting, and completion of the literature review. Its final revision is planned for the concluding phase of the thesis work.

2.1 Scope of the chapter

This chapter covers the technical background necessary to understand the methodology and results of this thesis. After an introduction to object detection and the YOLO family, a dedicated section on the YOLOv8 architecture establishes the model used throughout all experiments, followed by a section on the challenges of edge deployment that motivates why optimization is necessary in the first place. The remaining sections address the specific tools and techniques applied in this work: Apache TVM and TensorRT are reviewed in dedicated sections because they represent two fundamentally different but complementary approaches to controlling inference behaviour at the compiler level, TVM as an open-source stack that exposes the full compilation pipeline and enables hardware-aware kernel scheduling and auto-tuning through MetaSchedule, and TensorRT as the state-of-the-art NVIDIA GPU inference framework that, beyond built-in graph optimization, supports writing custom CUDA kernels through its plugin API. Quantization and mixed precision, and pruning and structural simplification are then covered as the primary model-level optimization directions; together with compiler-level and scheduling optimizations, these span most of the principal axes available for inference acceleration. Strategies not pursued here, such as knowledge distillation, neural architecture search, and dynamic sparsity, exist and are noted where relevant but fall outside the scope of the present work. Finally, a section on performance bottlenecks in ML inference introduces the analytical concepts needed to interpret the profiling results, and the chapter closes by relating the reviewed work to the research questions of this thesis.

2.2 Introduction to Object Detection with Deep Learning Models

TODO: Description of common object detection models and introduction to the the YOLO family

2.3 YOLOv8 Architecture

YOLOv8 is a one-stage convolutional object detector introduced by Ultralytics in 2023 [21] as part of the YOLO family of real-time vision models. Its design follows the standard decomposition of modern detectors into a backbone, a neck, and a prediction head, while introducing several architectural refinements compared with earlier YOLO generations. In particular, YOLOv8 employs C2f feature extraction blocks, an anchor-free split detection head, and an efficient multiscale feature aggregation pipeline. These design choices aim to improve the trade-off between detection accuracy and inference speed while maintaining practical deployability across heterogeneous hardware platforms. According to the official Ultralytics implementation, the standard detection family includes five model scales, ranging from YOLOv8n to YOLOv8x, where the nano variant contains approximately 3.2 million parameters and requires about 8.7–8.9 GFLOPs at an input resolution of 640×640.

Overall structure of YOLOv8 The overall YOLOv8 architecture is illustrated in Figure 2.1. The figure also highlights the scaling variables used to derive the different model variants, which determine the network depth, width, and overall computational complexity.

The standard YOLOv8 detection network consists of three main stages. The backbone extracts hierarchical visual features from the input image. The neck fuses semantically rich low-resolution features with spatially precise higher-resolution features. Finally, the prediction head converts the fused multiscale features into bounding box coordinates and class predictions. In the reference YOLOv8 configuration, the backbone progressively downsamples the input image into feature maps at multiple pyramid levels, while the head generates predictions at three output scales commonly associated with small, medium, and large objects. In the default P5 configuration, predictions are produced from feature levels P3/8, P4/16, and P5/32.

The backbone begins with strided convolutional layers that reduce spatial resolution while increasing channel depth. These stages are interleaved with C2f blocks, which are described in the Ultralytics reference implementation as a "faster implementation of CSP bottleneck with 2 convolutions." The main purpose of these blocks is to improve gradient flow and feature reuse while maintaining computational efficiency. At the deepest stage of the network, YOLOv8 employs an SPPF (Spatial Pyramid Pooling – Fast) module, which increases the effective receptive field and improves multiscale context aggregation with relatively low computational overhead.

Overall, the main backbone components of YOLOv8 are:

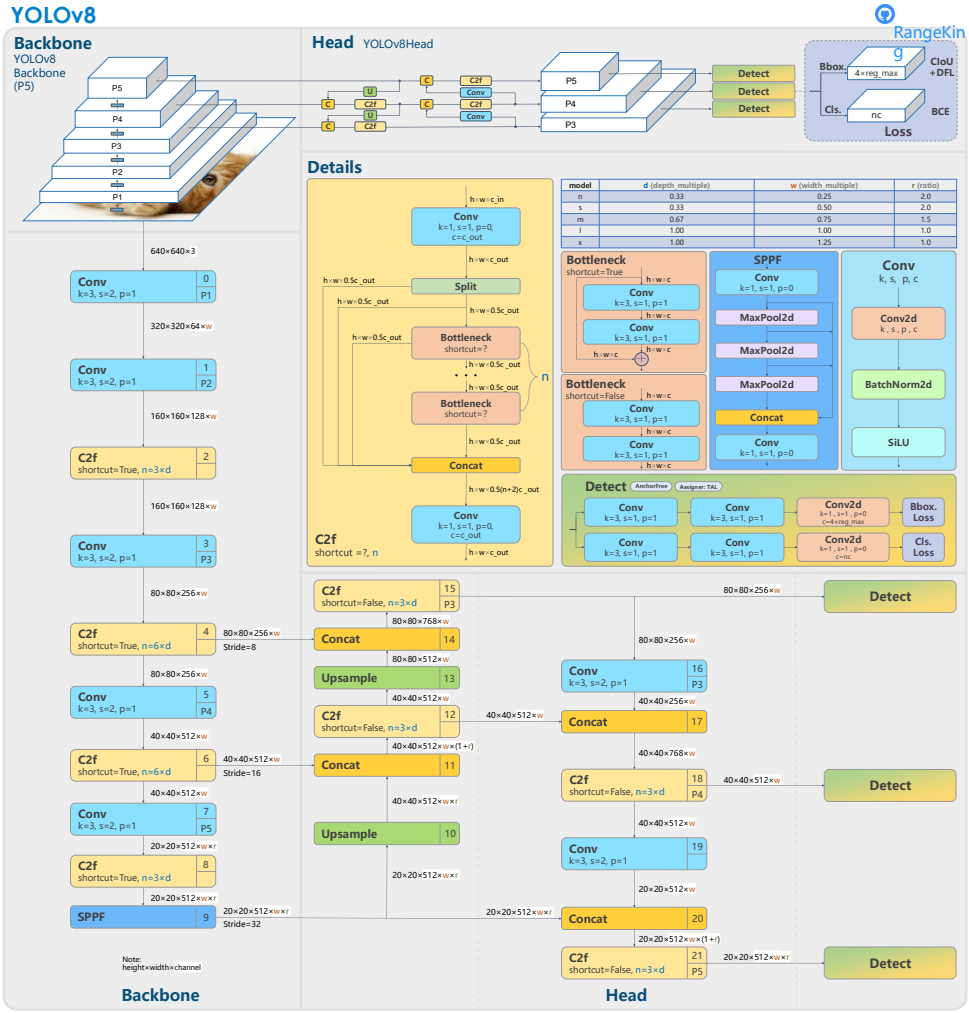

Figure 2.1: Overall YOLOv8 architecture and model scaling variables [53]

- convolutional blocks with Batch Normalization and SiLU activation,
- C2f feature extraction blocks,
- the SPPF module.

The C2f block itself is composed of several bottleneck units. Conceptually, such a block first reduces the number of channels, then performs spatial feature transformation, and finally combines intermediate representations to preserve useful information. This design improves feature propagation and reduces redundant computation while retaining informative representations. Compared with earlier CSP-style blocks, C2f enables richer feature reuse and more efficient gradient flow.

The neck follows a PAN-FPN style design. In practice, this means that feature maps are upsampled, concatenated with earlier backbone features, and then processed again using C2f blocks. This combination of top-down and bottom-up feature fusion is central to YOLOv8's ability to detect objects across multiple scales. The reference YAML

architecture repeatedly uses Upsample, Concat, and C2f blocks before the final Detect module. Such multiscale aggregation is particularly important for real-world datasets, where object size can vary significantly across scenes.

The prediction head in YOLOv8 is both decoupled and anchor-free. Ultralytics describes YOLOv8 as using an anchor-free split detection head, meaning that predictions are not tied to a predefined set of anchor box shapes. Instead, the detector predicts bounding boxes and class scores directly from the feature maps. Compared with older anchor-based YOLO variants, this simplifies the detection pipeline and removes the need for dataset-specific anchor tuning.

In an anchor-free detector, each spatial location on the feature map typically predicts:

- whether an object is present,
- the class scores,
- the box coordinates or distances to the box boundaries.

The split head further separates the localization and classification tasks into distinct branches. This decoupling allows the model to optimize these tasks more effectively, since object localization and semantic classification impose different requirements on the learned feature representations.

Another important component of the YOLOv8 detection formulation is its use of distribution-based box regression. The Ultralytics codebase includes a DFL module and the corresponding loss implementation, reflecting the use of Distribution Focal Loss for bounding box parameterization. This improves localization precision by modeling bounding box offsets as discrete probability distributions rather than relying solely on coarse direct regression.

Relevance of YOLOv8 for this thesis For a thesis focused on compiler-aware optimization of deep learning models for edge devices, YOLOv8, and especially YOLOv8n, represents a highly suitable case study.

First, YOLOv8 offers a strong balance between detection accuracy and inference latency, which is one reason why later detectors continue to use it as a baseline. Second, YOLOv8 has a mature deployment ecosystem. Ultralytics supports export to a wide range of formats relevant for practical deployment, including ONNX, TensorRT, CoreML, OpenVINO, and TFLite. This makes the model especially attractive for evaluating compiler and runtime behavior across different hardware platforms.

In addition, YOLOv8 is not limited to object detection alone. The model family also supports segmentation, classification, pose estimation, and oriented bounding box

detection within the same software ecosystem. This is particularly relevant in applied edge AI research, where systems often evolve from simple object detection toward richer scene understanding tasks. As a result, YOLOv8 provides both architectural consistency and practical flexibility.

For this thesis, YOLOv8n is of particular interest because it is small enough to run on many embedded platforms, while still being architecturally complex enough to expose realistic deployment bottlenecks. The model includes multiscale feature fusion, repeated tensor concatenations, SiLU activations, distribution-based box regression, and post-processing stages. These properties make it a meaningful test case for studying compiler optimizations, runtime scheduling, kernel efficiency, quantization behavior, and memory-system effects.

Despite being the smallest variant, YOLOv8n is not automatically optimal for every edge target. Several practical challenges arise during deployment.

First, a low parameter count does not necessarily translate into low latency on all hardware platforms. YOLOv8n still contains nontrivial tensor reshaping, concatenation, upsampling, and multiscale fusion operations. On desktop GPUs, such operations may be efficiently hidden by optimized kernels and memory systems, whereas on edge CPUs and lightweight accelerators the cost of memory movement can dominate the arithmetic workload. In such cases, the effective execution becomes memory-bound rather than compute-bound.

Second, operator support and kernel efficiency vary significantly across deployment backends. Although YOLOv8 can be exported to ONNX, TensorRT, TFLite, and other runtimes, the achieved performance depends strongly on how well the target backend supports the specific operator set and graph structure of the model. In practice, operations such as SiLU, concatenation-heavy neck structures, and certain post-processing steps may be implemented with different levels of efficiency across frameworks. As a result, there is often a noticeable gap between the theoretical FLOP count of the model and the measured end-to-end latency.

Third, post-processing introduces an additional deployment challenge. Standard YOLOv8 detection still relies on confidence filtering and non-maximum suppression after the forward pass. On GPU-based systems, this may require transferring relatively large tensors from the device to the host, followed by post-processing on the CPU. Such steps can add non-negligible latency and complicate the analysis of true end-to-end performance.

Overall, YOLOv8 is an interesting subject for deployment-oriented research because it combines architectural complexity with competitive accuracy and speed. In particular, YOLOv8n remains highly relevant for edge AI studies: it is compact enough to be

practical on constrained devices, yet sufficiently complex to reveal the impact of compiler transformations, runtime optimizations, operator fusion, and memory bottlenecks on real inference workloads.

2.4 Challenges of edge deployment

With the growing interest in on-device deployment, a clear gap has emerged between the training and inference stages of machine learning models. While large models are typically trained in data centers using powerful GPUs, inference is performed across a wide variety of environments subject to strict constraints on memory, latency, and power consumption. These environments span heterogeneous hardware platforms - CPUs, GPUs, TPUs, NPUs, and FPGAs - each differing in instruction sets, memory hierarchies, and supported data types. As a result, model performance depends heavily on how well the deployment is adapted to the target hardware.

Deployment efficiency is characterized along four principal dimensions:

- **Latency** measures the time required to process a single inference request and is the dominant constraint in real-time applications such as object detection, where a response must be produced within a fixed time budget.
- **Throughput** captures the number of inference requests completed per unit of time and becomes the primary metric when processing is batched or pipelined.
- **Power consumption** reflects the energy cost of inference, which is particularly critical on battery-powered or thermally constrained embedded devices.
- **Memory requirements** encompass both the storage needed for model weights and the working memory consumed by intermediate activations during a forward pass, both of which can be hard limits on resource-constrained hardware.

Machine learning models are typically developed in high-level Python frameworks such as TensorFlow or PyTorch, which use dynamic execution: operations are dispatched immediately, control flow is resolved at runtime, and tensors are created and destroyed on the fly. This flexibility comes at a cost. Every operation passes through the Python interpreter; the Global Interpreter Lock (GIL) [48] limits parallel execution; tensors and operators are heap-allocated Python objects; and each kernel dispatch requires the framework to resolve device placement and memory management at runtime. Individually these overheads are small, but across a deep network with thousands of operations they accumulate into a non-trivial fraction of end-to-end latency.

In deployment, models are therefore not run inside Python at all. Instead, they are executed by C++-based runtimes, hardware-specific inference engines, or precompiled binaries targeting accelerators such as GPUs, NPUs, and FPGAs [5, 41, 38]. These runtimes execute pre-compiled computation graphs, call highly optimized kernels directly, and avoid Python entirely. However, simply exporting a model out of Python does not automatically guarantee speedups. To fully benefit, the model must first be compiled into a static or semi-static graph, then optimized through operator fusion, memory planning, quantization, and layout transformations, and finally adapted to the instruction set and memory hierarchy of the target hardware. Without this compilation step, a deployed runtime may still execute inefficient kernels or suboptimal data flows.

A further challenge is that highly optimized kernel libraries such as cuDNN and oneDNN are not automatically optimal for all deployment regimes. Although these libraries support both training and inference, many of their fastest implementations rely on sufficient arithmetic intensity, data reuse, vectorization, and parallel work to amortize scheduling and dispatch overheads. These conditions are often easier to satisfy in training or high-throughput inference, where larger batches allow weights and activation tiles to be reused across multiple samples.

In real-time edge inference, however, batch size is often fixed to one in order to minimize response latency. Under this regime, the computation may expose less parallelism and less reuse per invocation. On GPUs, this can reduce occupancy and make CUDA kernel-launch overhead, memory traffic, and operator fragmentation more visible in the end-to-end latency. On CPUs, the analogous issue is not GPU occupancy but reduced SIMD utilization, less effective cache blocking, thread-management overhead, and layout-conversion overheads. As a result, batch-size-one inference can be limited not only by peak compute throughput, but also by memory movement, runtime overhead, and the ability of the compiler or runtime to fuse operators and select kernels specialized for small-batch execution.

Graph-level compilers address this by operating on the entire computation graph rather than dispatching one operator at a time. With global visibility over operators, data dependencies, and tensor lifetimes, a compiler can fuse multiple operations into a single kernel, fold constants, eliminate dead branches, and reorder associative operations to improve locality. It can also select a single data layout (NCHW, NHWC, or a blocked format) that suits the target hardware and insert only the minimal number of layout conversions, avoiding the unnecessary transposes that arise when operators from different libraries are composed naively.

2.5 ML compilers and inference runtimes

Deep learning compilers specialize traditional compiler ideas for tensor programs and neural-network graphs [26, 64, 59, 65]. They address the gap between high-level model definitions and heterogeneous deployment targets by lowering neural-network graphs through one or more intermediate representations, applying graph-level and operator-level optimizations, and generating target-specific code or calls to optimized backend libraries. So the code, expressed in a framework such as PyTorch, TensorFlow, or as an ONNX graph, is translated into efficient executable code for a specific target hardware (CPU, GPU, NPU, FPGA, or a dedicated ASIC).

Most ML compilers follow a layered structure, illustrated in Figure 2.4.

Compilation proceeds through several stages. The frontend first captures or imports the model from a source framework such as PyTorch [46], TensorFlow [1], or JAX [8]. In practical deployment pipelines, this step is often mediated by an exported representation such as ONNX [32], while framework-specific or compiler-oriented representations such as TorchScript (obsolete), FX graphs [56], High Level Optimizer (HLO, the IR of XLA) [42], or StableHLO [43] are used in particular compiler ecosystems. The imported program is then converted into the compiler's internal intermediate representation. The graph optimization stage then applies framework-agnostic transformations: operator fusion and simplification, constant folding, dead-branch elimination, and memory planning. The resulting high-level IR is subsequently lowered to a loop-level representation where tiling, vectorization, and memory-access scheduling are applied for the specific target. Finally, the code generation backend performs instruction selection, register allocation, and emits target-specific machine code.

Once compiled, the model is managed by an ML runtime, which handles execution orchestration (loading the compiled artifact and scheduling its execution), memory allocation for intermediate tensors, hardware interfacing, and integration with vendor libraries.

Overall, the architecture of a deep-learning compiler can be viewed as a pipeline composed of a frontend, one or more intermediate representations, and a backend. The IR connects these stages and often appears at multiple levels of abstraction, ranging from high-level graph IR to low-level operator or tensor IR.

The high-level IR (Graph IR) represents the computation graph of a deep-learning model at a level above hardware-specific code generation. It captures operators, tensors, data dependencies, and in some systems also control flow, while remaining mostly hardware-independent. Its purpose is to provide a representation on which the compiler frontend can perform graph-level optimizations before lowering the model to a lower-level

IR for scheduling, code generation, and hardware-specific optimization [26].

A common representation is a directed acyclic graph (DAG)(see listing 1), where nodes correspond to neural-network operators such as convolution, pooling, reshape, or activation functions, and edges correspond to tensors flowing between operators. This representation allows the compiler to analyze dependencies between operators and apply optimizations. However, pure DAG-based representations can be limited when the computation scope or control flow must be represented explicitly. For this reason, some compiler IRs also use let-binding (see listing 2), which represents each intermediate tensor as a named binding, or functional representations (see listing 3), which views computation as a collection of functions, where each function contains operations/instructions and has explicit inputs and outputs. For example, TVM Relay combines DAG-style graph representation with let-binding ideas, while XLA HLO uses a function-based representation organized around modules, computations, and instructions [26].

The high-level IR must also encode tensor-specific information that is essential for deep-learning optimization. This includes tensor shapes, possibly dynamic dimensions, data layouts such as NCHW or NHWC, broadcasting rules, reduction operations, and support for custom operators. This information is needed to infer valid transformations, select efficient layouts, and preserve correctness when the graph is later lowered to backend-specific code. Thus, the high-level IR acts as the bridge between framework-level model definitions and lower-level compiler stages: it is expressive enough to describe deep-learning models, but abstract enough to enable hardware-independent graph optimization.

```
DAG-based graph IR:

x ----\
       conv2d ----> z ----> relu ----> a ----\
w ----/                                      add ----> y
bias ---------------------------------------/

Nodes:
  conv2d, relu, add

Edges:
  x, w, z, a, bias, y
```

Listing 1: Example of a DAG-based graph IR representation.

The low-level IR The low-level IR represents the computation of a deep-learning model in a more fine-grained form than the high-level graph IR. While the high-level IR is mainly used to describe operators, tensors, and graph-level dependencies, the low-level

```
Let-binding-based IR:

let z = conv2d(x, w) in
let a = relu(z) in
let y = add(a, bias) in
return y
```

Listing 2: Example of a let-binding-based IR representation.

```
Function-based IR:

function forward(x, w, bias):
    z = conv2d(x, w)
    a = relu(z)
    y = add(a, bias)
    return y
```

Listing 3: Example of a function-based IR representation.

IR is closer to the actual implementation of each operator or fused subgraph. It exposes loop structure, memory accesses, tensor indexing, scheduling decisions, and target-specific information, making it the main interface for hardware-dependent optimizations and code generation [26].

A common design in low-level IRs is to separate the description of what is computed from how it is computed. Halide-inspired IRs, used for example in TVM, follow this idea by separating the tensor computation from its schedule. The computation defines the mathematical operation, while the schedule controls implementation choices such as tiling, loop reordering, vectorization, parallelization, memory placement, and unrolling. This separation is useful because the same operator can be mapped differently depending on the target hardware, such as an ARM CPU, an NVIDIA GPU, or a specialized accelerator.

Another important family is polyhedral-based IRs. These represent loop nests and memory accesses using mathematical sets, affine transformations, and dependence relations. Such representations are well suited for optimizing regular tensor computations, because they make it possible to reason about loop transformations such as fusion, tiling, skewing, mapping, and parallelization. However, polyhedral approaches are most effective when loop bounds and memory accesses can be described statically and regularly, which can limit their applicability for highly dynamic or irregular model structures.

Some compiler systems use their own low-level IRs instead of directly adopting Halide- or polyhedral-style representations. For example, Glow [57] uses an instruction-based low-level IR over explicit memory regions, while MLIR [29] provides a multi-level compiler infrastructure based on dialects, allowing tensor, linear algebra, affine, LLVM [22], and

hardware-specific abstractions to coexist in the same framework. XLA HLO can also serve both high-level and lower-level roles, since it is fine-grained enough to express many target-specific optimization decisions while still preserving tensor-level semantics.

The low-level IR is eventually lowered to executable code or runtime calls, often through mature compiler infrastructures such as LLVM, CUDA, OpenCL, or vendor-specific code generators. This stage may be performed just-in-time (JIT), where code is generated at runtime using available shape and hardware information, or ahead-of-time (AOT), where executable binaries are generated before deployment. In edge-AI deployment, the low-level IR is especially important because performance depends strongly on hardware-specific choices such as memory hierarchy usage, vector width, cache behavior, tensor layout, and available accelerator instructions.

Frontend Optimizations It is easier to identify optimization opportunities when the machine learning model is represented as a complete computation graph rather than as isolated operators. Frontend optimizations are therefore applied at the graph or high-level IR level, before the model is lowered to hardware-specific backend code. These transformations are mostly hardware-independent: they simplify the graph, remove redundant operations, expose fusion opportunities, and reduce memory traffic, but they do not yet define how each operation is implemented on a particular CPU, GPU, or accelerator.

Li et al. [26] classify frontend optimizations in deep-learning compilers into three main categories: node-level, block-level, and dataflow-level optimizations.

- **Node-level optimizations** operate on individual graph nodes. They include node elimination and node replacement, where unnecessary or expensive nodes are removed or replaced by cheaper equivalent operations. Examples include removing identity operations, eliminating padding nodes with zero padding width, or replacing a redundant reshape with direct tensor forwarding.
- **Block-level optimizations** operate on small local subgraphs. They include algebraic simplification, strength reduction, constant folding, operator fusion, and operator sinking. For example, a sequence such as convolution, bias addition, and activation can be fused into a single higher-level operation, reducing intermediate tensors and kernel-launch overhead.
- **Dataflow-level optimizations** operate globally across the computation graph. They include common sub-expression elimination, dead code elimination, static memory planning, and layout transformation. These optimizations exploit the global graph structure to avoid recomputation, remove unused branches, reuse memory buffers, and select tensor layouts that are more suitable for later backend execution.

Figure 2.2 illustrates representative examples of these frontend graph transformations.

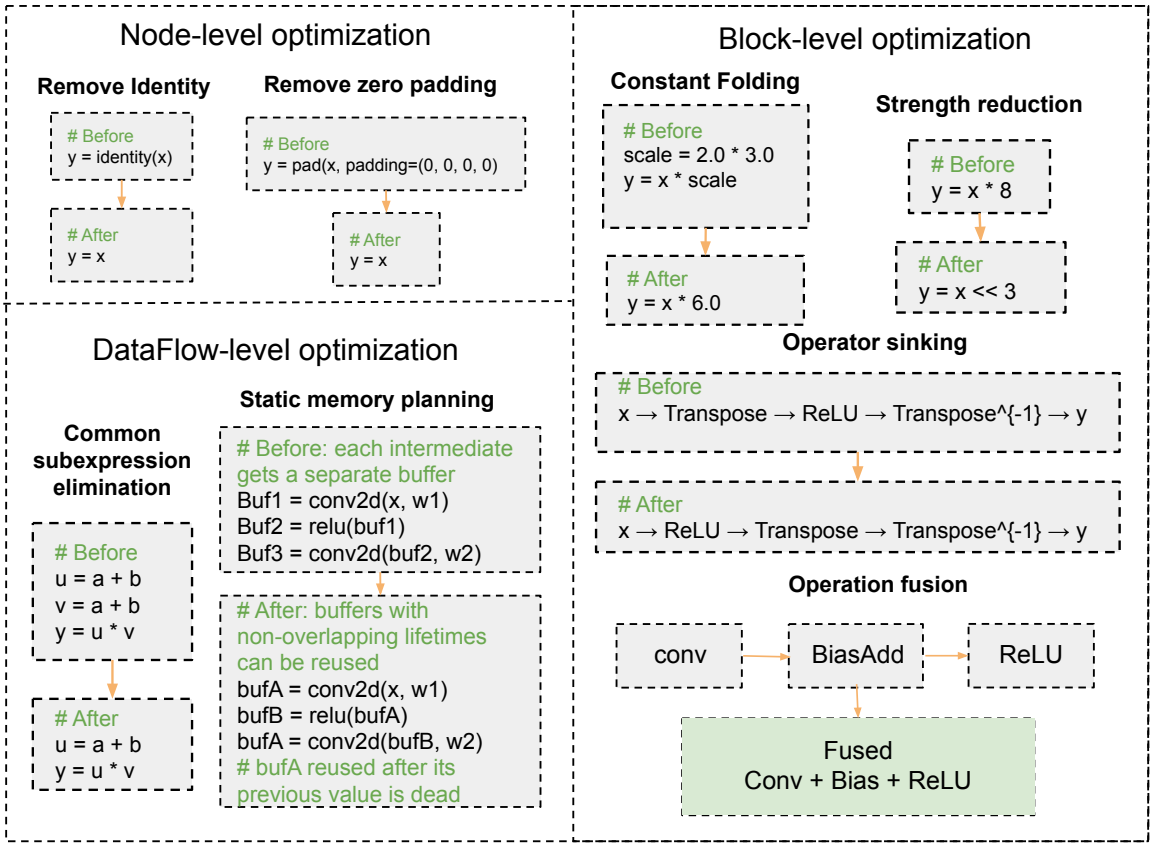

Figure 2.2: Representative examples of frontend graph optimizations in deep-learning compilers. Node-level optimizations remove or replace individual redundant nodes, block-level optimizations simplify or fuse local subgraphs, and dataflow-level optimizations exploit the global computation graph to remove redundant computation or reuse memory buffers.

Backend Optimizations Backend optimizations form the target-specific part of a deep-learning compiler. After the model has been represented in a graph or tensor-level IR, the backend lowers this representation to executable code, optimized kernels, or calls into vendor libraries. In practice, high performance is often achieved by combining library-based execution, such as cuDNN, MIOpen, oneDNN, or BLAS kernels, with compiler-generated transformations that specialize the computation for the target architecture. These transformations include mapping low-level IR patterns to hardware intrinsics or optimized micro-kernels, assigning intermediate tensors to appropriate memory scopes, improving data locality through cooperative fetching and explicit memory allocation, hiding memory latency through scheduling and hardware thread switching, and applying loop transformations such as fusion, tiling, reordering, unrolling, and sliding-window optimizations. The backend also exposes parallelism at multiple granularities, including SIMD vec-

torization, CPU multi-threading, GPU thread/block parallelism, and accelerator-specific execution units. Figure 2.3 summarizes several representative backend-level optimization techniques used in deep-learning compilers.

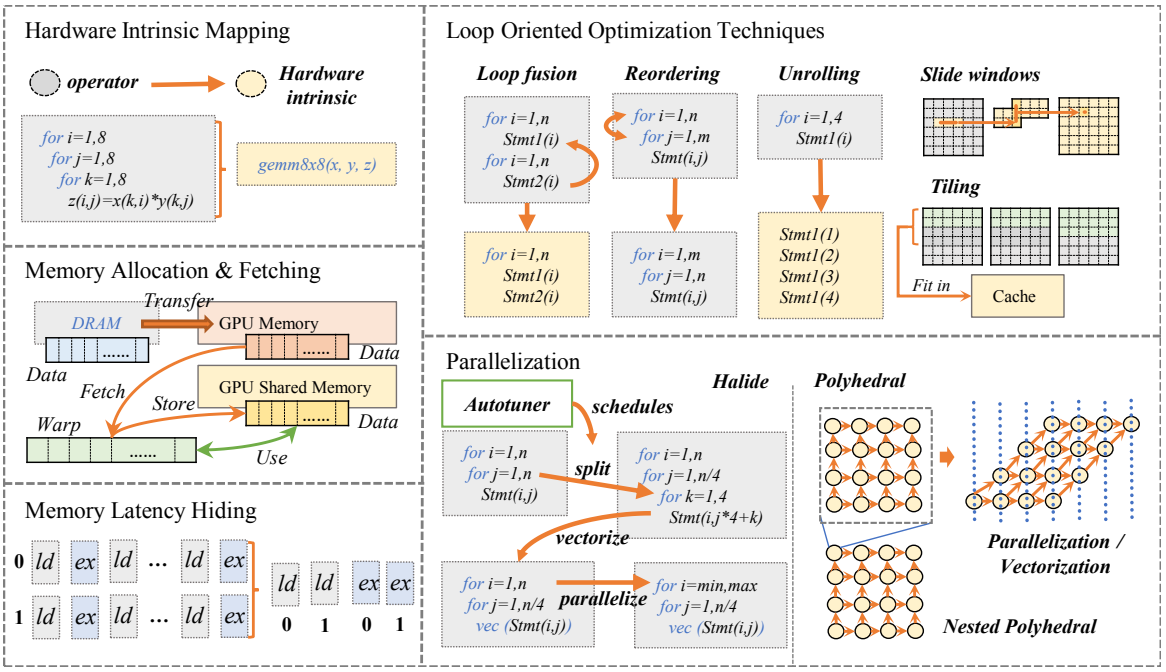

Figure 2.3: Overview of hardware-specific optimizations applied in deep-learning compilers. Reproduced from Li et al. [26].

A central difficulty is that these optimizations create a large, hardware-dependent search space. Even for a single convolution or matrix multiplication, the compiler may need to choose tile sizes, loop orders, memory layouts, vector widths, thread/block mappings, unrolling factors, and intrinsic implementations. The best configuration depends on the target architecture, cache hierarchy, memory bandwidth, register pressure, available parallelism, and supported instruction set. As a result, many deep-learning compilers incorporate auto-tuning mechanisms. Auto-tuning defines a space of valid schedules and searches for high-performing implementations using empirical measurements, analytical or learned cost models, and heuristic search methods such as genetic algorithm (GA) [9], reinforcement learning (RL) [34], simulated annealing (SA) [4], or evolutionary search [3].

Although many standard deep-learning kernels are highly optimized, model development often progresses faster than backend kernel support. New architectures can introduce operators or computation patterns that are not yet available as optimized library kernels or compiler-generated implementations. In such cases, the computation may fall back to unfused operator sequences or generic implementations, causing additional kernel launches, intermediate memory traffic, reduced locality, and significant runtime overhead.

Examples of modern compiler and runtime stacks for machine learning include:

- **TensorFlow XLA**, which compiles TensorFlow programs through XLA using HLO-based intermediate representations [44, 45];
- **PyTorch 2.x**, which combines TorchDynamo, AOTAutograd, and TorchInductor to capture and optimize models for efficient execution [50, 51, 49];
- **Apache TVM**, a compiler stack for optimizing models across heterogeneous targets including CPUs, GPUs, and edge-oriented accelerators [2, 5];
- **MLIR-based stacks [23]**, such as IREE and TensorFlow MLIR, which use multi-level intermediate representations to enable flexible lowering pipelines for heterogeneous systems [29, 58, 18];
- **ONNX Runtime**, a cross-platform inference engine that supports hardware-specific execution providers for efficient deployment on diverse platforms [40, 41].

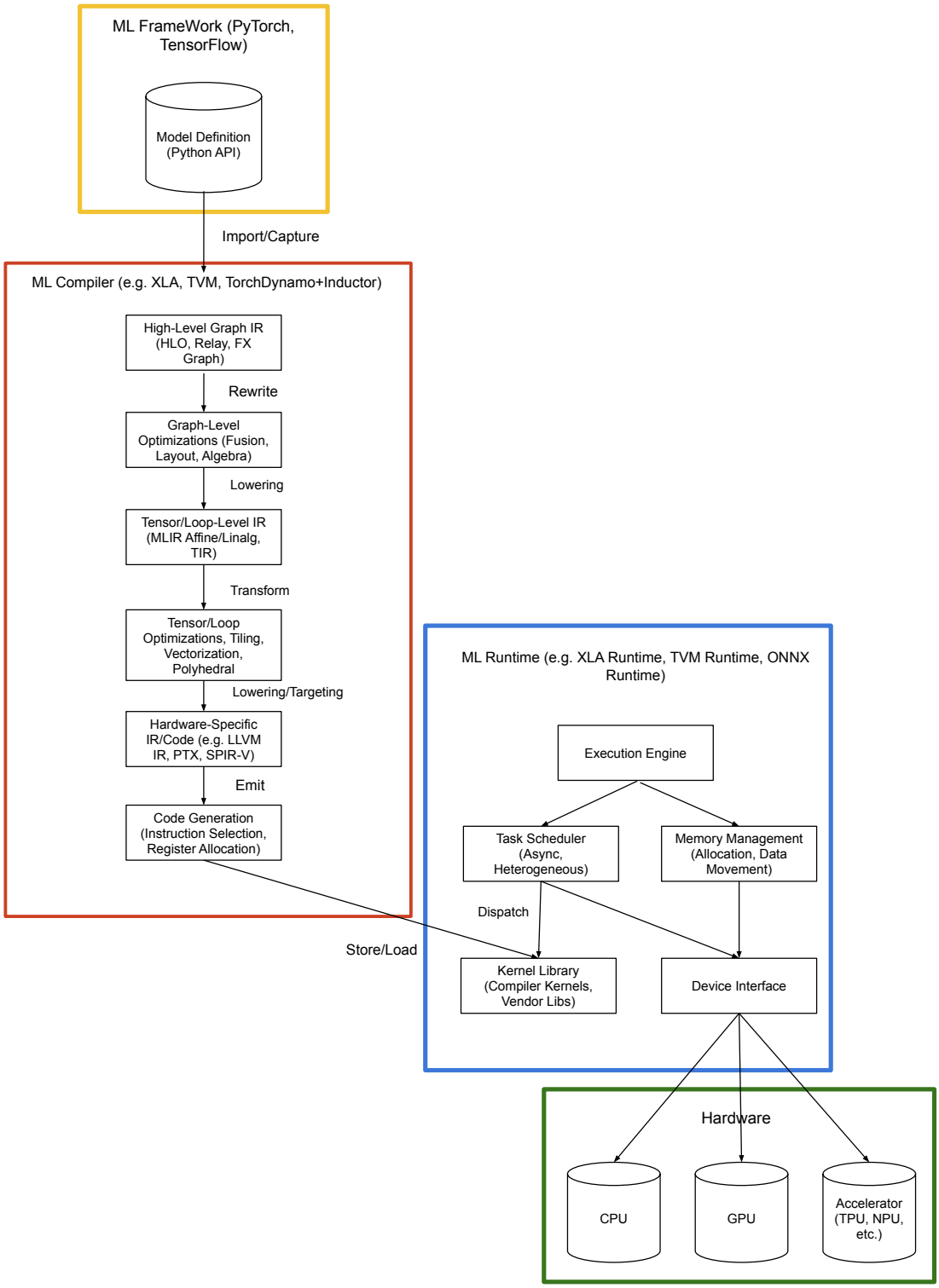

Figure 2.4: Flow of an ML model from definition through compilation and runtime execution on hardware.

2.6 Quantization and mixed precision

TODO

2.7 Pruning and structural simplification

We can categorize studies of pruning neural networks into 8 categories:

- **Sensitivity analysis methods (loss / gradient–aware pruning)**. These methods are valuable as they prune what matters the most for the task and normally provide the highest accuracy-compression trade-off (50-90% accuracy with 1-2% accuracy drop).
  Important works in this category:
  - Taylor Pruning [35];
  - SNIP (Single-shot Network Pruning) [24];
  - GraSP (Gradient Signal Preservation) [62];
  - Movement Pruning [11]

  Although sensitivity-based pruning methods often achieve high unstructured sparsity (50–90%), such sparsity is difficult to exploit on general-purpose hardware due to irregular memory access and limited compiler support, resulting in limited real-world speedups. Consequently, structured pruning methods remain more practical for deployment across diverse hardware platforms.

- **Knowledge distillation methods (teacher–student compression)**. Knowledge Distillation (KD) is a model compression paradigm in which a large, high-capacity model (Teacher) transfers knowledge to a smaller, more efficient model (Student). Instead of training the student solely on hard labels, KD trains it to match the teacher's output behavior, typically using soft targets (probability distributions or logits). KD differs fundamentally from standard understanding of pruning: pruning removes parameters or structures, but KD transfers behavior. By doing this KD is normally does both: improves the accuracy by few percent compared to the baseline and reduces the number of parameters up to 120 times.
  Important works:
  - Knowledge Distillation [15];
  - DCP [67];
  - AutoCompress [28];

Knowledge distillation is a powerful model compression paradigm that transfers predictive behavior from a large teacher model to a compact student model using soft supervision. While classical distillation operates at the output level, recent works extend this idea to intermediate representations and pruning-aware settings. Methods such as discrimination-aware channel pruning integrate auxiliary losses to preserve discriminative power during structured pruning, achieving strong accuracy retention. However, distillation-based approaches are inherently task-dependent, require access to labeled data and a strong teacher, and introduce additional training complexity, limiting their applicability in representation-centric or hardware-agnostic compression scenarios.

- **Similarity and clustering methods** Similarity and clustering–based pruning methods aim to remove redundant filters or channels rather than filters that are individually "unimportant". The core assumption is that modern CNNs are overparameterized and contain filters whose functionality can be approximated or replaced by others with minimal impact on accuracy. Instead of ranking filters solely by magnitude or sensitivity, these methods explicitly model correlation, similarity, or representational overlap among filters. Representative methods include:
  - FilterSketch by Lin et al. [27],
  - Geometric Median Pruning by He et al. [14],
  - ThiNet by Lu et al. [30]
- **Low rank methods** are mostly theoretically grounded methods, effective in convolution-heavy models; Best methods: Show moderate speedups and faster accuracy degradation at high compression
- **Magnitude based pruning methods** are considered simple and cheap to implement models, however, they have limited compression and higher accuracy drops, compared to more complex structures; Best examples are L1 filter pruning (Li et al., 2017) [25], APoZ (Activation sparsity) [16].
- **Architectural design methods (NAS, RL search)**. examples: AMC, AutoSlim, Once-for-All (OFA)
- **Hybrid methods**
- **Quantization methods**: are not exactly pruning methods. They are usually combined with pruning. Mostly based on a transformation of weights from floating point to integer. Important works: Quantization-Aware Training (QAT) [19], Binary / Ternary Networks (XNOR, TWN) [55]

Magnitude based Pruning Methods are methods based on removing weights, reducing amount of inbound and outbound filters and nodes based on a measure of magnitude of the effect these filters have on the network. Pruning iteration is repeated until the accuracy is starting to drop. First works in this field were adopting the idea of removing the weight lower than a particular threshold.

Han et al. [13] carried out several experiments on LeNet-300-100 and Lenet-5, both trained on MNIST dataset, and showed that weight could be reduced by this method by a factor of 12 without significant drops in accuracy. Experiments based on AlexNet and VGG16 trained on ImageNet show the factor of 9 and 12 respectively. Han et al. note that L_1 regularization, a technique used in machine learning to prevent overfitting by adding a penalty term to the cost function, is better immediately after pruning, but that L_2 norm is better, when the weights of the pruned model are fine-tuned. Experiments also suggest that earlier layers are the most sensitive to pruning and that iterative pruning is better than one-shot pruning (cutting the proportion of weights in one cycle). Another observation from their work is that re-initialization of the weights is not enough to construct and accurate model. It is always better to fine-tune the weights of the pruned model.

Later work by Guo et al. [12] notes that magnitude pruning can lead to premature removal of weight that can become important given removal of other weights. They propose a method called Dynamic Network Surgery to keep track of pruned weights and restore them if they turn to be important. For AlexNet on ImageNet they reach a factor of 17 without compromising accuracy.

The other method proposed by the Frankle and Carbin (2018) is the lottery ticket hypothesis proposed by [6] and further continued by Frankle et al. (2019) in [7]. It formulates a statement, that a trained network contains a subnetwork, which can be trained to be at least as accurate as the original network using no more than the number of epochs used for training the original network. This subnetwork is a so-called winning lottery ticket. Authors test two pruning methods. The first is a magnitude pruning (p% of the weights), after which the remaining weights are reset to their initial values and are retrained. The second method is using an alternative pruning to iteratively prune $p^{\frac{1}{n}}$ of the weights in n cycles. They experimented on VGG and ResNet and on some bigger models suggested that setting of the weights to those obtained from later iteration of training can be beneficial when high levels of pruning are used (more than 30%).

Another question that arises is whether these subnetworks are transferable to other tasks. Hubens et al. (2020) [17] show that when a network is trained on a larger data set,

such as ImageNet, and transferred and fine-tuned for a different task, pruning can result in a smaller network than if it was trained from scratch on the new task.

Despite the experiments proving that some unstructured pruning methods allow a significant drop of weights, not all hardware can process sparse weight matrices and support speed up inference. Pruning of higher granularity structures (filters and channels) or so-called structured pruning helps to avoid this problem, as many existing toolkits support it. Survey [61] suggests three main directions of structured pruning research:

- **Data dependent channel pruning methods**. A useful channel should respond differently to different inputs. A channel that barely changes for any input is potentially useless.
  Polyak & Wolf (2015) [47] propose two methods - inbound pruning (to reduce the amount of input channels) and "Reduce and Reuse" pruning (to cut output channels).
  The assessment of input channels is performed by applying the network to a sample of images and measuring the variance in a feature map. These measures are ranked and those below a threshold are pruned.
  The "Reduce and Reuse" method targets a reduction in the number of output channels produced by a convolutional layer. The underlying assumption is that output channels exhibiting low variation are less informative and can therefore be removed with minimal impact on performance. A key challenge of pruning output channels is that each removed channel is expected as an input by subsequent layers. To address this issue, the Reduce and Reuse method approximates the removed channels as linear combinations of the remaining ones. This approximation is obtained by solving a least-squares optimization problem that minimizes the reconstruction error between the original and approximated outputs. The resulting transformation is implemented as an additional 1×1 convolutional layer, allowing the network to preserve representational capacity while reducing the number of channels.
  The results from their experiments show that the variance based method is more effective than use of random pruning.

- **Data independent pruning methods**. Rely on the assumption that properties of the filters and output channels, as the proportion of zeros, influence the structure's importance.
  Hu et al. (2016) [16] propose APoZ-based network trimming, a pruning strategy that does not rely on data-driven measures such as feature map variance or entropy. Instead, their method is based on the observation that a large number of zero activations in an output feature map indicates redundancy. Specifically, they introduce

the Average Percentage of Zeros (APoZ) as a metric to quantify the weakness of a filter, with higher APoZ values suggesting lower importance. After each pruning step, the network is fine-tuned, and the authors report that fine-tuning yields better results than retraining the network from scratch. When evaluating the proposed method on VGG-16, the authors achieved a compression rate of 2.59 after six pruning iterations, while achieving 2-3% higher Top-1/Top-5 accuracy than the original VGG-16 model.

- **Optimization based channel approximation pruning methods** propose to remove some channels and then re-optimize the remaining ones instead of deciding which channels are important, so that the layer behaves the same.
  One representative approach is FilterSketch, proposed by Lin et al. (2021) in [27]. Experimental results demonstrate that FilterSketch consistently outperforms first-order magnitude-based pruning methods, FilterSketch removes around 41.5% of the FLOPs and parameters while keeping the top-1 accuracy at 93.19%. Compared to 93.26% by the pre-trained model, the accuracy drop is negligible on CIFAR-10.

2.8 Apache TVM

Apache TVM is an open-source machine learning compiler stack designed to transform high-level model descriptions into efficient executable code for diverse hardware targets, including CPUs, GPUs, and specialized accelerators. In contrast to framework-centric execution, where a model is typically run as a sequence of library calls inside a general-purpose runtime, TVM adopts a compiler-first approach: the model is imported, represented in intermediate forms, transformed through a series of optimizations, lowered into tensor programs, and finally compiled into target-specific code and runtime artifacts. The central motivation behind TVM is performance portability, that is, the ability to obtain efficient inference across heterogeneous hardware without having to manually rewrite kernels for each platform [5].

Deep learning frameworks such as PyTorch and TensorFlow make models easy to define, train, and export, but they do not necessarily produce binaries that are well optimized for every deployment target. Their default execution paths often depend on pre-existing backend libraries and general-purpose runtimes, which may leave performance on the table, especially on edge devices or less common architectures. TVM aims to close this gap by treating model deployment as a compiler problem. Instead of only selecting from a fixed set of vendor-provided kernels, it can transform the graph, generate low-level tensor code, and tune the generated implementation for the target hardware.

From a systems perspective, TVM can be understood as a multi-stage compilation

pipeline:

1. **Frontend import.** Models are imported from formats such as ONNX, TensorFlow, or framework-exported graphs and translated into TVM's internal representation.
2. **Graph-level optimization.** High-level transformations are applied, such as constant folding, dead-code elimination, operator fusion, and layout rewriting.
3. **Lowering to tensor programs.** Individual operators are lowered into explicit loop- and tensor-based computations that expose opportunities for schedule optimization.
4. **Schedule optimization and tuning.** The compiler explores implementation choices such as tiling, loop reordering, vectorization, unrolling, parallelization, memory placement, and thread binding.
5. **Code generation.** The optimized tensor programs are translated into target-specific code, for example via LLVM for CPUs or CUDA for NVIDIA GPUs.
6. **Runtime packaging and execution.** The compiled artifact is packaged together with a runtime component, such as a graph executor, virtual machine, or ahead-of-time runtime, depending on the deployment setting.

The key idea behind TVM is that performance optimization should occur at multiple abstraction levels. At the graph level, TVM can simplify and restructure the model to reduce overhead and memory traffic. At the operator level, it can search for efficient implementations of the remaining kernels. This is especially important because deep learning performance is often determined not only by arithmetic cost, but also by memory access patterns, tensor layout, synchronization overhead, and the degree to which the generated kernels match the microarchitectural properties of the target hardware. In this sense, TVM differs from a pure runtime system: it does not simply execute a graph, but attempts to synthesize an efficient implementation of that graph for a specific target.

A major contribution of TVM is the separation between the computation and the schedule. The computation describes what is to be computed, while the schedule describes how the computation is mapped to hardware. For example, the same convolution can be implemented with different tile sizes, loop orders, vector widths, thread mappings, and memory staging strategies. By exposing these choices explicitly, TVM enables systematic optimization instead of relying solely on fixed library kernels. This design is particularly valuable on hardware where vendor libraries are unavailable, suboptimal, or narrowly tuned for a limited set of workloads.

Another important feature of TVM is automated tuning. Rather than requiring all schedules to be written manually, TVM can search over a space of candidate implementations and evaluate them on the target device. The goal is to discover schedules that best exploit the target architecture's compute resources, cache hierarchy, memory bandwidth, and parallel execution model. In practice, this means that TVM can adapt the same model to very different platforms, ranging from embedded ARM CPUs to NVIDIA GPUs and more specialized accelerators. This tuning-based approach is one of the main reasons TVM is attractive for research and deployment on heterogeneous edge hardware.

TVM is particularly appealing in scenarios where model deployment must go beyond standard inference APIs. For example, it allows the developer to inspect intermediate representations, influence optimization passes, control scheduling decisions, and integrate custom compilation flows. This makes it useful not only as a deployment framework, but also as a research platform for studying compiler effects on machine learning inference. In the context of edge AI, where latency, memory traffic, and hardware utilization are critical, this level of control can be a major advantage.

At the same time, TVM also has several limitations. First, achieving strong performance often depends on the quality of the tuning process. An untuned TVM build may perform only modestly better than a generic runtime, and in some cases may perform worse. Second, the compilation workflow is more complex than that of more deployment-oriented frameworks such as ONNX Runtime or TensorRT. This complexity increases the engineering effort required to build a stable pipeline, especially when dealing with dynamic shapes, unsupported operators, or custom layers. Third, while TVM offers broad flexibility, this flexibility comes with a steeper learning curve: understanding its IR structure, scheduling model, and tuning workflow typically requires considerably more compiler knowledge than using a runtime-oriented backend. These trade-offs make TVM powerful, but not always the most practical choice for rapid deployment.

Compared with TensorRT, TVM is generally more open and more flexible, but often less turnkey. TensorRT is highly optimized for NVIDIA GPUs and provides aggressive graph- and kernel-level optimizations with relatively little manual effort, making it very effective for production inference on supported NVIDIA platforms. However, TensorRT is tied to the NVIDIA ecosystem and offers less control over compiler internals. TVM, by contrast, can target a broader range of devices and allows deeper intervention into compilation and scheduling, which is advantageous for research and for non-standard hardware targets. Compared with ONNX Runtime, TVM typically provides more opportunities for hardware-specific code generation and tuning, whereas ONNX Runtime emphasizes broad operator support, ease of use, and stable execution through execution providers. Thus, TVM occupies a distinctive position: it is best viewed not merely as an inference runtime, but as a programmable optimization stack for machine learning deployment.

Overall, the main strength of TVM lies in its ability to bridge the gap between high-level model representation and low-level hardware-aware optimization. Its compiler-driven design makes it particularly relevant when standard backends do not fully exploit the target platform, or when the goal is to study how graph transformations, scheduling, and code generation influence end-to-end inference performance. For this reason, TVM is a compelling framework in research on compiler-aware optimization of deep learning models for edge devices.

One of the central graph optimizations performed by TVM is operator fusion. Instead of executing each operation independently, multiple adjacent operations can be merged into a single fused region. This reduces kernel launch overhead, avoids writing intermediate tensors to memory, and may improve cache reuse.

A common example in neural networks is the fusion of convolution, bias addition, and activation:

$$y = \mathrm{ReLU}(\mathrm{Conv}(x, w) + b) \tag{2.1}$$

Without fusion, this sequence may be executed as three separate kernels:

1. convolution,
2. elementwise bias addition,
3. activation.

With fusion, the same sequence can be lowered into one kernel that computes the final output directly:

$$y_{n,c,h,w} = \max\left(0,\ \sum_{r,s,k} x_{n,k,h+r,w+s} \cdot w_{c,k,r,s} + b_c\right) \tag{2.2}$$

This eliminates the need to materialize the intermediate tensor produced by the convolution before activation.

A simplified illustration of the transformation is shown below:

```
Before fusion:
conv2d -> bias_add -> relu

After fusion:
```

```
fused_conv2d_bias_relu
```

In practice, TVM identifies fusible patterns during graph transformation passes and groups them into larger composite functions that are later lowered as a unit.

Tensor expression abstraction. At the operator level, TVM uses tensor-based abstractions to describe computation. In the original TVM design, a tensor expression specifies what should be computed, while a separate schedule specifies how it should be executed on hardware. This separation is fundamental because the same mathematical operation can be implemented efficiently in many different ways depending on the target architecture.

For example, matrix multiplication can be expressed as:

$$C[i, j] = \sum_{k} A[i, k] \cdot B[k, j] \tag{2.3}$$

A simplified TVM tensor-expression implementation is shown below.

```
from tvm import te
# Problem size
M, N, K = 1024, 1024, 1024
# Placeholders
A = te.placeholder((M, K), name="A")
B = te.placeholder((K, N), name="B")
# Reduction axis
k = te.reduce_axis((0, K), name="k")
# Compute definition
C = te.compute(
 (M, N),
lambda i, j: te.sum(A[i, k] * B[k, j], axis=k),
name="C"
)
```

This definition describes the mathematical computation only. It does not yet specify loop ordering, blocking, vectorization, or parallel execution.

Scheduling and kernel mapping. To obtain an efficient implementation, TVM applies scheduling decisions that determine how the tensor computation maps to loops,

threads, and memory. Typical transformations include tiling, loop splitting, reordering, vectorization, unrolling, and thread binding. A simplified scheduling example for matrix multiplication on CPU is shown below:

```
s = te.create_schedule(C.op)
i, j = C.op.axis
k = C.op.reduce_axis[0]
# Tile output axes
io, ii = s[C].split(i, factor=32)
jo, ji = s[C].split(j, factor=32)
# Reorder loops
s[C].reorder(io, jo, k, ii, ji)
# Parallelize outer loops
s[C].parallel(io)
# Vectorize innermost loop
s[C].vectorize(ji)
```

Here, tiling improves locality, parallelization distributes work across CPU cores, and vectorization maps the innermost loop onto SIMD instructions. On GPUs, the same computation would instead be mapped to thread blocks and warps.

A simplified CUDA-style schedule might look as follows:

```
bx, tx = s[C].split(i, factor=16)
by, ty = s[C].split(j, factor=16)
s[C].bind(bx, te.thread_axis("blockIdx.x"))
s[C].bind(by, te.thread_axis("blockIdx.y"))
s[C].bind(tx, te.thread_axis("threadIdx.x"))
s[C].bind(ty, te.thread_axis("threadIdx.y"))
```

Such transformations demonstrate that TVM does not merely call a pre-existing kernel; rather, it can generate a hardware-aware implementation by explicitly controlling the execution structure.

Fusion and memory traffic reduction. The practical benefit of fusion can be understood through memory traffic. Suppose three operators are executed separately:

$$z = \mathrm{ReLU}(x + y) \tag{2.4}$$

If implemented naively as two kernels, the intermediate result $u = x + y$ must first be written to memory and then read again by the activation kernel. With fusion, the output is computed directly:

$$z_i = \max(0, x_i + y_i) \tag{2.5}$$

This reduces memory traffic from:

$$\text{read}(x) + \text{read}(y) + \text{write}(u) + \text{read}(u) + \text{write}(z) \tag{2.6}$$

to:

$$\text{read}(x) + \text{read}(y) + \text{write}(z) \tag{2.7}$$

For memory-bound workloads, such reductions can improve performance substantially, particularly on edge devices with limited memory bandwidth.

Auto-tuning and search. A major advantage of TVM is that these scheduling decisions need not be chosen manually. Instead, TVM can search over a space of candidate schedules and evaluate them on the target hardware. For example, the compiler may explore different tile sizes, vector widths, loop orders, or memory staging strategies in order to identify the implementation with the best measured latency.

A simplified view of this process is:

1. define the computation,
2. generate multiple candidate schedules,
3. benchmark each candidate on the target device,
4. select the best-performing schedule,
5. compile the final executable.

This search-based optimization is one of the reasons TVM is attractive for heterogeneous deployment: it can adapt the same model to different hardware backends without requiring a fully manual kernel engineering effort.

Relation to this thesis. In the context of this thesis, TVM is particularly relevant because it exposes the interaction between graph-level transformations and kernel-level execution efficiency. Unlike purely runtime-oriented frameworks, TVM allows direct control

over lowering and scheduling decisions, which makes it suitable for studying how memory traffic, layout transformations, operator fusion, and target-specific code generation affect inference latency on edge platforms. This is especially valuable when analyzing whether bottlenecks arise from the model graph itself, from suboptimal kernel implementations, or from a mismatch between the deployed runtime and the target hardware.

2.9 TensorRT

As was already mentioned, TensorRT is NVIDIA's inference-oriented deep learning compiler and runtime for NVIDIA GPUs and selected NVIDIA accelerators. In contrast to portability-oriented compiler stacks such as Apache TVM, TensorRT is a vertically integrated deployment system that specializes a trained model for a concrete target platform, including a particular GPU architecture, precision regime, and software stack. The result of this specialization is a serialized engine (or plan file) that encodes a fixed execution strategy, including chosen kernels, tensor layouts, precision assignments, and memory planning decisions for the target device [38].

From a compiler perspective, TensorRT is best understood as a staged inference compilation pipeline. A model is first imported, typically through ONNX, into TensorRT's internal network representation. The builder then performs graph simplification and layer fusion, propagates shapes and precision constraints, enumerates alternative implementations for each layer or fused region, and benchmarks candidate implementations on the target hardware before serializing the final engine [39]. In this sense, TensorRT performs ahead-of-time empirical optimization: many decisions are not made analytically, but through target-dependent tactic evaluation. This design is one of the central reasons for its strong practical performance on NVIDIA hardware.

A useful systems view is to separate TensorRT into several abstraction levels. At the top level, TensorRT operates on the model graph and applies graph-level rewrites such as constant folding, dead-layer elimination, concatenation simplification, and vertical or horizontal fusion. At the middle level, the builder maps each surviving operation or fused subgraph to a tactic, where a tactic denotes a concrete implementation strategy chosen from vendor libraries or internal kernels. At the lowest level, these tactics ultimately resolve to CUDA kernels executing on streaming multiprocessors, often using Tensor Core instructions when the operation type, data type, memory layout, and dimension alignment are suitable .

This multi-level structure is important because it explains where performance gains and bottlenecks actually arise. A TensorRT engine does not simply "run convolutions faster." Instead, performance emerges from an interaction between graph rewrites, ker-

nel selection, tensor layout choices, workspace allocation, and reduced-precision execution. For compute-dominated layers such as large convolutions and matrix multiplications, TensorRT attempts to select implementations that use Tensor Cores through libraries such as cuDNN and cuBLASLt, or through internally specialized kernels. For convolution in particular, many implementations reduce the operation to an implicit GEMM-like form, which allows the same blocked matrix multiplication machinery to be reused for deep learning workloads [**nvidia˙cudnn˙frontend**, **cutlass˙overview**, **cutlass˙implicit˙gemm**, **yan2020tensorcores**, 20]. This is one reason why matrix multiplication is the right mental model for understanding a large fraction of modern inference kernels: convolution, attention, and linear layers are often executed through GEMM-style tiled pipelines rather than through direct textbook formulations.

Accordingly, the effective kernel stack beneath TensorRT can be summarized as follows. At the graph level, the model arrives as ONNX and is parsed into a TensorRT network. At the engine-building level, TensorRT partitions the graph into supported regions, fuses eligible operations, and benchmarks tactics. At the implementation level, these tactics are commonly backed by cuDNN, cuBLASLt, Tensor Core matrix-multiply pipelines, and, for unsupported or intentionally replaced regions, user-defined plugins. At runtime, the execution context launches the selected kernels on CUDA streams using the precompiled execution plan, without performing a second round of graph optimization [**nvidia˙tensorrt˙plugins**, **nvidia˙tensorrt˙engine˙tools**, **nvidia˙tensorrt˙cmd**, **nvidia˙cudnn˙frontend**, 38]. This layered view is especially useful in a compiler-aware thesis because it clarifies that TensorRT's optimization problem spans both graph compilation and kernel dispatch, but does not expose the full schedule space to the user in the way that TVM does.

TensorRT's performance advantage is closely tied to NVIDIA hardware features, especially Tensor Cores and mixed-precision arithmetic. Tensor Cores provide very high throughput for matrix multiply-accumulate operations, but only when shapes, data layouts, and operand types satisfy hardware-specific constraints. Independent microbenchmarking work has shown that Tensor Core performance depends strongly on operand tiling, instruction form, and memory staging, meaning that nominal precision support alone is not sufficient to guarantee high realized throughput [**yan2020tensorcores**, **sun2022dissecting**, 31, 20]. For inference, this is highly relevant: the theoretical gain from FP16, BF16, TF32, or INT8 is achieved only when the surrounding kernel implementation can sustain data movement and feed the compute units efficiently. TensorRT's tactic-selection mechanism exists precisely to navigate this design space on a concrete target device [**nvidia˙tensorrt˙support˙mat**, 39].

An important practical issue is operator support. TensorRT does not execute arbitrary ONNX graphs without restriction. Instead, the ONNX parser supports a large but

incomplete subset of ONNX, and unsupported operators must either be rewritten, decomposed into supported subgraphs, or implemented as plugins [**nvidia˙tensorrt˙quickstart**, **nvidia˙tensorrt˙support˙matrix**]. This means that deployment success depends not only on the neural network architecture, but also on how the model is exported. Two ONNX graphs with equivalent mathematical meaning can lead to different TensorRT outcomes if one export pattern matches TensorRT-supported operators and fusion rules more closely than the other. This is particularly important in research codebases, where novel blocks are often expressed as long sequences of primitive operators that are semantically simple but structurally unfriendly to vendor parsers and fusion passes.

For this reason, TensorRT's plugin interface is a central mechanism for controlled intervention. In current TensorRT releases, custom layers are implemented through the `IPluginV3` interface, while earlier plugin APIs are deprecated [**nvidia˙tensorrt˙plugins˙cpp**, **nvidia˙tensorrt˙plugin˙migration**]. A plugin declares its inputs, outputs, formats, data types, shape behavior, and runtime implementation, and is then inserted into the network either programmatically or through ONNX parser integration. In the common ONNX-based workflow, the user first performs subgraph surgery, typically with ONNX GraphSurgeon, replacing a region of the exported graph with a custom node whose operator name matches the registered plugin creator. TensorRT then parses that node as a plugin layer and includes it as a first-class component during engine building [**nvidia˙tensorrt˙plugins**, **nvidia˙tensorrt˙plugin˙migration**, **nvidia˙onnx˙graphsurgeon**, **nvidia˙tensorrt˙plugin˙advanced**]. This is the cleanest route for injecting a hand-written CUDA implementation into an otherwise TensorRT-optimized model.

Conceptually, plugin injection can occur in three stages. First, a subgraph is identified whose default lowering is unsupported or inefficient. Second, that region is replaced in the ONNX graph by a plugin node carrying the required metadata or serialized weights. Third, the plugin shared library is loaded during engine build and runtime so that TensorRT can benchmark the plugin against surrounding layers and embed the plugin into the final engine plan [**nvidia˙tensorrt˙plugins**, **nvidia˙tensorrt˙plugin˙migration**, **nvidia˙tensorrt˙cmd**]. This mechanism is narrow compared with a general compiler IR transformation framework, but it is highly valuable in practice because it permits targeted replacement of persistent bottlenecks without abandoning the rest of TensorRT's optimization pipeline.

The plugin path also makes the kernel stack visible in a way that the standard TensorRT workflow often does not. When a model is executed entirely through built-in TensorRT support, the concrete backend kernels are mostly opaque. By contrast, a custom plugin forces the developer to specify the exact computational strategy: whether the operation is implemented as a direct CUDA kernel, as a CUTLASS-based implicit GEMM pipeline, as a cuBLASLt matrix multiplication with fused epilogues, or through

another low-level approach. This matters for research because these choices correspond to different trade-offs in arithmetic intensity, shared-memory usage, tensor layout, and fusion opportunities. In other words, plugins are not only an extensibility mechanism; they are an experimental interface for studying what TensorRT leaves on the table.

TensorRT does provide limited introspection tools that are useful for this purpose. The engine inspector and `trtexec` can expose layer information, per-layer runtimes, and selected engine structure via flags such as `--profilingVerbosity=detailed`, `--dumpLayerInfo`, and `--dumpProfile` [**nvidia˙tensorrt˙engine˙tools**, **nvidia˙tensorrt˙cmd**]. While these tools do not reveal full internal scheduling logic, they are sufficient to identify whether overhead is dominated by specific kernels, reformat operations, shape handling, or data transfers. Combined with Nsight Systems and Nsight Compute, they allow the researcher to bridge the gap between graph-level descriptions and actual kernel execution on the GPU.

Compared with Apache TVM, TensorRT offers a narrower but more mature optimization surface. TVM exposes intermediate representations, schedule search, tensorization, and generated code, making it suitable for research on compilation and code generation across different hardware targets [5, 66]. TensorRT instead prioritizes deployment effectiveness on one hardware family, relying on proprietary tactic libraries, empirically tuned heuristics, and vendor-maintained kernels. Compared with ONNX Runtime, TensorRT is less general but typically more aggressive on supported NVIDIA targets: ONNX Runtime organizes acceleration through execution providers, including both CUDA EP and TensorRT EP, and therefore emphasizes portability and broad operator coverage over deep specialization to a single kernel ecosystem [**onnxruntime˙docs**, **onnxruntime˙cuda˙ep**, **onnxruntime˙trt˙ep**]. This distinction is analytically important: ONNX Runtime is primarily a runtime abstraction layer over multiple backends, whereas TensorRT is itself a backend compiler and engine builder.

TensorRT also has important limitations. First, unsupported operators, dynamic control flow, and export idiosyncrasies can prevent full graph lowering or degrade fusion quality. Second, the engine is strongly tied to the target platform and should be treated as hardware- and software-specific rather than portable [**nvidia˙tensorrt˙support˙matrix**, **nvidia˙tensorrt˙glossary**]. Third, the user has only partial visibility into why a particular tactic was chosen, which complicates rigorous compiler analysis. Finally, per-layer or per-region optimization can still leave global inefficiencies unresolved, especially when model performance is limited by reformatting, synchronization, or memory traffic between otherwise efficient kernels. Recent systems work on GPU inference and tensor-program optimization repeatedly shows that kernel-by-kernel execution can remain suboptimal relative to more holistic scheduling and fusion approaches [**rammer2020**, **roller2022**, **mononn2024**]. This observation helps explain why even TensorRT-optimized models

can retain stubborn bottlenecks around custom architectural blocks.

Relation to this thesis. TensorRT is relevant to this thesis in two distinct roles. First, it is the strongest practical baseline for low-latency inference on the Jetson Orin Nano GPU and therefore serves as a reference point for evaluating alternative deployment paths such as ONNX Runtime and Apache TVM. Second, its plugin interface creates a controlled experimental boundary inside an otherwise vendor-optimized stack. By replacing a single YOLOv8n subgraph, such as a C2f block, with a custom plugin, it becomes possible to study not only whether a hand-tuned kernel improves latency, but also where the remaining time is spent: in arithmetic kernels, tensor reformatting, launch overhead, or memory movement between optimized and non-optimized regions. This makes TensorRT especially useful for a compiler-aware investigation of the boundary between graph-level optimization and kernel-level specialization.

2.10 Performance Bottlenecks in ML Inference

TODO

A workload is compute-bound if increasing the raw processing power (e.g., higher clock speed, more compute units) directly leads to a proportional decrease in execution time, assuming memory and latency are not limiting factors.

Inference for complex neural networks requires millions of multiply-accumulate (MAC) operations. If hardware is not fully utilized and code not map vectors/tensors effectively, the processing bottlenecks are created.

Many ML operations are memory-bound. This means the execution time is dominated by the time spent transferring data between memory levels (e.g., DRAM to caches, cache levels, host-to-device memory) rather than the computation itself.

Latency bounded operations: single-batch inference, overhead for dispatching operations, pipeline stalls.

These strategies reduce dispatch overhead for frequently executed execution paths:

- **Ahead-of-Time (AOT):** Compile the entire model before execution, as in traditional C/C++ workflows and many edge deployment scenarios.
- **Just-In-Time (JIT):** Compile code or computation graphs at runtime, typically upon first execution, leveraging runtime information such as tensor shapes, data types, and target hardware.

2.11 Relation to this thesis

TODO

3 Theory

3.1 Metrics Description

To evaluate the performance of object detection models, a set of standard metrics is used throughout this report. These metrics assess both localization accuracy and classification quality, providing a comprehensive view of model behavior.

- **Intersection over Union (IoU)**:
  For a predicted bounding box p and a ground-truth box g, the Intersection over Union is defined as:
  $$\text{IoU}(p, g) = \frac{|p \cap g|}{|p \cup g|}.$$
  IoU measures localization accuracy by quantifying the overlap between predicted and ground-truth boxes. It is computed as the ratio of the intersection area to the union area of the two boxes, resulting in a value between 0 and 1. A score of 1 indicates perfect alignment, while 0 indicates no overlap.

  In practice, an *IoU threshold* (e.g., 0.5) is used to determine whether a prediction is correct. Predictions with IoU $\geq \tau$ are considered true positives (TP), while those below the threshold are treated as false positives (FP). IoU serves as a fundamental building block for higher-level evaluation metrics such as precision, recall, and mAP.

- **Precision and Recall**:
  Precision and recall evaluate the quality of classification and detection results. Precision measures the proportion of correct positive predictions:
  $$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$
  Recall measures the proportion of correctly detected instances among all ground-truth objects:
  $$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$
  Here, TP, FP, TN, and FN denote true positive, false positive, true negative, and false negative predictions, respectively. Precision reflects the ability to avoid false detections, while recall captures the ability to detect all relevant objects.

- **Average Precision (AP)**:
  Average Precision summarizes the precision-recall trade-off by computing the area

under the precision-recall (P–R) curve [10]:

$$\mathrm{AP}_\tau = \int_0^1 p(r;\tau)\,dr.$$

The P–R curve is constructed by varying the detection confidence threshold:

- Each detection is assigned a confidence score $s \in [0, 1]$.
- Detections are sorted in descending order of confidence.
- Predictions are progressively included, and precision and recall are recomputed.
- This process generates the precision-recall curve.

AP corresponds to the area under this curve and provides a single scalar measure of detection performance.

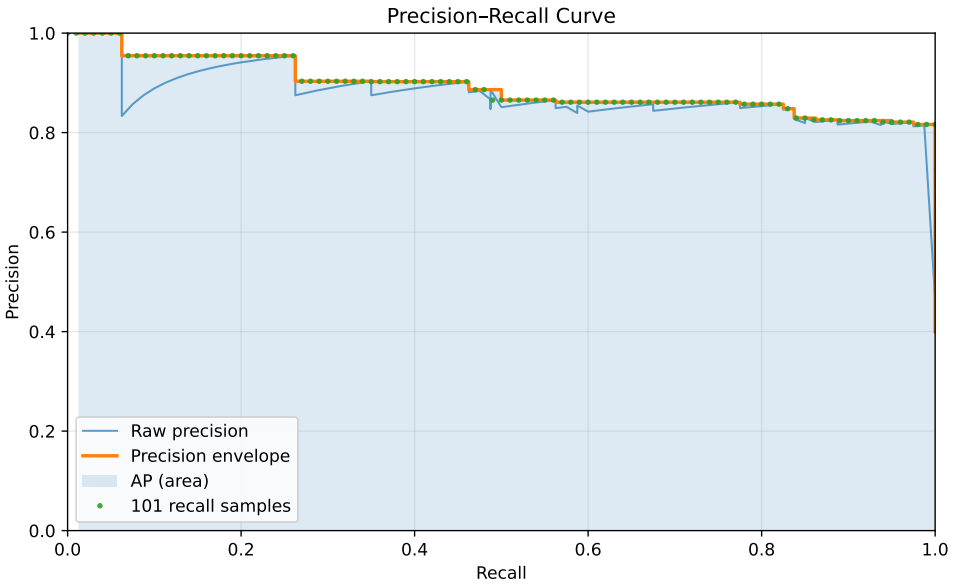

Figure 3.1: Precision–Recall curve.

- **Mean Average Precision (mAP)**:
  Mean Average Precision extends AP to multi-class settings by averaging AP over all object classes:

$$\mathrm{mAP}_{0.50} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{AP}_{c,\,0.50},$$

$$\mathrm{mAP}_{[0.50:0.95]} = \frac{1}{C}\sum_{c=1}^{C}\frac{1}{10}\sum_{k=0}^{9} \mathrm{AP}_{c,\,(0.50+0.05k)}.$$

  Here, C denotes the number of classes. The notation indicates the IoU threshold(s) used to determine true positives. For example, $\mathrm{mAP}_{0.50}$ considers predictions with

IoU ≥ 0.5, while $\text{mAP}_{[0.50:0.95]}$ averages performance across multiple thresholds, providing a more comprehensive evaluation.

3.2 Roofline Model

The Roofline model, introduced by Williams et al. [63], provides an intuitive way to analyze whether a workload is limited by memory bandwidth or computational throughput. It plots performance (e.g., GFLOP/s) versus arithmetic intensity (FLOPs per byte of memory traffic). The model defines two limits: a sloped memory roof determined by peak memory bandwidth, and a flat compute roof determined by peak floating-point throughput. Any kernel's performance is bounded by the minimum of these two ceilings. If a point lies on the sloped region, it is memory-bound; if it lies on the flat region, it is compute-bound. The Roofline model is widely used to analyze whether a workload is limited by memory movement or computation and to guide optimization efforts.

Optimization strategy of a neural network model is dependent on whether our device limits us in the speed of reading/writing to/from memory or the computational speed overall.

- **If the workload is Memory-Bound:**
  This means performance is limited by memory bandwidth, not by compute units. The arithmetic intensity (FLOPs/byte) is low, so the kernel spends most of its time moving data. The goal is then to either increase arithmetic intensity, or reduce memory traffic.
  Optimization strategies:
  - Improve data locality: reuse data in cashes (tiling/blocking); Keep tensors in shared memory (GPU);
  - Fuse operations to avoid writing intermediate tensors to memory (Conv $\rightarrow$ BatchNorm $\rightarrow$ Silu) $\rightarrow$ (Fused Conv-BN-Activation)
  - Reduce Precision (FP32 $\rightarrow$ FP16 $\rightarrow$ INT8 $\rightarrow$ INT4)
  - Reduce tensor size: pruning, structured sparsity, channel reduction, smaller input resolution
  - Improve memory layout: avoid unnecessary copies, channel-last format (NHWC)
- **If the workload is Compute-Bound:**
  This means arithmetic units are saturated. Arithmetic intensity is high enough that bandwidth is no longer limiting. The goal is then to increase compute efficiency. This case is harder, as it requires the work with scheduling, parallelization, kernel

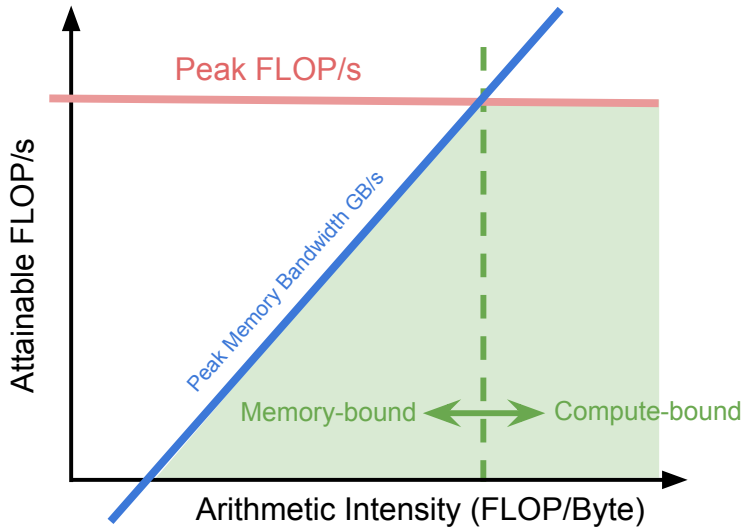

Figure 3.2: A Roofline model illustrating performance limits based on compute capability (horizontal line) and memory bandwidth (sloped line) versus the arithmetic intensity of operations (dots). Operations below the "roof" are limited by either memory bandwidth or compute peak.

optimization and algorithmic improvements.

- **Peak computational performance (in FLOP/s)** is obtained by benchmarking a compute-bound kernel that saturates the floating-point execution units. In practice, this can be achieved using large dense matrix multiplications of size $m \times k$ and $k \times n$, for which the floating-point operation count is given by:

$$\text{FLOPs} = 2 \times m \times k \times n. \tag{3.1}$$

  The sustained peak performance is then computed as

$$P_{\text{peak}} = \frac{\text{FLOPs}}{t_{\text{compute}}}, \tag{3.2}$$

  where t_{compute} denotes the measured execution time.

- **Peak memory bandwidth (in GB/s)** is determined using a streaming memory benchmark. A large device-to-device copy or STREAM-like triad kernel is executed over a sufficiently large buffer to avoid cache effects. The sustained memory

bandwidth is computed as

$$B_{\text{peak}} = \frac{\text{Bytes transferred}}{t_{\text{memory}}}. \tag{3.3}$$

- **Arithmetic intensity (AI)** is defined as the ratio between the number of floating-point operations and the volume of data transferred to and from main memory:

$$\text{AI} = \frac{\text{FLOPs}}{\text{Bytes}}. \tag{3.4}$$

Since exact DRAM traffic is generally unavailable without hardware performance counters, a lower-bound estimate of memory traffic is commonly used. For convolutional or linear layers, this lower bound consists of the input tensor, weight parameters, and output tensor:

$$\text{Bytes}_{\text{LB}} = \text{Bytes}_{\text{input}} + \text{Bytes}_{\text{weights}} + \text{Bytes}_{\text{output}}. \tag{3.5}$$

This estimate ignores cache reuse and intermediate buffering, thereby providing an upper bound on arithmetic intensity.

- **The achieved performance of a workload** is computed as

$$P_{\text{achieved}} = \frac{\text{FLOPs}}{t_{\text{execution}}}. \tag{3.6}$$

The Roofline bound is then defined as

$$P_{\text{roof}}(\text{AI}) = \min\left(P_{\text{peak}},\ B_{\text{peak}} \times \text{AI}\right). \tag{3.7}$$

If P_{achieved} approaches $B_{\text{peak}} \times \text{AI}$, the workload is memory-bound, if it approaches P_{peak}, the workload is compute-bound. This methodology enables systematic classification of performance bottlenecks and guides optimization strategies such as improving data locality, increasing arithmetic intensity, or enhancing computational parallelism.

3.3 Activation Functions: ReLU and SiLU

Activation functions play a critical role in introducing non-linearity into neural networks, enabling them to learn complex representations. In this work, we consider two commonly used activation functions: Rectified Linear Unit (ReLU) and Sigmoid Linear Unit (SiLU).

The ReLU activation function was introduced by [36] and is widely used due to its computational simplicity. More recently, smooth activation functions such as SiLU (or Swish) [52] have been proposed, offering improved accuracy at the cost of increased computational complexity.

The ReLU activation is defined as:

$$\mathrm{ReLU}(x) = \max(0, x) \tag{3.8}$$

ReLU is computationally simple, requiring only a comparison and selection operation. This makes it highly efficient on modern hardware, as it can be implemented with minimal arithmetic and memory overhead. Due to its simplicity, ReLU is widely used in performance-critical applications and is well-supported by hardware accelerators.

In contrast, the SiLU activation (also known as the Swish function) is defined as:

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{3.9}$$

where $\sigma(x)$ denotes the sigmoid function. Compared to ReLU, SiLU introduces additional non-linearity and has been shown to improve model accuracy in many cases. However, it is computationally more expensive, as it requires evaluation of the exponential function, a division, and a multiplication.

From a computational perspective, the key difference between the two functions lies in their complexity. ReLU involves only a simple thresholding operation, which is typically memory-bound and incurs minimal computational cost. In contrast, SiLU requires multiple floating-point operations, including transcendental functions, making it more compute-intensive and potentially slower on hardware with limited support for such operations.

This difference has practical implications for model optimization. Replacing SiLU with ReLU can reduce inference latency and improve hardware efficiency, especially on edge devices. However, this simplification may lead to a degradation in model accuracy. Therefore, the choice of activation function represents a trade-off between computational efficiency and predictive performance.

While smooth activation functions such as SiLU have been shown to improve accuracy, recent work suggests that simpler functions like ReLU can offer advantages in terms of computational efficiency and activation sparsity [33]. This sparsity can be exploited by hardware and compilers to reduce the number of effective computations, leading to improved inference performance.

4 Methodology

This chapter describes the methodology used to evaluate and optimize deep learning inference on edge-oriented hardware platforms. The study is centered on the YOLOv8n object detection model and investigates how compiler-based optimization frameworks, numerical precision reduction, structural model simplification, and low-level kernel customization affect inference performance and detection accuracy.

The methodology is organized as a sequence of experimental stages. First, a baseline is established using the original FP32 PyTorch implementation of YOLOv8n. Next, bottlenecks are analyzed using hardware-aware performance models and profiling tools. Based on the observed limitations, several optimization directions are explored, including framework-level compilation, quantization, precision reduction, pruning, and custom TensorRT plugin development. Each optimization is evaluated with respect to latency, throughput, and accuracy in order to characterize the trade-offs between performance and model quality.

4.1 Model Selection

YOLOv8n is selected as the reference model for this study. The model represents a practical and relevant target for edge deployment because it combines a compact architecture with competitive detection accuracy. It is widely used in real-world object detection tasks, is actively supported by the research and engineering community, and can be exported to multiple deployment backends. These properties make it suitable for comparative analysis across inference frameworks.

From a methodological perspective, YOLOv8n is large enough to expose non-trivial optimization challenges, including operator fusion, memory movement overhead, kernel launch inefficiencies, and sensitivity to reduced precision. At the same time, it remains sufficiently lightweight to allow repeated experimentation, retraining, export, and validation on commonly available development hardware. This enables a controlled and reproducible study of both high-level and low-level optimization strategies.

4.2 Overall Experimental Procedure

The experimental workflow follows five main stages:

1. Establish a baseline using the original FP32 PyTorch model.
2. Identify performance bottlenecks through roofline analysis and runtime profiling.
3. Apply optimization techniques at the framework, numerical, and architectural levels.
4. Evaluate the optimized models on target hardware using identical benchmarking conditions.
5. Compare the resulting variants in terms of latency, throughput, and accuracy.

This procedure ensures that each optimization step is assessed relative to a common reference and that conclusions are based on measurable changes in both performance and predictive quality.

4.3 Baseline Establishment

The starting point of the study is the pretrained FP32 YOLOv8n model provided by Ultralytics and executed in the original PyTorch runtime. This baseline serves two purposes. First, it provides a reference for all later comparisons. Second, it makes it possible to determine whether the observed performance is already close to the limits of the target hardware or whether substantial optimization opportunities remain.

The baseline is evaluated on the selected hardware platforms under controlled conditions. For each platform, inference latency and throughput are measured using repeated runs with warm-up iterations to reduce initialization effects. Accuracy is evaluated on the validation dataset using standard object detection metrics. These baseline results form the reference against which all optimized variants are compared.

4.4 Bottleneck Analysis

After establishing the baseline, the next step is to determine the dominant performance limitations of the model on each platform. Two complementary approaches are used for this purpose: roofline-based reasoning and runtime profiling.

The roofline model is used as a coarse analytical framework to determine whether performance is likely to be limited by memory bandwidth or by available compute throughput. This provides a hardware-aware interpretation of the measured performance and helps identify whether the main limitation arises from insufficient arithmetic intensity, inefficient memory access, or underutilization of the compute units.

To refine this analysis, profiling tools are used to inspect the execution of individual operators and kernels. This makes it possible to identify layers with high latency, excessive memory movement, frequent kernel launches, or inefficient execution patterns. In particular, the profiling stage is used to answer the following questions:

- Which layers dominate end-to-end inference time?
- Whether the execution is primarily memory-bound or compute-bound.
- Whether the framework introduces avoidable overhead through layout conversions, intermediate buffers, or host-device synchronization.
- Whether operator fusion and backend-specific optimizations are effectively applied.

The outcome of this stage determines which optimization direction is most relevant for a given backend and hardware target.

4.5 Optimization Directions

Based on the bottleneck analysis, the study investigates several classes of optimization methods.

4.5.1 Compiler and Runtime Optimization

The first optimization direction consists of evaluating how different inference frameworks transform and execute the same model. Starting from the original PyTorch implementation, the model is exported or compiled for alternative backends such as TensorRT and TVM. The purpose of this comparison is to assess how much performance can be gained from compiler-level graph transformations, backend-specific kernel selection, operator fusion, and scheduling optimizations.

This stage is not limited to reporting end-to-end latency. It also examines how the internal execution differs between backends, for example in the number of kernels launched, the use of fused operators, the presence of data layout transformations, and the distribution of execution time across layers. In this way, the comparison provides insight not only into which backend is faster, but also into why it is faster.

4.5.2 Numerical Precision Reduction

The second optimization direction focuses on reducing numerical precision in order to decrease memory traffic and accelerate arithmetic operations. Two forms of precision

reduction are considered in this work.

First, inference in FP16 is evaluated against the FP32 baseline. This comparison isolates the effect of reduced floating-point precision and shows whether the target hardware benefits from half-precision execution.

Second, quantization to INT8 is explored where supported by the backend. Quantization has the potential to reduce model size, memory bandwidth requirements, and arithmetic cost, but it may also introduce accuracy degradation. Therefore, each quantized model is evaluated not only in terms of runtime performance but also in terms of detection quality. The purpose of this stage is to determine whether the performance gains justify the loss, if any, in predictive accuracy.

4.5.3 Model Compression and Structural Simplification

The third optimization direction targets the structure of the model itself. Rather than relying only on framework-level improvements, this part of the study investigates whether the network can be simplified in ways that improve runtime characteristics.

Two approaches are considered. The first is pruning, which reduces model complexity by removing less important parameters or channels. The second is replacement of computationally expensive operations with simpler alternatives. In particular, the effect of replacing SiLU activations with ReLU is evaluated. This substitution is motivated by the fact that simpler activations may improve fusion opportunities and reduce kernel complexity, even if they slightly affect model accuracy.

The purpose of this stage is to understand whether modest architectural simplifications can produce a favorable trade-off between accuracy and efficiency, especially on hardware with limited compute capability or reduced support for complex fused kernels.

4.5.4 Low-Level TensorRT Customization

The final optimization direction extends beyond standard framework usage and examines low-level customization within TensorRT. In this stage, custom plugins are developed to replace selected subgraphs with tailored implementations. The main case considered in this work is a custom plugin for the C2f block.

The objective of this step is to investigate whether performance can be improved further by reducing backend overhead, controlling kernel behavior more explicitly, or eliminating inefficient execution patterns that are not resolved by the default TensorRT conversion pipeline. This stage complements the framework-level comparison by showing the extent to which manual low-level intervention can outperform standard automated

optimization.

4.6 Evaluation Criteria

All model variants are evaluated using the same set of criteria in order to ensure consistent comparison.

The primary performance metrics are inference latency and throughput. Latency is used to measure the response time of a single inference pass, while throughput indicates the sustained processing rate under repeated execution. Where relevant, profiling data is also used to analyze kernel-level execution time and the contribution of individual layers to the total runtime.

The primary quality metric is object detection accuracy on the validation dataset. Standard detection metrics are used to verify whether a given optimization preserves acceptable predictive performance. This is particularly important for quantization, pruning, and activation replacement, where improvements in runtime may come at the cost of degraded detection quality.

The final analysis therefore considers both efficiency and accuracy, with emphasis on the trade-off between them. An optimization is not treated as beneficial solely because it accelerates execution; it must also maintain sufficient model quality for practical deployment.

4.7 Methodological Summary

The methodology combines baseline benchmarking, bottleneck analysis, framework comparison, numerical optimization, structural simplification, and low-level backend customization. The goal is not only to identify the fastest deployment configuration for YOLOv8n, but also to understand which classes of optimization are most effective on edge-oriented hardware and under which conditions they provide measurable benefit.

Figure 4.1 summarizes the scope of the study and the main experimental directions considered in this thesis.

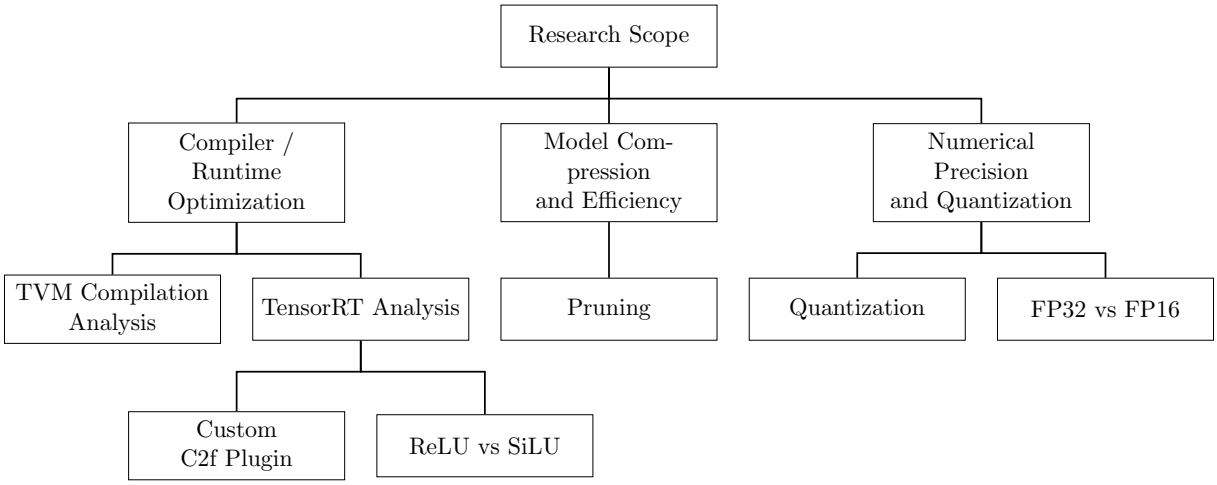

Figure 4.1: Overview of the thesis scope and main experimental directions.

5 Testing and Results

This chapter is currently in draft form and requires further structuring, formatting, and completion of the subsections: c2f-plugin, relu_silu activation functions substitution and prunning. Its final revision is planned for the concluding phase of the thesis work.

5.1 Experimental Setup

The experimental evaluation is conducted on two edge-oriented target platforms: the NVIDIA Jetson Orin Nano and the Raspberry Pi 5. These devices represent two different deployment scenarios for embedded inference: a GPU-accelerated AI platform with dedicated support for mixed-precision computation, and a general-purpose low-power ARM platform without a dedicated on-chip neural network accelerator.

Model training, export, and selected modification experiments, such as pruning and operator substitution, are performed on a host workstation equipped with an NVIDIA GeForce RTX 3070 GPU. This system is used as the primary development and experimentation environment, while all deployment-oriented inference benchmarks are executed on the target edge devices.

The Jetson Orin Nano is designed for edge AI and robotics workloads. Its system-on-chip integrates a 6-core Arm CPU, an Ampere-based GPU, and shared LPDDR5 memory accessible by both CPU and GPU. This architecture is particularly relevant for this study because it supports CUDA execution, Tensor Cores, mixed-precision inference, and INT8 acceleration, making it suitable for evaluating framework-level and low-level optimization strategies.

In contrast, the Raspberry Pi 5 represents a resource-constrained edge platform without a comparable integrated AI accelerator. It is therefore used to assess how optimization techniques behave on a more conventional embedded CPU-based system, where memory bandwidth and general-purpose compute resources are more limited.

Only the hardware characteristics most relevant to performance analysis are reported in this section, namely compute capability, memory capacity, and memory bandwidth.

Table 5.1: Host GPU specifications used for training and model development

Property	**Value**
Device	NVIDIA GeForce RTX 3070
Architecture	Ampere
FP32 Performance	∼20 TFLOPs
Memory	8 GB GDDR6
Memory Bandwidth	∼448 GB/s

Table 5.2: Specifications of the target edge devices

Property	**Jetson Orin Nano**	**Raspberry Pi 5**
CPU	6-core Arm Cortex-A78AE	4-core Arm Cortex-A76
GPU	Ampere (1024 CUDA cores)	VideoCore VII
FP32 Performance	∼0.5–1 TFLOPs (GPU)	—
Memory	8 GB LPDDR5	8 GB LPDDR4X
Memory Bandwidth	∼102 GB/s	∼17 GB/s
Accelerators	Tensor Cores	None*

* The Raspberry Pi 5 can be extended with external AI accelerators, such as the Hailo-8, via PCIe. However, such accelerators are not used in the experiments reported in this thesis.

5.2 Roofline Model

Constructing an accurate Roofline model for real-world workloads presents several challenges. First, the performance characteristics reported by hardware manufacturers often differ from measured values under practical conditions. Second, the effective behavior of a model depends on the execution framework, including factors such as cache utilization, memory reuse, and implicit optimizations (e.g., operator fusion), which are typically not visible at the model level.

Official YOLOv8 model characteristics are summarized in Table 5.3. The smallest variant, YOLOv8n, contains 3.2M parameters and requires 8.7B floating-point operations for a single forward pass at 640×640 resolution.

While vendor specifications indicate that the Raspberry Pi 5 provides up to 17 GB/s of memory bandwidth [54], and the Jetson Orin Nano offers up to 102 GB/s [37], measured performance is often significantly lower in practice.

Table 5.3: Comparison of YOLO models (trained on COCO) at 640×640 resolution [60].

Model	CPU ONNX (ms)	A100 TensorRT (ms)	Params (M)	FLOPs (B)
v8n	80.4	0.99	3.2	8.7
v8s	128.4	1.20	11.2	28.6
v8m	234.7	1.83	25.9	78.9
v8l	375.2	2.39	43.7	165.2
v8x	479.1	3.53	68.2	257.8

5.2.1 Roofline Analysis on Raspberry Pi 5

Using the methodology described in Chapter 3, the sustained memory bandwidth of the Raspberry Pi 5 was measured at 5.67 GB/s, and peak compute throughput at 73.95 GFLOP/s. All measurements were obtained with PyTorch configured to use 4 CPU threads, and layer timings were reported as the median over 20 runs after warmup.

These values define the device Roofline:

$$P(I) = \min\left(73.95,\ 5.67 \cdot I\right),$$

with a ridge point at approximately 13.04 FLOPs/byte, separating memory-bound and compute-bound regimes.

Table 5.4 summarizes representative per-layer performance characteristics of YOLOv8n executed on the Raspberry Pi 5 CPU using eager-mode PyTorch.

Layers with relatively low arithmetic intensity (AI $\approx$ 7–10 FLOPs/byte) achieve only modest throughput, typically in the range of 4.8–8.0 GFLOP/s, indicating memory-bound or memory-sensitive behavior under the measured Roofline parameters. In contrast, deeper convolutional layers with higher arithmetic intensity (AI $>$ 70 FLOPs/byte) achieve substantially higher effective throughput, often in the range of 40–60 GFLOP/s. Some layers with very high arithmetic intensity exceed 50 GFLOP/s, showing that smaller deep feature maps benefit from improved locality and higher compute utilization.

The distribution focal loss (DFL) convolution in the detection head has extremely low arithmetic intensity (0.47 FLOPs/byte) and achieves only 0.6 GFLOP/s, confirming strong memory limitation.

Although several individual convolutional layers exhibit relatively high effective throughput, the overall model achieves only 13.54 GFLOP/s, corresponding to approximately 18.3% of the measured device-level peak compute throughput of 73.95 GFLOP/s.

This discrepancy indicates that end-to-end inference performance is not determined solely by the efficiency of individual convolution kernels. Instead, it is also affected by

Table 5.4: Per-layer performance statistics of YOLOv8n on Raspberry Pi 5 (CPU, FP32).

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
0.conv	18.39	7.71	4.8	Memory-bound
2.cv1.conv	8.84	7.99	5.9	Memory-bound
2.cv2.conv	9.84	9.59	8.0	Memory-bound
4.m.0.cv2.conv	2.74	70.42	43.1	Compute-favored
6.m.1.cv2.conv	1.93	122.03	61.1	Compute-favored
18.m.0.cv2.conv	2.02	122.03	58.5	Compute-favored
22.cv3.0.1.conv	13.53	170.41	54.5	Compute-favored
22.dfl.conv	1.89	0.47	0.6	Strongly memory-bound
Full Model	327.00	157.76	13.54	Mixed / overhead-dominated

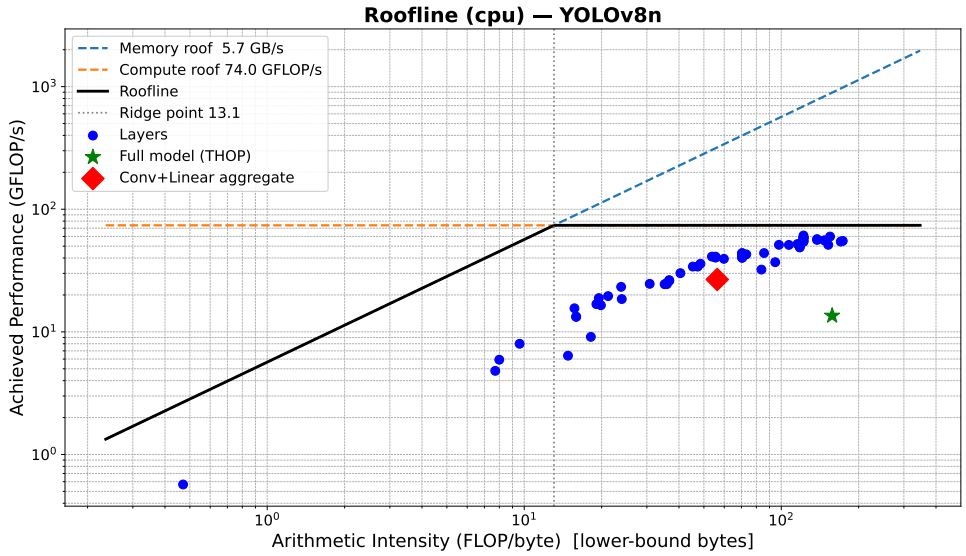

Figure 5.1: Roofline model of YOLOv8n on Raspberry Pi 5 (CPU, FP32).

system-level and framework-level overheads, including kernel dispatch overhead, synchronization, non-convolutional operations, tensor reshaping and concatenation, inter-layer memory traffic, and limited fusion between adjacent operators. Thus, the Roofline serves primarily as an upper bound and an interpretive tool rather than a directly attainable end-to-end target.

From an optimization perspective, these results suggest that further improvements should focus not only on reducing memory traffic, but also on improving execution efficiency at the graph and kernel level. Promising directions include operator fusion, graph-

level optimization, backend-specific kernel tuning, quantization, and reducing framework overhead through more optimized runtimes.

The observed performance behavior can be further explained by the relationship between layer activation sizes and the cache hierarchy:

- L1: 64 KB per core
- L2: 512 KB per core
- L3: 2 MB shared
- DRAM bandwidth: $\approx$ 5.67 GB/s (measured)

In particular, the shared L3 cache of the Raspberry Pi 5 is limited to 2 MB, which constrains the amount of feature-map data that can be reused without returning to main memory.

Early layers of YOLOv8n operate on high-resolution feature maps (e.g., 320×320), resulting in activation tensors that far exceed the available cache capacity. Consequently, these layers experience frequent DRAM accesses and comparatively low effective performance, which is consistent with their lower arithmetic intensity and more memory-bound behavior.

In contrast, deeper layers operate on progressively smaller feature maps (e.g., 40×40 or 20×20), allowing a larger fraction of their working set to remain in cache. This improves locality and data reuse, enabling substantially higher effective throughput. Thus, even though the full model remains far below the ideal Roofline bound, the layerwise results still reveal a clear transition from memory-bound early layers to more compute-favored deeper layers.

5.2.2 Roofline Analysis on Jetson Orin Nano

The same analysis was performed on the NVIDIA Jetson Orin Nano GPU. With PyTorch configured to use a single host thread and using median timings after warmup, the measured sustained memory bandwidth was 55.68 GB/s, while peak compute throughput reached 810.68 GFLOP/s.

The resulting Roofline model is defined as:

$$P(I) = \min\left(810.68,\ 55.68 \cdot I\right),$$

with a ridge point at:

$$I^* = \frac{810.68}{55.68} \approx 14.56 \text{ FLOPs/byte}.$$

Compared to the Raspberry Pi 5, the Orin Nano exhibits both substantially higher compute throughput and much higher memory bandwidth. At the same time, the ridge point remains in a moderate arithmetic-intensity range, which means that many YOLOv8n convolution layers lie to the right of the knee and can benefit from the GPU's high compute capability.

Table 5.5: Representative per-layer performance statistics of YOLOv8n on Jetson Orin Nano (GPU, FP32).

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.0.conv	0.78	7.71	114.1	Memory-bound
model.2.cv1.conv	0.26	7.99	204.2	Memory-bound
model.4.m.0.cv2.conv	0.56	70.42	211.8	Compute-bound
model.6.m.1.cv2.conv	0.23	122.03	502.4	Compute-bound
model.12.m.0.cv2.conv	0.24	122.03	499.3	Compute-bound
model.22.cv2.0.1.conv	0.56	137.80	842.6	Compute-bound
model.22.cv3.0.1.conv	0.83	170.41	892.5	Compute-bound
model.22.dfl.conv	0.27	0.47	3.9	Strongly memory-bound
Full Model	26.02	157.76	170.22	Mixed / Overhead-dominated

The full model achieves 170.22 GFLOP/s, corresponding to approximately 21.0% of the measured peak compute throughput. This is substantially higher than on the Raspberry Pi 5, but still far below the theoretical device peak.

Per-layer analysis shows that many convolutional layers achieve several hundred GFLOP/s, with some layers approaching or even exceeding 800 GFLOP/s. In particular, layers with high arithmetic intensity (AI $>$ 70 FLOPs/byte) clearly operate in the compute-bound regime and make effective use of the GPU's compute resources.

However, low-intensity layers remain limited by memory behavior. The DFL convolution in the detection head, with AI $\approx$ 0.47 FLOPs/byte, achieves only 3.9 GFLOP/s and remains strongly memory-bound even on this more capable platform.

Table 5.5 highlights that, compared to the Raspberry Pi 5, the majority of convolutional layers on the Orin Nano operate in the compute-bound regime and achieve

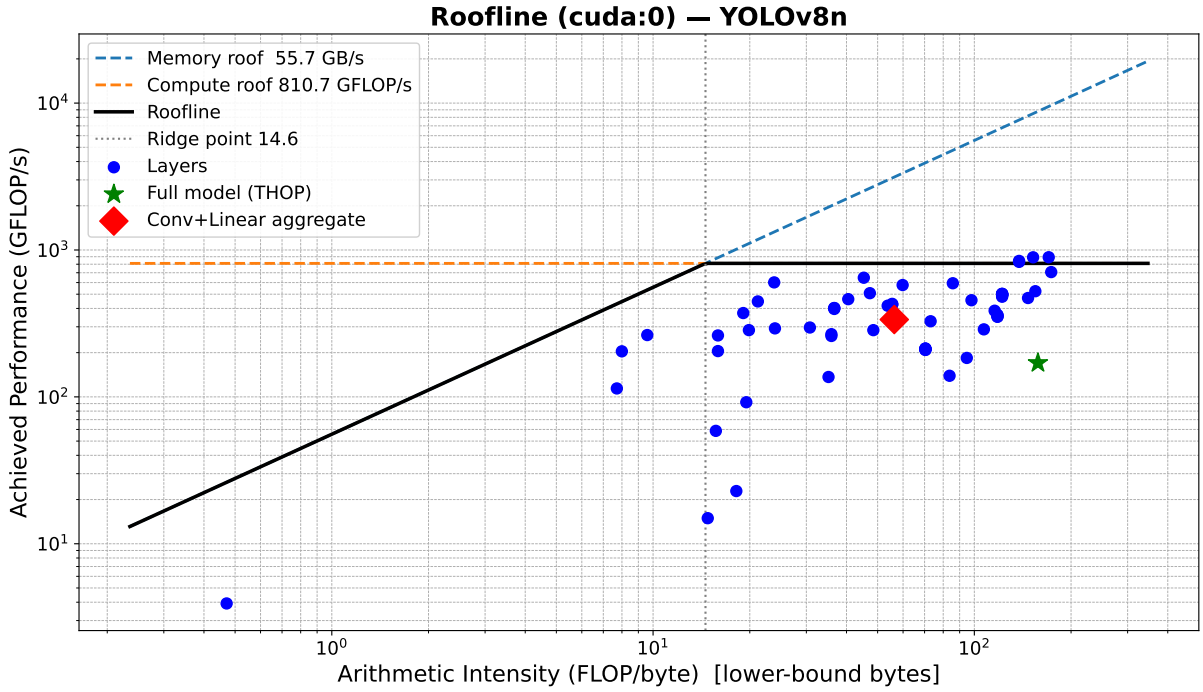

Figure 5.2: Roofline model of YOLOv8n on Jetson Orin Nano (GPU, FP32).

substantially higher throughput due to the GPU's much larger compute capacity and memory bandwidth.

At the same time, the results also show that high per-layer kernel efficiency does not directly translate into equally high end-to-end model efficiency. Although several layers approach the measured peak throughput, the overall model reaches only 170.22 GFLOP/s. This indicates the continued presence of system-level overhead, including kernel launch overhead, intermediate tensor movement, non-convolutional operations, and synchronization costs between layers.

Thus, the Orin Nano provides a much more favorable platform for YOLOv8n inference than the Raspberry Pi 5, but the Roofline analysis still reveals that end-to-end performance remains constrained by factors beyond raw compute throughput alone.

Detailed per-layer benchmarking results are provided in Appendix A.

5.3 Performance Comparison

We evaluated the performance of YOLOv8n models on two hardware platforms: the NVIDIA Orin Nano and the Raspberry Pi 5. Following this evaluation, we investigated different model compilation approaches tailored to each target platform.

Due to substantial differences in hardware architectures, the set of available inference frameworks also differs. Hardware-specific frameworks such as TensorRT are limited to

NVIDIA GPUs and therefore cannot be used for a direct cross-platform comparison. Instead, we assess the performance improvements provided by each framework relative to a common baseline on the same hardware.

The baseline execution is performed in PyTorch eager mode. In this mode, operations are executed immediately as Python code is interpreted line by line. The computational graph is constructed dynamically at runtime, which makes debugging straightforward and enables inspection of intermediate results, such as per-layer performance metrics presented in the previous section. However, eager execution introduces several limitations: Python-level overhead for each operation, lack of global graph-level optimization, and limited hardware-specific tuning. As a result, the model is effectively executed as a sequence of independent operators rather than as an optimized computational graph.

To address these limitations, the model can be treated as a static graph and optimized using compiler-based frameworks. A variety of such compilers have been developed to enable hardware-aware optimizations, including operator fusion, memory planning, and kernel selection. In this section, we evaluate these approaches on the target hardware platforms.

A standard step in this workflow is converting the model to the ONNX (Open Neural Network Exchange) format. In this work, we use ONNX export with opset version 17 and enable graph simplification. The opset defines the set of available operators (e.g., Conv, Add, Resize) and their semantics. Choosing an appropriate opset version is important: newer opsets may not be fully supported by all backends, while older versions may lack required operators or optimizations. The opset selection can therefore impact operator decomposition, kernel count, memory traffic, and opportunities for fusion.

Graph simplification is applied to reduce computational redundancy and streamline the graph structure. This includes:

- Constant folding: precomputing constant expressions (e.g., $x + (2 \times 3) \rightarrow x + 6$)
- Node elimination: removing redundant operations such as identity mappings
- Subgraph simplification: collapsing sequences of operations into simpler equivalents when possible
- Dead code elimination: removing unused branches of the graph

While these transformations simplify the graph, backend-specific compilers (e.g., TensorRT, TVM) may perform additional, more aggressive optimizations such as kernel fusion and layout transformations.

Once converted, the ONNX model can be executed using a variety of inference frameworks. On CPU-based platforms, commonly used frameworks include PyTorch (via Torch-

Script or TorchInductor), ONNX Runtime, and TensorFlow Lite. These frameworks provide different levels of optimization, ranging from general-purpose execution to lightweight deployment on resource-constrained devices.

On GPU-based platforms, available options include PyTorch with CUDA execution, ONNX Runtime with GPU or TensorRT execution providers, and TensorRT as a dedicated compiler and runtime for NVIDIA hardware. TensorRT performs aggressive graph-level and kernel-level optimizations, such as operator fusion, precision calibration (e.g., FP16/INT8), and memory layout transformations, making it particularly effective for high-performance inference on NVIDIA devices.

Another notable framework that supports both CPU and GPU targets is Apache TVM. Unlike runtime-focused frameworks, TVM operates as a deep learning compiler stack that performs end-to-end optimization, including graph transformations, operator scheduling, and hardware-specific code generation. This approach provides a high degree of flexibility and control over low-level execution compared to more API-driven frameworks, although it typically requires more manual tuning and expertise.

5.3.1 TVM MetaSchedule Tuning

Unlike the other evaluated deployment frameworks, TVM does not rely only on a fixed set of pre-implemented backend optimisations. Instead, it exposes the execution schedule itself as an optimisation problem and searches for hardware-specific implementations of individual operators. In practice, this means that TVM requires an additional tuning stage before final compilation, in which candidate schedules are generated, measured on the target device, and ranked according to their runtime performance. This makes TVM fundamentally different from frameworks such as ONNX Runtime, LiteRT, or PyTorch eager mode, which mainly depend on predefined kernels and runtime-level optimisations.

For this reason, the TVM results are discussed separately from the general framework comparison. The purpose of the following subsection is not only to report the final benchmark numbers, but also to analyse how MetaSchedule improved the execution of YOLOv8n, which operators benefited most from tuning, and where the main remaining bottlenecks are. This additional discussion is important because, in the case of TVM, the observed inference performance is directly shaped by the quality of the schedule search rather than by the default behaviour of a fixed runtime backend.

5.3.2 TVM MetaSchedule Tuning Results on Raspberry Pi 5 - FP32

TVM MetaSchedule was evaluated on the Raspberry Pi 5 (4-core Cortex-A76, AArch64, 2.4 GHz) using the full YOLOv8n inference graph imported through the Relax frontend. The tuning run lasted approximately 6 hours and 15 minutes, during which TVM explored 7 221 valid schedules across 65 extracted operator tasks. The resulting tuned model achieved a summed task latency of 173.6 ms, while the measured end-to-end inference latency was approximately 185–188 ms. The difference can be attributed to runtime overhead outside the tuned kernels, including Python execution, input preparation, and postprocessing in the detection head.

General Tuning Behaviour. The tuning results show that TVM was able to improve the execution of the convolution-heavy parts of the network through schedule search and cache-aware tiling. However, no task in this run could be tensorised into a dedicated ARM intrinsic pattern. In practice, this means that the observed performance gains came mainly from improved loop ordering, tiling, and LLVM-driven vectorisation, rather than from explicitly tensorised micro-kernel generation. This is an important observation because it highlights both the strength and the limitations of the current FP32 compilation path on Raspberry Pi 5: TVM can still produce competitive schedules, but it does not yet exploit the hardware as fully as a tensorised implementation would.

Best-Performing Operators. The highest efficiency was achieved by 1×1 convolutions, particularly in the neck and detection head. These layers are structurally simpler than 3×3 convolutions and map more naturally to vectorised execution, which allowed them to reach throughput above 80 GFLOPS in the best cases. Several mid-network 3×3 convolutions also performed well once the channel dimensions became sufficiently large. For such layers, the tuned schedules were able to keep the working set in cache and reduce memory traffic, which is especially important on a bandwidth-constrained CPU platform.

Main Bottlenecks. The weakest-performing operator was the initial stem convolution. Although its FLOP count is relatively modest, it operates on a very large spatial input and only three input channels, resulting in poor arithmetic intensity and limited opportunity for data reuse. Consequently, it consumed a disproportionately large share of inference time. Other early-stage 3×3 convolutions with low channel counts and large feature maps showed similar behaviour. In addition, the softmax operation in the detection head remained highly inefficient, not because of high computational complexity, but because it is dominated by repeated memory accesses and reductions rather than dense arithmetic.

Weighted Bottleneck Analysis. A per-layer view alone does not fully capture the true runtime bottlenecks of YOLOv8n. Some convolution tasks appear multiple times in the graph, especially inside repeated C2f blocks, and therefore contribute much more to total inference time than their individual latency would suggest. In the tuned model, several repeated 3×3 convolutions in the backbone and neck became the dominant contributors to total runtime once their multiplicity was taken into account. This confirms that tuning effort should be focused not only on the slowest individual operators, but also on operators that are executed many times.

Overall Assessment. Overall, the tuned TVM model reached an end-to-end latency of about 192 ms on Raspberry Pi 5, compared with approximately 290.1 ms for PyTorch eager mode under comparable conditions. The improvement appears to come mainly from better cache-aware scheduling of convolution layers, particularly the more expensive 3×3 operators in the middle of the network.

At the same time, the results also highlight the limitations of FP32 tuning on this platform. While MetaSchedule improved many layers, the absence of tensorisation meant that the performance gains remained incremental rather than transformative. The most difficult kernels were the early low-channel convolutions, where memory access patterns dominate and schedule search alone has limited effect. Therefore, the main conclusion is that TVM tuning is beneficial on Raspberry Pi 5, but the achievable gains in FP32 remain bounded. Larger improvements would likely require either a more targeted tuning budget for the most expensive tasks or a reduced-precision compilation path such as FP16.

5.3.3 TVM MetaSchedule Tuning Results on the Jetson Orin Nano GPU - FP32

TVM MetaSchedule was evaluated on the NVIDIA Jetson Orin Nano GPU using the full YOLOv8n inference graph imported through the Relax frontend. TVM explored 7 257 valid schedules across the same 65 extracted operator tasks. The resulting tuned model achieved a summed task latency of 22 920 μs, while the measured end-to-end inference latency was 23.38 ms. The difference between these values was small, indicating that the tuned kernel execution accounted for almost all of the observed runtime.

Compared with the Raspberry Pi 5 CPU result of 173 591 μs summed task latency, the GPU tuning result corresponds to a kernel-level speedup of approximately 7.6×. This confirms that the transition from CPU to GPU substantially accelerated the convolution-dominated parts of the model, even though the graph structure and extracted operator tasks remained unchanged.

General Tuning Behaviour. The tuning results show that TVM was able to improve execution on the GPU by searching over CUDA-specific schedule parameters such as thread-block structure, tiling configuration, and memory movement patterns. However, as in the CPU case, no task in this run could be tensorised into a dedicated intrinsic pattern. In practice, this means that the observed gains came from improved CUDA scheduling rather than from an explicit tensor-core-style mapping. This is an important observation, because it highlights both the strength and the limitation of the current FP32 GPU compilation path: TVM can still generate competitive CUDA schedules, but it does not yet exploit all hardware-specific acceleration paths that may be available on the platform.

Best-Performing Operators. The most efficient GPU kernels were large mid-network 3×3 convolutions, particularly those in repeated C2f blocks and in the neck. Unlike the CPU case, where 1×1 convolutions achieved the highest throughput, the GPU benefited most from larger dense convolutional operators with sufficiently high channel counts to expose substantial parallelism. The best-performing task reached 585.6 GFLOPS, while several other large 3×3 convolutions exceeded 500 GFLOPS. These results indicate that once the working set becomes large enough, the GPU can effectively amortise memory movement and keep a large number of execution units occupied.

Main Bottlenecks. The weakest-performing convolution remained the initial stem layer. Although it benefited substantially from GPU acceleration compared to the CPU, it still achieved much lower efficiency than the large mid-network operators. The reason is structural: the layer has only three input channels and a very large spatial resolution, which limits arithmetic intensity and reduces the amount of useful parallel work available per memory access. Several other early-stage 3×3 convolutions with low channel counts showed similar behaviour.

The softmax operation in the detection head also remained inefficient. Its absolute latency was much lower than on the CPU, but its achieved throughput was still very small compared with dense convolutions. This indicates that the operation remained dominated by reductions and repeated memory access rather than by highly parallel arithmetic.

Weighted Bottleneck Analysis. As in the CPU analysis, a per-layer view alone does not fully capture the true runtime bottlenecks of YOLOv8n. Several convolution tasks appear multiple times in the graph, especially inside repeated C2f blocks, and therefore contribute more to total inference time than their individual latency would suggest. On the GPU, the dominant weighted bottlenecks were again repeated 3×3 convolutions in the backbone and neck. In particular, task 50 (`conv2d24`) contributed the largest weighted

latency, followed by task 16 (`conv2d11`), which appeared seven times in the graph. This is qualitatively similar to the CPU case: the most important optimisation targets are not necessarily the single slowest kernels, but rather the kernels that are executed many times.

Overall Assessment. Overall, the tuned TVM model reached an end-to-end latency of 23.38 ms on the Jetson Orin Nano GPU, compared with approximately 25.12 ms for PyTorch eager mode under comparable conditions. This means that TVM provided a measurable but modest improvement over the baseline GPU execution path. At the same time, the results also highlight the remaining limitations of the current FP32 TVM GPU path. While MetaSchedule reduced the latency of the convolution-heavy parts of the model substantially relative to the CPU, the absence of tensorisation meant that the gains remained bounded relative to the best available inference engine. In this experiment, TensorRT remained significantly faster at 11.38 ms, indicating that additional performance is still available beyond the schedules found by MetaSchedule. Therefore, the main conclusion is that TVM tuning is effective on the Jetson Orin Nano GPU and produces a strong result, but in FP32 it remains a competitive general solution rather than the fastest deployment path for this workload.

5.3.4 Raspberry Pi 5 Benchmark Overview - FP32

All Raspberry Pi 5 experiments were conducted on a Linux system based on the `6.12.62+rpt-rpi-2712` kernel using the AArch64 architecture. The CPU frequency governor was fixed to `performance`, and the CPU was configured to 2.4 GHz during the measurements. To reduce thermal and scheduling-related variation, the benchmarking procedure included a cooldown phase before each repetition and checks for frequency throttling. Each backend was evaluated over five repeated runs, with 200 timed inferences per repetition after 30 warm-up iterations. In addition, each repetition was executed in a fresh subprocess in order to reduce cross-framework interference, such as cache contamination or residual runtime state. The comparison was performed using all four available CPU cores on the Raspberry Pi 5 in order to reflect the maximum CPU throughput available in this deployment setting. This setup was chosen to reduce short-term variance and to provide a stable estimate of sustained inference performance under controlled CPU conditions.

The evaluated CPU backends were TVM, ONNX Runtime, TorchCompile, Google AI Edge LiteRT, and PyTorch eager mode used as a baseline. ONNX Runtime was evaluated with the `CPUExecutionProvider`. Table 5.6 summarises the measured latency and throughput, while Table 5.7 summarises power results. These measurements are further

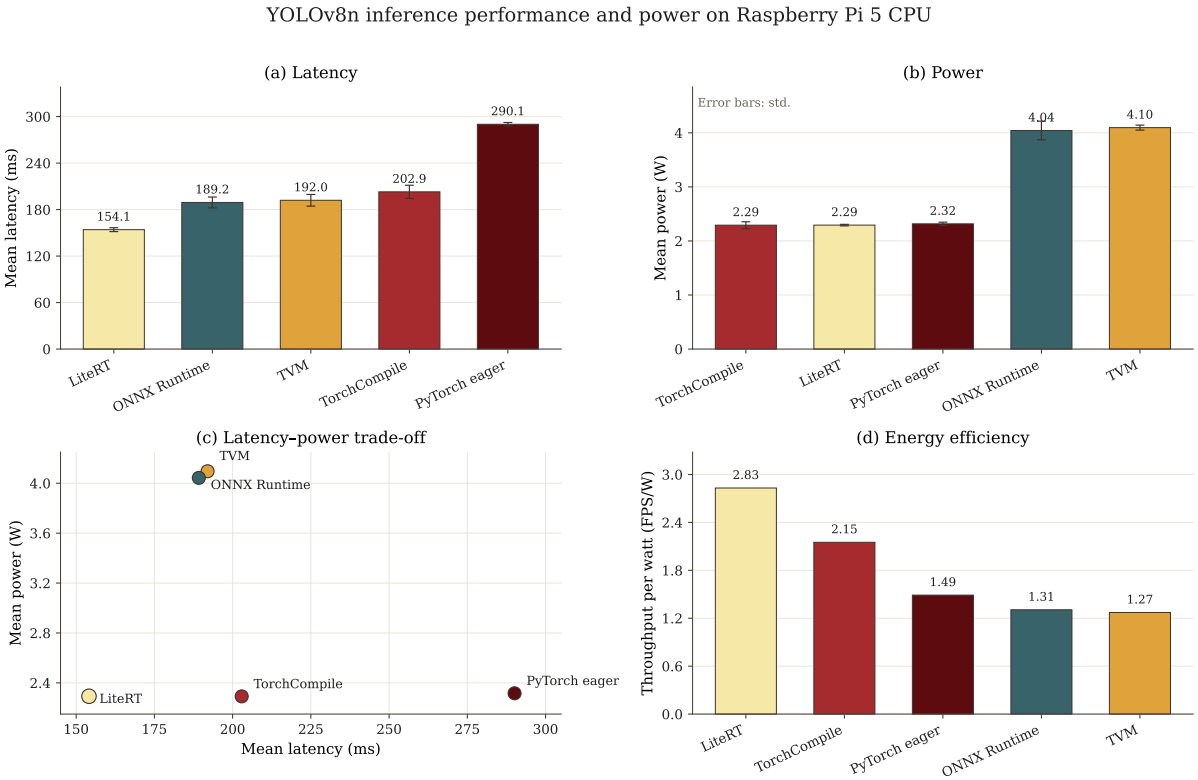

Figure 5.3: YOLOv8n inference performance and power consumption on the Raspberry Pi 5 CPU across the evaluated FP32 backends. Subfigure (a) shows the mean end-to-end inference latency with 95% confidence interval error bars. Subfigure (b) reports mean board power consumption, with error bars indicating standard deviation. Subfigure (c) summarizes the latency–power trade-off, where lower latency and lower power are preferable. Subfigure (d) shows energy efficiency in terms of throughput per watt. LiteRT achieves the lowest latency and the highest energy efficiency, while TVM and ONNX Runtime exhibit similar latency but substantially higher power consumption.

visualised in Figure 5.3, which compares latency, power consumption, the latency–power trade-off, and throughput per watt across the evaluated CPU backends.

Overall Performance. Among all evaluated frameworks, LiteRT achieved the best overall latency on the Raspberry Pi 5, with a mean inference time of 154.1 ms and a throughput of 6.49 FPS. ONNX Runtime and TVM formed the next performance group, with mean latencies of 189.2 ms and 192.0 ms respectively. TorchCompile was slower than both compiled deployment backends, reaching a mean latency of 202.9 ms and a throughput of 4.93 FPS. PyTorch eager mode remained the slowest configuration, with a mean latency of 290.1 ms and a throughput of 3.45 FPS. The difference between ONNX Runtime and TVM was only 2.8 ms, whereas TorchCompile lagged behind ONNX Runtime by 13.6 ms and behind TVM by 10.8 ms.

Table 5.6: End-to-end YOLOv8n inference performance on Raspberry Pi 5 CPU. Values are reported as mean latency ± half-width of the 95 % confidence interval across 5 isolated repetitions, with 200 timed runs per repetition after 30 warm-up iterations.

Backend	Latency (ms)	FPS mean	Median (ms)
TVM	192.0 ± 7.5	5.21	188.8
ONNX Runtime	189.2 ± 7.0	5.28	187.2
TorchCompile	202.9 ± 8.5	4.93	202.2
LiteRT	154.1 ± 2.5	6.49	155.2
PyTorch eager	290.1 ± 2.3	3.45	289.8

Table 5.7: Average power consumption during YOLOv8n inference on Raspberry Pi 5 CPU.

Backend	Mean Power (mW)	Max Power (mW)	Std (mW)
TVM	4096.2	4783.1	46.1
ONNX Runtime	4043.2	5415.1	172.5
TorchCompile	2291.8	2742.5	63.2
LiteRT	2292.8	2591.5	14.2
PyTorch eager	2316.4	2759.8	30.3

Stability of Measurements. The repeated-run statistics show that TVM and ONNX Runtime had comparable variability, with standard deviations of 6.03 ms and 5.65 ms respectively. TorchCompile showed a similar but slightly higher variability of 6.86 ms. In contrast, PyTorch eager mode and LiteRT were more stable, with standard deviations of 1.84 ms and 2.05 ms. LiteRT therefore combined the best average performance in this benchmark with comparatively consistent execution.

Power Behaviour. TVM and ONNX Runtime showed similar average power draw, measuring 4.10 W and 4.04 W respectively. In contrast, TorchCompile, LiteRT, and PyTorch eager mode all operated near 2.3 W average board power. LiteRT exhibited the lowest average power consumption at 2.29 W, although the difference relative to TorchCompile was very small. This result is notable because LiteRT was both the fastest backend in this experiment and the one with the lowest measured average power consumption.

Interpretation of the Results. Although TVM did not outperform LiteRT, it remained competitive with ONNX Runtime and outperformed TorchCompile in this end-to-end benchmark. TVM achieved slightly lower latency than TorchCompile while relying on a fully compiler-driven deployment pipeline based on Relax and MetaSchedule tuning. This suggests that ahead-of-time compilation through a dedicated inference compiler

remains more effective on this workload than applying `torch.compile` to the original PyTorch model on CPU. At the same time, the results also show that LiteRT provided the most efficient execution path among the tested frameworks on Raspberry Pi 5 for this FP32 model.

Conclusion. The benchmark results show four practical performance tiers on the Raspberry Pi 5 CPU. LiteRT delivered the best end-to-end result, combining the lowest latency, the highest throughput, and the lowest average power consumption. ONNX Runtime and TVM achieved similar performance, with ONNX Runtime being slightly faster in the measured configuration. TorchCompile improved substantially over PyTorch eager mode, but remained slower than both ONNX Runtime and TVM. PyTorch eager mode was the slowest baseline by a large margin.

Relative to the PyTorch eager baseline, TVM achieved a 1.51× speedup, corresponding to a 33.8 % reduction in mean inference latency. ONNX Runtime achieved a 1.53× speedup, corresponding to a 34.8 % latency reduction. TorchCompile achieved a 1.43× speedup, corresponding to a 30.1 % latency reduction. LiteRT provided the largest improvement, reaching a 1.88× speedup and a 46.9 % reduction in mean latency. Overall, these results indicate that TVM is capable of producing competitive CPU inference performance on Raspberry Pi 5, but in this experiment the most efficient backend was LiteRT rather than TVM.

5.3.5 Jetson Orin Nano GPU Benchmark Overview - FP32

GPU experiments were conducted on the NVIDIA Jetson Orin Nano using the full YOLOv8n model in FP32 precision with batch size 1. The evaluated backends were TVM, ONNX Runtime, TensorRT, PyTorch eager mode, and `torch.compile`. As in the CPU experiments, the benchmark procedure used isolated repetitions in order to reduce interference between backends. Table 5.8 summarises the measured latency and throughput, while Table 5.9 summarises power results. These measurements are further visualised in Figure 5.4, which compares latency, power consumption, the latency–power trade-off, and throughput per watt across the evaluated GPU backends.

Overall Performance. TensorRT achieved by far the best end-to-end latency on the Jetson Orin Nano, reaching 11.38 ms, which corresponds to 87.9 FPS. ONNX Runtime and TVM formed a second performance group with nearly identical results, at 23.18 ms and 23.38 ms respectively. PyTorch eager mode was slightly slower at 25.12 ms, while `torch.compile` performed substantially worse than all other GPU backends, reaching 72.34 ms.

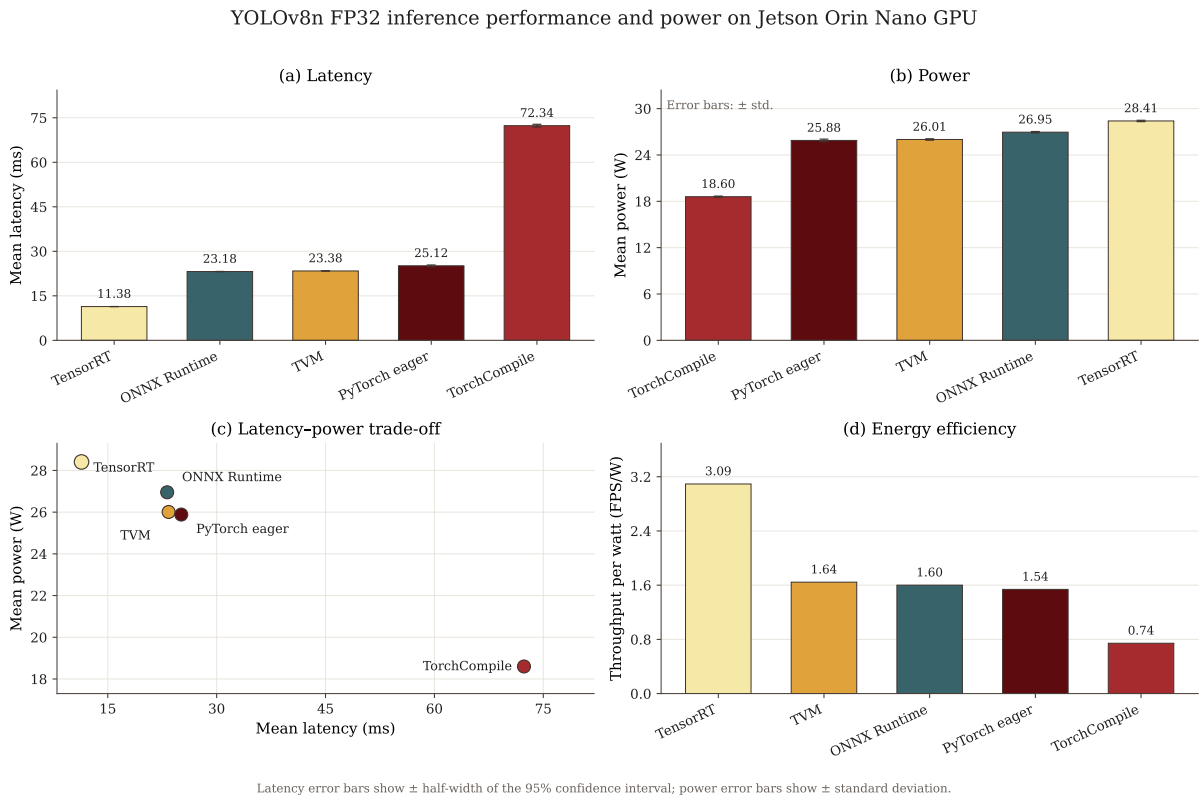

Figure 5.4: YOLOv8n FP32 inference performance and power consumption on the NVIDIA Jetson Orin Nano GPU. Subfigure (a) shows mean end-to-end inference latency with 95% confidence interval error bars. Subfigure (b) reports mean board power consumption, with error bars indicating standard deviation. Subfigure (c) summarizes the latency–power trade-off, where lower latency and lower power are preferable. Subfigure (d) shows energy efficiency in terms of throughput per watt. TensorRT achieves the lowest latency and the highest energy efficiency among the evaluated FP32 GPU backends, while torch.compile is substantially slower despite lower power draw.

Relative to the PyTorch eager baseline, TVM achieved a 1.07× speedup, corresponding to a 6.9 % reduction in mean latency. ONNX Runtime achieved a 1.08× speedup and a 7.7 % latency reduction. TensorRT provided the largest improvement, reaching a 2.21× speedup and a 54.7 % reduction in mean latency. In contrast, `torch.compile` was slower than PyTorch eager mode by a factor of 2.88×.

Power and Efficiency. In absolute power draw, the GPU backends operated within a relatively narrow range, from 18.6 W for `torch.compile` to 28.4 W for TensorRT. However, when throughput is taken into account, TensorRT was also the most power-efficient backend, achieving 3.10 FPS/W. This is approximately 1.9 times higher than TVM and ONNX Runtime, which both reached about 1.6 FPS/W, and about twice the efficiency of PyTorch eager mode. Although `torch.compile` consumed the least power, its very low throughput made it the least efficient backend overall.

Table 5.8: End-to-end YOLOv8n inference performance on the NVIDIA Jetson Orin Nano GPU. Values are reported as mean latency ± half-width of the 95 % confidence interval across 5 isolated repetitions, with 200 timed runs per repetition after 30 warm-up iterations.

Backend	Latency (ms)	FPS mean	Median (ms)
TVM	23.38 ± 0.02	42.77	23.39
ONNX Runtime	23.18 ± 0.04	43.14	23.17
TorchCompile	72.34 ± 0.47	13.82	72.26
TensorRT	11.38 ± 0.02	87.87	11.39
PyTorch eager	25.12 ± 0.27	39.81	25.12

Table 5.9: Average power consumption during YOLOv8n inference on the NVIDIA Jetson Orin Nano GPU.

Backend	Mean Power (mW)	Max Power (mW)	Std (mW)
TVM	26007.9	26781.1	88.7
ONNX Runtime	26952.4	29632.6	84.8
TorchCompile	18599.2	19967.0	76.7
TensorRT	28405.8	30936.2	86.4
PyTorch eager	25884.4	29186.0	168.4

Interpretation of the TVM Result. The TVM GPU result is notable for two reasons. First, its end-to-end latency was essentially identical to ONNX Runtime, with a difference of only 0.20 ms. Second, TVM still outperformed PyTorch eager mode, although the margin was much smaller than on the CPU. This suggests that, on the Orin Nano GPU, the main convolution-heavy parts of the model are already executed efficiently by mature CUDA libraries across multiple frameworks, which reduces the practical advantage of compiler-level scheduling alone. In other words, TVM remained competitive, but did not establish a clear lead over ONNX Runtime on this platform.

Conclusion. The GPU benchmark results reveal three practical performance tiers. TensorRT forms the clear top tier, achieving both the lowest latency and the highest power efficiency. ONNX Runtime and TVM form a second tier with nearly identical performance, while PyTorch eager mode is only slightly slower. In contrast, `torch.compile` was not competitive for this workload in the tested configuration. Overall, these results show that TensorRT was the most effective deployment backend for YOLOv8n on the Jetson Orin Nano GPU, whereas TVM and ONNX Runtime delivered similar and competitive FP32 performance.

Relative to the PyTorch eager baseline, TVM achieved a 1.07× speedup, equivalent to a 6.9 % reduction in mean inference latency, and ONNX Runtime achieved a

1.08× speedup with a 7.7 % latency reduction. TensorRT provided the largest improvement, reaching a 2.21× speedup and a 54.7 % reduction in mean latency. In contrast, `torch.compile` failed to improve upon the baseline and instead increased latency by a factor of 2.88×. These results indicate that TVM can provide competitive GPU inference performance on the Jetson Orin Nano, but in this experiment the largest practical advantage was obtained with TensorRT.

5.4 FP16 Inference Results

Before discussing the measured FP16 results, it is important to clarify that conversion to FP16 is not the same as quantization. In FP16 inference, weights and activations remain in floating-point representation, but with reduced numerical precision compared to FP32. By contrast, quantization usually refers to mapping tensors into lower-bit integer or fixed-point formats, such as INT8, which changes not only the storage format but also the arithmetic model used during execution. FP16 should therefore be understood as reduced-precision floating-point inference rather than true quantization.

This distinction is important for interpreting the following measurements. Reducing precision from FP32 to FP16 can decrease memory traffic and improve throughput on hardware with native half-precision support, often with only a small impact on model accuracy. However, the resulting performance behaviour is different from that of integer quantization, since the model still operates in a floating-point domain. The following subsection therefore evaluates FP16 as a separate optimisation path and analyses its effect on latency, throughput, and overall deployment efficiency.

5.4.1 TVM MetaSchedule Tuning Results on the Jetson Orin Nano GPU - FP16

TVM MetaSchedule was evaluated a second time on the NVIDIA Jetson Orin Nano GPU using a float16 version of the YOLOv8n inference graph. The model was converted from the FP32 ONNX export using `onnxconverter-common` with `keep\_io\_types=False`, producing a graph in which the input, weights, and intermediate activations are represented in FP16. The Relax frontend imported this graph without further modification. During the tuning process, TVM explored 6 477 valid schedules across 66 extracted operator tasks over approximately 18 hours. The resulting tuned model achieved a summed task latency of 9 872 μs, while the measured end-to-end inference latency was 12.45 ms. As in the FP32 case, the gap between the summed task latency and the measured end-to-end latency remained small, indicating that the tuned kernel execution accounted for most of

the observed runtime.

Compared with the FP32 GPU tuning result of 22 920 μs summed task latency, the FP16 tuned model achieved a kernel-level improvement of approximately 2.3×. At the end-to-end level, the improvement was also substantial: latency decreased from 23.38 ms in FP32 to 12.45 ms in FP16, corresponding to an end-to-end speedup of approximately 1.88×. This result shows that reduced-precision compilation was not merely a small incremental optimisation, but a major change in the execution regime of the model on this platform.

Tensorisation as the Main Structural Change. The most important difference between the FP32 and FP16 tuning campaigns was that FP16 enabled tensorisation through WMMA-style matrix multiply-accumulate operations on the Orin Nano GPU. In the FP32 campaign, no task could be mapped to a dedicated hardware intrinsic, and all schedules remained conventional CUDA thread-block implementations. In the FP16 campaign, by contrast, a large fraction of tasks could be mapped to tensorised execution. As a result, the optimisation problem changed qualitatively: instead of relying only on thread-block tiling and memory scheduling, MetaSchedule was able to search over implementations that directly exploited the GPU's matrix-multiplication hardware.

This shift is clearly visible in the achieved arithmetic throughput. The peak throughput increased from 585.6 GFLOPS in the FP32 campaign to 1 872.7 GFLOPS in FP16, representing more than a 3.2× increase in peak operator efficiency. This confirms that the performance gain observed in FP16 was not only a consequence of reduced data size, but also of a more favourable optimisation path enabled by the lower-precision representation.

Best-Performing Operators. The best-performing tasks in the FP16 tuning campaign were large mid-network convolutions, especially fused operators in the backbone and neck. These layers benefited most from tensorised execution because their channel dimensions and spatial sizes were large enough to keep the available execution resources well utilised. The most efficient task reached 1 872.7 GFLOPS, while several others exceeded 1 200 GFLOPS. Compared with the FP32 results, this represents a substantial improvement in computational efficiency and confirms that the largest dense convolutions were the main beneficiaries of the FP16 compilation path.

Main Bottlenecks. Despite the overall improvement, the main bottlenecks remained qualitatively similar to those observed in FP32. The least efficient convolution was still the initial stem layer, whose very low input channel count limited its ability to benefit from tensorised execution. Even in FP16, this layer remained dominated by poor arithmetic intensity and limited data reuse, and therefore showed only modest improvement relative

to the rest of the network. Several other early-stage convolutions with small channel counts exhibited similar behaviour.

Non-convolutional operators in the detection path also remained relatively lightweight in absolute latency terms, but their optimisation potential was much smaller than that of the large mid-network convolutional kernels. As a result, the overall runtime distribution in FP16 became even more strongly concentrated in the repeated backbone and neck convolutions.

Weighted Bottleneck Analysis. As in the CPU and FP32 GPU analyses, per-task latency alone does not fully describe the runtime profile of YOLOv8n because some operators are executed multiple times in a single forward pass. When task multiplicity is taken into account, the dominant contributors to total runtime in FP16 remained repeated 3×3 convolutions in the backbone and neck. In particular, task 50 (`conv2d24`) and task 16 (`conv2d11`) emerged as the most important weighted bottlenecks. The stem layer also remained a notable contributor because of its disproportionately high latency relative to its position in the graph. These results reinforce the conclusion that the most important optimisation targets are repeated mid-network convolutions rather than simply the single slowest layer.

Overall Assessment. Overall, the FP16 tuning campaign demonstrates that TVM MetaSchedule can exploit the Jetson Orin Nano GPU much more effectively in FP16 than in FP32. The end-to-end latency of the tuned TVM model decreased from 23.38 ms in FP32 to 12.45 ms in FP16, while tuning time itself was reduced from 39 days to approximately 18 hours. The key structural reason for this improvement was that FP16 enabled tensorised execution for a large fraction of convolutional tasks, whereas FP32 did not. This establishes FP16 as the more appropriate precision target for TVM-based deployment of YOLOv8n on Orin-class hardware.

At the same time, the results also show that the current FP16 TVM path still does not fully match the fastest available inference engine. Although TVM improved substantially and achieved a strong result, TensorRT remained faster in the final benchmark comparison. Therefore, the main conclusion is that FP16 MetaSchedule tuning is highly effective on the Jetson Orin Nano GPU and substantially strengthens TVM's competitiveness, but it does not yet eliminate the remaining performance gap to the most specialised deployment backend.

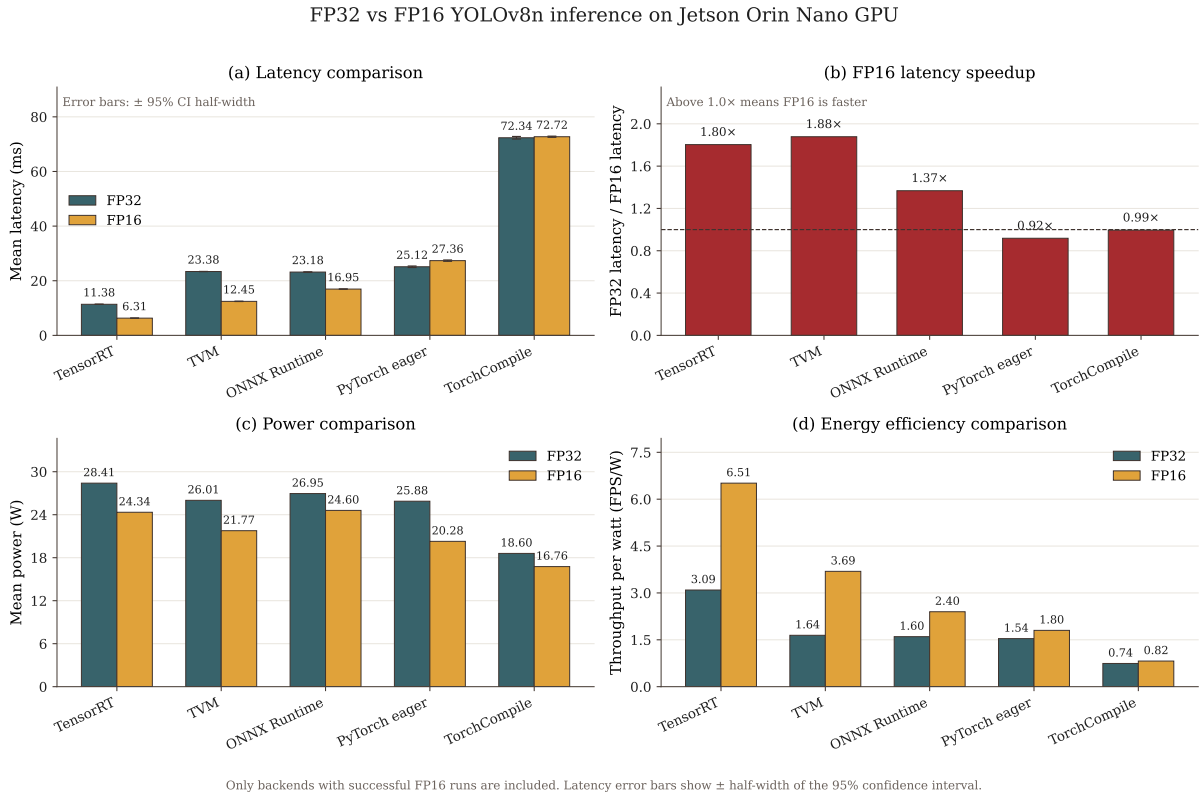

Figure 5.5: FP32 versus FP16 YOLOv8n inference on the NVIDIA Jetson Orin Nano GPU. Subfigure (a) compares mean end-to-end latency for each backend, with error bars indicating the half-width of the 95% confidence interval. Subfigure (b) reports the FP16 latency speedup relative to FP32 for the same backend, where values above 1.0× indicate that FP16 is faster. Subfigure (c) compares mean board power consumption, and Subfigure (d) compares energy efficiency in terms of throughput per watt. TensorRT and TVM obtain the strongest FP16 latency improvements, while `torch.compile` and PyTorch eager mode do not benefit from FP16 in this experiment.

5.4.2 Jetson Orin Nano GPU Benchmark Overview - FP16

GPU experiments were also conducted on the NVIDIA Jetson Orin Nano using the full YOLOv8n model in FP16 precision with batch size 1. The evaluated backends were TVM, ONNX Runtime, TensorRT, PyTorch eager mode, and `torch.compile`. As in the previous benchmark sections, the comparison was performed using isolated repetitions in order to reduce interference between frameworks. Table 5.10 summarises the measured latency and throughput, while Table 5.11 summarises power results. Figure 5.5 further compares the FP16 measurements against the corresponding FP32 results, showing the latency, FP16 speedup, power consumption, and energy efficiency for each backend.

Overall Performance. TensorRT achieved the best end-to-end latency in the FP16 setting, reaching 6.31 ms, which corresponds to 158.6 FPS. TVM formed the second-best result at 12.45 ms and substantially outperformed ONNX Runtime, which reached 16.95 ms.

Table 5.10: End-to-end YOLOv8n inference performance on the NVIDIA Jetson Orin Nano GPU in FP16 precision. Values are reported as mean latency ± half-width of the 95 % confidence interval across 5 isolated repetitions, with 200 timed runs per repetition after 30 warm-up iterations.

Backend	Latency (ms)	FPS mean	Median (ms)
TVM	12.45 ± 0.05	80.35	12.43
ONNX Runtime	16.95 ± 0.02	59.00	16.96
TorchCompile	72.72 ± 0.21	13.75	72.76
TensorRT	6.31 ± 0.01	158.55	6.31
PyTorch eager	27.36 ± 0.24	36.55	27.34

PyTorch eager mode was considerably slower at 27.36 ms, while `torch.compile` again performed substantially worse than all other backends, reaching 72.72 ms.

Relative to the PyTorch eager baseline, TVM achieved a 2.20× speedup, corresponding to a 54.5 % reduction in mean latency. ONNX Runtime achieved a 1.61× speedup and a 38.1 % latency reduction. TensorRT provided the largest improvement, reaching a 4.34× speedup and a 76.9 % reduction in mean latency. In contrast, `torch.compile` was slower than PyTorch eager mode by a factor of 2.66×.

Table 5.11: Average power consumption during YOLOv8n inference on the NVIDIA Jetson Orin Nano GPU in FP16 precision.

Backend	Mean Power (mW)	Max Power (mW)	Std (mW)
TVM	21770.0	24514.2	122.6
ONNX Runtime	24601.7	29104.0	88.3
TorchCompile	16760.5	17874.5	33.9
TensorRT	24342.7	26500.5	19.0
PyTorch eager	20282.6	20852.0	79.9

Power and Efficiency. In absolute power draw, the FP16 GPU backends again operated within a relatively narrow range, from 16.8 W for `torch.compile` to 24.6 W for ONNX Runtime. However, throughput differed much more strongly than power, which made efficiency comparisons especially informative. TensorRT achieved the best efficiency at approximately 6.51 FPS/W, followed by TVM at about 3.69 FPS/W. PyTorch eager mode reached about 1.80 FPS/W, and ONNX Runtime about 2.40 FPS/W. Although `torch.compile` consumed the least power, its low throughput again made it the least competitive backend in practice.

Interpretation of the TVM Result. The TVM FP16 result is particularly notable because, unlike in FP32, TVM no longer merely matched ONNX Runtime but clearly

outperformed it. Its latency was 4.50 ms lower than ONNX Runtime, corresponding to an additional reduction of approximately 26.6 %. This indicates that FP16 compilation gave TVM access to optimisation opportunities that were not available in the FP32 case, most importantly tensorised execution for a large fraction of the convolutional operators. As a result, TVM moved from being merely competitive in FP32 to becoming the second-best backend overall in FP16.

At the same time, TensorRT still remained significantly faster, with a latency of 6.31 ms compared with 12.45 ms for TVM. This shows that even though TVM benefited greatly from FP16 compilation, additional backend-specific optimisations still remain available beyond what MetaSchedule found in this work.

Conclusion. The FP16 benchmark results reveal a clearer separation between deployment backends than in the FP32 GPU case. TensorRT formed the top tier with the best latency, throughput, and efficiency. TVM formed a strong second tier and substantially outperformed ONNX Runtime, PyTorch eager mode, and `torch.compile`. ONNX Runtime remained competitive but was noticeably slower than TVM, while PyTorch eager mode was substantially slower than all three specialised deployment paths. As in the FP32 case, `torch.compile` was not competitive for this workload in the tested configuration.

Overall, these results indicate that FP16 is the most favourable precision setting for TVM deployment on the Jetson Orin Nano GPU. In this configuration, TVM no longer merely matched other general deployment runtimes, but delivered a clear practical advantage over ONNX Runtime and PyTorch eager mode. However, TensorRT still provided the fastest overall execution and remained the most effective deployment backend in the final comparison.

5.4.3 FP16 Inference on the Raspberry Pi 5 CPU

To investigate whether half-precision arithmetic provides a runtime benefit on the Raspberry Pi 5, the YOLOv8n model was re-evaluated at FP16 precision across all applicable backends. The FP16 ONNX model was generated from the FP32 export using `onnxconverter-common` with `keep\_io\_types=False`, producing a graph in which all weights, activations, and the input tensor are represented as 16-bit floats. TVM was re-tuned on this model, yielding a separately compiled `.so` library. All other backends received the same FP16 ONNX as input.

TVM FP16 Tuning Session. TVM MetaSchedule tuned 65 operator tasks over approximately 4 hours and 6 minutes - compared with 6 hours and 15 minutes for the FP32 campaign on the same graph. The faster convergence reflects the more distinct latency

signal that FP16 kernels produce: once the vectoriser finds a schedule that fills the 128-bit NEON register with eight `float16` values instead of four `float32` values, the improvement is immediately visible and the tuner concentrates its remaining trials on refinement rather than exploration. A total of 6,477 valid schedule candidates were evaluated across all tasks.

As in the FP32 CPU campaign, every task encountered the `workloadcannotbe tensorized` message from MetaSchedule, for the same structural reason: the three-dimensional reduction of a convolution (input channels, kernel height, kernel width) cannot be mapped onto the one-dimensional dot-product pattern required by the NEON intrinsic interface. The tuner therefore fell back to standard cache-blocked tiling with LLVM auto-vectorisation in all 65 tasks. At FP16, LLVM targets the `float16x8` NEON type on cortex-a76, doubling the number of values processed per vector instruction compared with the `float32x4` path used in FP32 tuning. The peak achieved throughput rose from 87.1 GFLOPS in the FP32 session to 169.2 GFLOPS in the FP16 session - a ratio of 1.94×, consistent with the theoretical 2× benefit from halving the element width within a fixed 128-bit register.

End-to-End Benchmark Results. Figure 5.6 visualises the successful FP16 runs against their corresponding FP32 measurements on the Raspberry Pi 5 CPU, including latency, FP16 speedup, power consumption, and energy efficiency. Table 5.12 presents the full benchmark comparison between FP32 and FP16 precision across all backends on the Raspberry Pi 5, including the unsuccessful or fallback FP16 cases.

Table 5.12: FP32 vs FP16 inference on the Raspberry Pi 5 CPU (5 repetitions × 200 runs, 30 warm-up runs, performance governor, batch size 1, 640×640 input). ORT and PyTorch FP16 results are estimated from single-repetition measurements; TorchCompile FP16 is from the rigorous run.

Backend	FP32 mean (ms)	FP16 mean (ms)	Ratio	FP16 status
TVM	192.01	122.44	1.57× faster	real fp16
ONNXRuntime	189.23	>900 s	timeout	no fp16 Conv
TFLite	154.06	153.27	1.00×	weights only
TorchCompile	200.39	983.08	4.9× slower	GEMM fallback
PyTorch	290.13	≈11 000	38× slower	scalar loop

TVM - the Only Backend with Real FP16 Acceleration. TVM reduced its end-to-end latency from 192.01 ms to 122.44 ms, a speedup of 1.57×. This improvement stems from a qualitative change in the generated code: LLVM emits `VFMA.F16` NEON instructions that operate on eight `float16` values in parallel within the same 128-bit register that previously held four `float32` values. The gap between the measured 1.57×

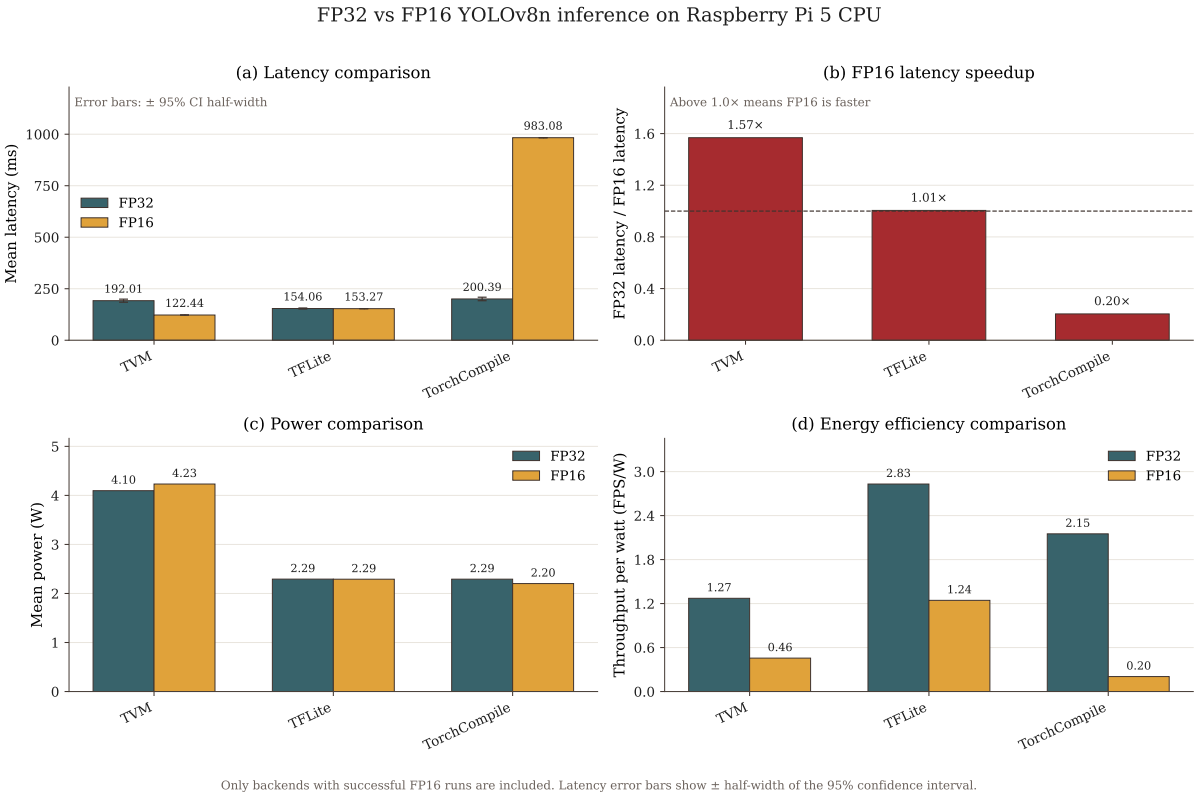

Figure 5.6: FP32 versus FP16 YOLOv8n inference on the Raspberry Pi 5 CPU for backends with successful FP16 runs. Subfigure (a) compares mean end-to-end latency, with error bars indicating the half-width of the 95% confidence interval where available. Subfigure (b) reports the FP16 latency speedup relative to FP32 for the same backend, where values above 1.0× indicate that FP16 is faster. Subfigure (c) compares mean board power consumption, and Subfigure (d) compares energy efficiency in terms of throughput per watt. TVM is the only backend that shows a clear FP16 latency improvement on the Raspberry Pi 5 CPU, while TFLite remains nearly unchanged and TorchCompile becomes substantially slower.

end-to-end speedup and the theoretical 2× kernel-level ceiling is explained by two factors. First, non-arithmetic operators - splits, concatenations, and the upsampling layers of the FPN neck - execute at the same speed regardless of element width. Second, the initial stem convolution, which has only three input channels, cannot fill the eight available FP16 NEON lanes; its latency therefore improves only marginally between precisions. Both of these effects are irreducible at the schedule level and represent genuine architectural constraints of the YOLOv8n graph on a scalar-SIMD processor.

ONNXRuntime - Timeout Due to Missing FP16 Convolution Kernel. The ONNXRuntime worker exceeded the 900-second subprocess timeout and produced no result. This is not a measurement artefact but a direct consequence of the MLAS math library not implementing a half-precision convolution kernel for ARM. When MLAS encounters a FP16 `Conv` operator, it inserts `Cast(float16$\to$float32)` nodes before and

after every convolution and computes the operation in FP32. The resulting per-inference latency is estimated at approximately 7 100 ms - derived by scaling the FP32 ORT result by the same regression factor observed in PyTorch eager FP16. With 30 warm-up and 200 timed iterations, the total subprocess time would reach approximately 1 630 seconds, well above the timeout. This behaviour has been documented in the ONNX Runtime issue tracker and reflects a fundamental gap in MLAS coverage rather than a configuration error.

TFLite - Storage Benefit Only. TFLite produced virtually identical latency at both precisions: 154.06 ms in FP32 and 153.27 ms in FP16, a difference smaller than one standard deviation. This result confirms that TFLite's FP16 model format stores weights as 16-bit values but promotes all arithmetic to FP32 before computation on this platform. The model occupies half the memory, which can reduce cache pressure and improve load time, but no arithmetic throughput benefit accrues at inference time. The near-identical power consumption at both precisions (2 293 mW FP16 vs 2 293 mW FP32) corroborates this interpretation.

TorchCompile and PyTorch - Regression Under FP16. Both PyTorch-based backends degraded significantly at FP16. PyTorch eager mode regressed by a factor of 38×, from 290 ms to approximately 11 000 ms. When `model.half()` is called on a CPU device, the ATen kernel dispatcher finds no half-precision GEMM implementation and falls back to an element-wise scalar loop that processes each `float16` value individually, upcasting to `float32` for each multiply-accumulate step.

TorchCompile degraded by 4.9×, from 200 ms to 983 ms. This is less severe than PyTorch eager because the Inductor backend successfully generates vectorised C++ code for element-wise operations such as the SiLU activation and residual additions. However, the convolution operators - which dominate total runtime - are not covered by Inductor's FP16 CPU code generation path and still fall through to the same ATen scalar fallback. The resulting graph alternates between fast Inductor-compiled segments and slow ATen scalar segments, with type-conversion overhead at every boundary. The net effect is that TorchCompile FP16 is slower than TorchCompile FP32 despite Inductor genuinely improving some of the individual operations.

Power Consumption. Table 5.13 reports the mean board power measured during inference for the backends that completed successfully.

TFLite achieves the best energy efficiency at FP16 (2.85 FPS/W), primarily because its lightweight runtime draws substantially less board power than the TVM or Inductor runtimes while delivering competitive latency. TVM draws more power (4 232 mW) due

Table 5.13: Power and energy-efficiency comparison on the Raspberry Pi 5 CPU at FP16 precision (successful runs only).

Backend	Mean latency (ms)	Power (mW)	Energy/frame (mJ)	FPS/W
TVM	122.44	4 232	517.7	1.93
TFLite	153.27	2 291	351.1	2.85
TorchCompile	983.08	2 202	2 164.0	0.45

to higher CPU utilisation from its densely vectorised kernels, but still outperforms TFLite in absolute latency. TorchCompile consumes the least power per unit time but has by far the highest energy per frame because of its poor throughput.

Overall Assessment. The Raspberry Pi 5 FP16 results demonstrate that half-precision acceleration on ARM CPU is a compiler problem rather than a hardware problem. The Cortex-A76 implements ARMv8.2-A FEAT_FP16, providing native `float16` arithmetic in its NEON units. The bottleneck is that no major CPU runtime library - MLAS, OpenBLAS, or the ATen dispatcher - implements a half-precision GEMM kernel for this target. Only TVM, which generates the convolution kernel directly as LLVM IR and lets the AArch64 backend select `float16x8` NEON instructions, can exploit the available hardware capability. The result is a 1.57× end-to-end speedup over FP32 TVM, achieved within a four-hour tuning session. All other frameworks either time out, regress, or show no improvement at FP16 precision. This finding has a direct practical implication: deploying FP16 models on edge CPU platforms without a compilation-based inference engine yields no latency benefit and may cause severe degradation.

5.5 Accuracy Comparison Across Inference Frameworks

To verify that different inference frameworks preserve the predictive quality of the original model, YOLOv8n was evaluated across several backends and compared against the default PyTorch eager FP32 implementation. Since large multi-backend evaluation is memory-intensive, especially when repeatedly loading different runtimes and exported artifacts, the comparison was performed on a 1000-image subset of the validation set rather than on the full validation split. Although smaller than the full COCO validation set, this subset is sufficient to determine whether export, compilation, or reduced precision introduce any meaningful degradation in detection quality.

All backends were evaluated using the same basic setup: image size 640×640, batch size 1, confidence threshold 0.001, IoU threshold 0.7, and maximum number of detections

300. Each backend was loaded and evaluated sequentially in isolation in order to reduce memory pressure and avoid interference between runtime environments.

Because the GPU-oriented and CPU-oriented backends were evaluated in separate comparison runs, they are presented separately below. The small difference between the corresponding PyTorch FP32 baseline values should therefore be interpreted as run-to-run variation rather than as a framework effect.

GPU-Oriented Backends

The GPU-side comparison included PyTorch eager FP32/FP16, ONNX Runtime CUDA FP32/FP16, TensorRT FP32/FP16, and TVM GPU FP32/FP16. Absolute results are shown in Table 5.14, and relative changes with respect to the PyTorch eager FP32 baseline of the same run are given in Table 5.15.

Table 5.14: Absolute detection metrics for GPU-oriented backends on 1000 validation images.

Backend	Precision	Recall	mAP50	mAP50–95
PyTorch eager FP32	0.6299	0.4878	0.5384	0.3899
PyTorch eager FP16	0.6301	0.4880	0.5385	0.3898
ONNX Runtime FP32	0.6283	0.4905	0.5386	0.3899
ONNX Runtime FP16	0.6300	0.4888	0.5387	0.3900
TensorRT FP32	0.6282	0.4893	0.5385	0.3899
TensorRT FP16	0.6248	0.4907	0.5385	0.3901
TVM GPU FP32	0.6319	0.4889	0.5390	0.3899
TVM GPU FP16	0.6290	0.4893	0.5381	0.3894

Table 5.15: Relative changes (%) for GPU-oriented backends with respect to PyTorch eager FP32.

Backend	Δ Precision (%)	Δ Recall (%)	Δ mAP50 (%)	Δ mAP50–95 (%)
PyTorch eager FP16	+0.03	+0.03	+0.02	-0.01
ONNX Runtime FP32	-0.25	+0.54	+0.05	+0.02
ONNX Runtime FP16	+0.02	+0.19	+0.06	+0.02
TensorRT FP32	-0.27	+0.30	+0.02	+0.01
TensorRT FP16	-0.81	+0.58	+0.03	+0.07
TVM GPU FP32	+0.32	+0.23	+0.12	+0.00
TVM GPU FP16	-0.14	+0.29	-0.05	-0.12

The GPU results show that all backends preserved accuracy very well. The observed changes in mAP50 and mAP50–95 were very small, indicating that export to ONNX Runtime, TensorRT, and TVM did not introduce any practically meaningful loss in detector quality. FP16 also remained stable overall. PyTorch FP16, ONNX Runtime CUDA FP16,

and TensorRT FP16 all stayed extremely close to the FP32 baseline. TVM GPU FP16 showed the largest negative deviation on the GPU side, but the absolute change remained small.

CPU-Oriented Backends

A separate CPU-side comparison included PyTorch eager FP32/FP16, ONNX Runtime CPU, TFLite CPU FP32/FP16, and TVM CPU FP32/FP16. Absolute results are shown in Table 5.16, and relative changes with respect to the PyTorch eager FP32 baseline of that run are shown in Table 5.17.

Table 5.16: Absolute detection metrics for CPU-oriented backends on 1000 validation images.

Backend	Precision	Recall	mAP50	mAP50–95
PyTorch eager FP32	0.6291	0.4884	0.5384	0.3898
PyTorch eager FP16	0.6291	0.4884	0.5384	0.3898
ONNX Runtime CPU	0.6291	0.4884	0.5384	0.3898
TFLite FP32 CPU	0.6291	0.4884	0.5384	0.3898
TFLite FP16 CPU	0.6276	0.4905	0.5384	0.3899
TVM CPU FP32	0.6291	0.4884	0.5384	0.3898
TVM CPU FP16	0.6280	0.4894	0.5395	0.3900

Table 5.17: Relative changes (%) for CPU-oriented backends with respect to PyTorch eager FP32.

Backend	Δ Precision (%)	Δ Recall (%)	Δ mAP50 (%)	Δ mAP50–95 (%)
PyTorch eager FP16	+0.00	+0.00	+0.00	+0.00
ONNX Runtime CPU	-0.00	+0.00	+0.00	+0.00
TFLite FP32 CPU	-0.00	-0.00	+0.00	+0.00
TFLite FP16 CPU	-0.24	+0.43	+0.00	+0.03
TVM CPU FP32	-0.00	-0.00	+0.00	+0.00
TVM CPU FP16	-0.17	+0.20	+0.20	+0.06

The CPU results are similarly stable. PyTorch FP16, ONNX Runtime CPU, TFLite FP32 CPU, and TVM CPU FP32 reproduced the PyTorch FP32 baseline almost exactly. TFLite FP16 and TVM CPU FP16 introduced small shifts in the balance between precision and recall, but the aggregate mAP values remained effectively unchanged. In particular, TVM CPU FP16 slightly improved mAP50 and mAP50–95 while slightly reducing precision, again indicating only minor numerical variation rather than a meaningful change in model behavior.

Discussion

Across both GPU and CPU experiments, no framework caused a significant drop in detection quality. The most important result is that mAP50 and especially mAP50–95 remained very close to the corresponding PyTorch eager FP32 baselines in all runs. This shows that the predictive behavior of YOLOv8n is preserved well across the evaluated deployment frameworks.

Because all backends execute the same trained model, the remaining small differences should not be interpreted as one framework being inherently more accurate than another. Instead, they are better explained by runtime-dependent numerical variation. Different inference frameworks may use different kernel implementations, operator fusion patterns, accumulation orders, and precision conversion paths. In object detection, even tiny numerical differences can slightly shift confidence scores or bounding box coordinates, which may then change whether a detection is kept or removed during thresholding and non-maximum suppression. This explains the recurring pattern that some FP16 configurations slightly reduce precision while slightly increasing recall, without materially changing the overall mAP.

Overall, these results indicate that framework conversion and mixed-precision inference do not meaningfully degrade YOLOv8n accuracy in the evaluated setups. This justifies focusing the later comparison primarily on latency, throughput, and hardware efficiency, since accuracy was effectively preserved across both GPU-oriented and CPU-oriented backends.

5.6 Influence of Quantization on YOLOv8n

Quantization is often presented as a direct way to improve inference efficiency by reducing numerical precision, model size, and memory bandwidth requirements. In practice, however, quantization remains a difficult deployment problem. Its final effect depends not only on the quantization method itself, but also on the maturity of the compiler or runtime stack, the target hardware, and the way quantized operators are represented in the exported graph.

5.6.1 INT8 Inference on NVIDIA Orin Nano GPU

For this reason, the quantization study in this thesis focuses on practical deployment behavior rather than on theoretical INT8 speedup alone. The experiments were carried out on the Jetson Orin Nano GPU and compared FP32 and INT8 inference across several

deployment paths. The PyTorch FP32 model served as the reference baseline. ONNX Runtime was evaluated using statically quantized QDQ graphs, while TensorRT was evaluated both with an implicit INT8 calibration path starting from an FP32 ONNX model and with a QDQ-based INT8 ONNX graph.

A first important observation is that efficient quantization support is not uniformly available across deployment frameworks. For example, while Apache TVM provides strong support for FP32 compilation and scheduling through MetaSchedule, the Relax-based quantization flow remains immature in the tested version. In practice, implementing a custom INT8 path would require calibration tooling, graph rewriting, requantization logic, and validated low-precision kernels, which is beyond the scope of the present thesis. By contrast, ONNX Runtime and TensorRT both provide deployable INT8 paths, but their practical outcomes differ substantially.

ONNX Runtime static quantization inserts explicit `QuantizeLinear` and `DequantizeLinear` operators into the computational graph. Although this representation is portable, it may also introduce substantial overhead when these conversions are not fully fused away by the execution backend. In a convolution-heavy architecture such as YOLOv8n, this is especially problematic, since quantization boundaries may appear repeatedly throughout the network. TensorRT can ingest such QDQ graphs, but the resulting execution still contains additional quantization-related transformations, reformatting steps, and copy operations. An alternative TensorRT path is implicit INT8 calibration, where the engine is built from an FP32 ONNX model and calibrated directly for TensorRT's internal low-precision execution. This remains one of the most accessible hardware-native INT8 deployment mechanisms on Jetson-class devices.

Table 5.18 summarizes the detection accuracy results. All measurements were performed on the same set of 1000 images from the COCO validation split. TensorRT FP32 reproduces the PyTorch FP32 baseline almost exactly, confirming that TensorRT export by itself does not meaningfully affect model quality. In contrast, all evaluated INT8 paths reduce accuracy, but the magnitude of the degradation varies strongly. The ONNX Runtime QDQ models lead to a moderate drop of approximately 6–9% in mAP, depending on the calibration subset. TensorRT implicit INT8 calibration performs worse in accuracy than the FP32 TensorRT engine and also remains less accurate than the previously evaluated FP16 TensorRT configuration, but it is still clearly more accurate than the TensorRT QDQ path.

The TensorRT QDQ engine exhibits the largest accuracy degradation, with mAP50 decreasing by more than 25% and mAP50–95 by nearly 42%, which makes this variant impractical for deployment in the tested setup. A likely reason is that the ONNX QDQ graph requires additional adjustment before efficient TensorRT execution becomes possible. In particular, TensorRT does not support the default UINT8 zero-point convention

commonly used in ONNX graphs, while the exported QDQ graph used here also quantizes bias tensors and final network outputs. As a result, the graph is not directly aligned with TensorRT's preferred low-precision execution path and requires extra transformations that can negatively affect both numerical behavior and final accuracy.

Table 5.18: Accuracy comparison of FP32 and INT8 YOLOv8n variants on the Jetson Orin Nano.

Backend	Precision	Recall	mAP50	mAP50–95
PyTorch FP32	0.6299	0.4878	0.5384	0.3899
ONNX Runtime INT8 QDQ (1%)	0.6049	0.4573	0.5053	0.3548
ONNX Runtime INT8 QDQ (0.1%)	0.6122	0.4694	0.5067	0.3591
ONNX Runtime INT8 QDQ (10%)	0.5872	0.4682	0.5001	0.3533
TensorRT FP32	0.6288	0.4905	0.5386	0.3899
TensorRT INT8 implicit	0.5806	0.4418	0.4846	0.3481
TensorRT INT8 QDQ	0.4781	0.4162	0.4016	0.2264

The same trend is visible in relative form in Table 5.19. Among the ONNX Runtime QDQ runs, the 0.1% calibration subset, corresponding to roughly 130 images, produced the least severe degradation, but the differences between the tested calibration splits are relatively small compared with the overall gap between INT8 and FP32 execution. This suggests that calibration-set size alone is not the dominant factor in the observed loss. Instead, the larger issue appears to be the deployment path itself and the extent to which low-precision conversions are propagated and fused efficiently.

Table 5.19: Relative accuracy change (%) of INT8 and TensorRT backends with respect to the PyTorch FP32 baseline.

Backend	Δ Precision	Δ Recall	Δ mAP50	Δ mAP50–95
ONNX Runtime INT8 QDQ (0.1%)	-2.80	-3.78	-5.88	-7.89
ONNX Runtime INT8 QDQ (1%)	-3.96	-6.26	-6.15	-8.98
ONNX Runtime INT8 QDQ (10%)	-6.78	-4.02	-7.11	-9.37
TensorRT FP32	-0.18	+0.56	+0.05	+0.02
TensorRT INT8 implicit	-7.83	-9.43	-9.98	-10.71
TensorRT INT8 QDQ	-24.10	-14.68	-25.40	-41.93

Figure 5.7 visualises the INT8 benchmark results on the NVIDIA Jetson Orin Nano GPU, including latency, speedup relative to the PyTorch FP32 eager baseline, power consumption, and energy efficiency. Latency results, shown numerically in Table 5.20, reveal an equally important deployment trade-off. The fastest configuration in the measured benchmark was TensorRT implicit INT8 quantization, achieving approximately 5.34 ms

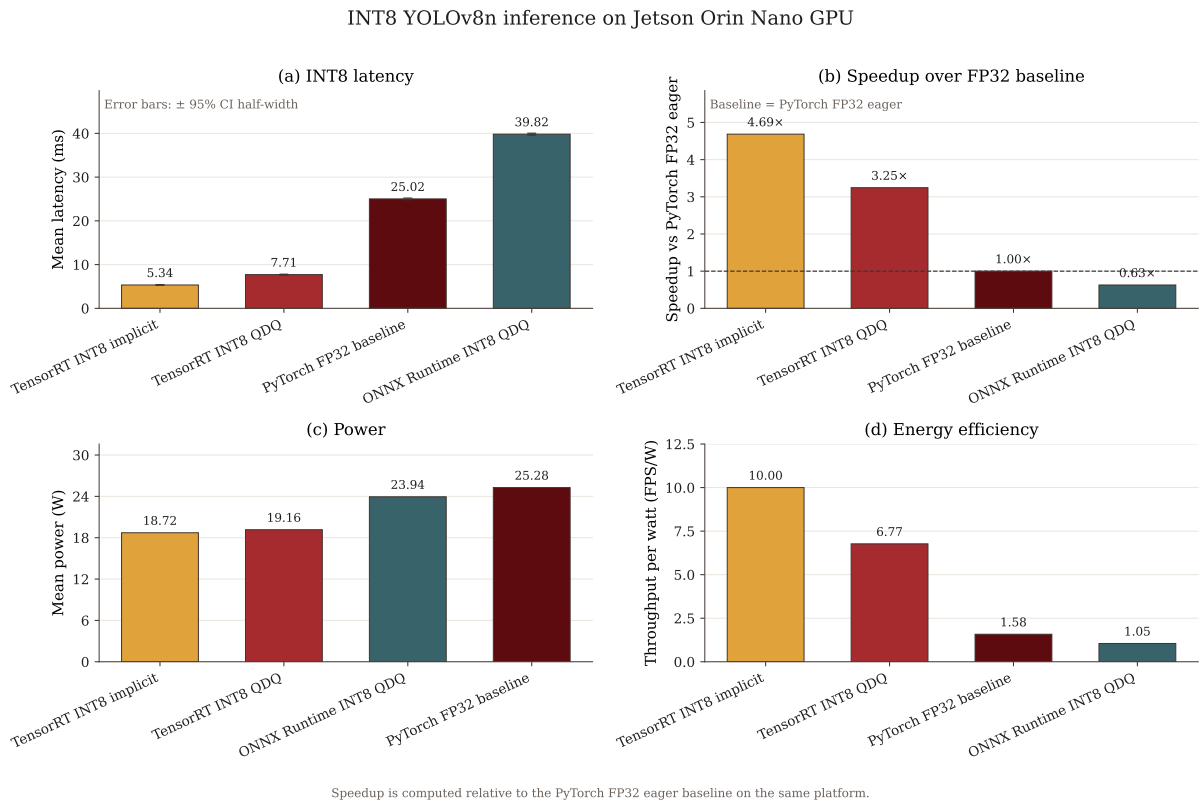

Figure 5.7: INT8 YOLOv8n inference performance on the NVIDIA Jetson Orin Nano GPU. Subfigure (a) shows mean end-to-end latency with error bars indicating the half-width of the 95% confidence interval. Subfigure (b) reports speedup relative to the PyTorch FP32 eager baseline, where values above 1.0× indicate faster execution than the baseline. Subfigure (c) compares mean board power consumption, and Subfigure (d) shows energy efficiency in terms of throughput per watt. TensorRT implicit INT8 achieves the best result, reaching the lowest latency, the highest speedup, and the highest throughput per watt. ONNX Runtime INT8 QDQ is slower than the PyTorch FP32 baseline despite using INT8 quantization.

mean latency and 187.23 FPS. This corresponds to a 4.69× speedup over the PyTorch FP32 eager baseline. ONNX Runtime INT8 QDQ, despite using an INT8 graph, was actually slower than the PyTorch FP32 baseline in the presented experiment, reaching approximately 39.82 ms per inference, or only 0.63× of the baseline speed. TensorRT FP16, discussed in the previous section, already provides a strong optimized reference at 6.31 ms, which means that the practical value of INT8 must be judged against both accuracy loss and the already highly optimized higher-precision TensorRT paths. The QDQ-based TensorRT engine reached 7.71 ms, which is still faster than PyTorch and ONNX Runtime, but notably slower than the best TensorRT implicit INT8 result.

The profiling data helps explain why these differences occur. For the implicit TensorRT INT8 engine, the NVTX summary shows that execution is dominated by the TensorRT enqueue stage, while the GPU kernel summary is largely composed of INT8 GEMM and IMMA-style convolution kernels. This indicates that TensorRT is able to map a large

Table 5.20: End-to-end YOLOv8n inference performance on the NVIDIA Jetson Orin Nano GPU. Values are reported as mean latency ± half-width of the 95% confidence interval across 5 isolated repetitions, with 200 timed runs per repetition after 30 warm-up iterations.

Backend	Latency (ms)	FPS mean	Median (ms)
PyTorch FP32 (baseline)	25.02 ± 0.18	39.96	25.10
ONNX Runtime INT8 QDQ	39.82 ± 0.21	25.11	39.82
TensorRT INT8 implicit	5.34 ± 0.05	187.23	5.34
TensorRT INT8 QDQ	7.71 ± 0.09	129.66	7.69

Table 5.21: Power and energy-efficiency results for selected YOLOv8n backends on the NVIDIA Jetson Orin Nano GPU.

Backend	Power mean (mW)	Power max (mW)	FPS/W
PyTorch FP32 (baseline)	25276	27646	1.58
ONNX Runtime INT8 QDQ	23941	27674	1.05
TensorRT INT8 implicit	18720	20466	10.00
TensorRT INT8 QDQ	19161	21678	6.77

fraction of the network to hardware-efficient INT8 kernels. At the same time, the profile still contains non-negligible reformat, copy, and permutation kernels, which shows that even the more successful implicit INT8 path does not achieve a fully clean low-precision execution pipeline.

For QDQ-based deployment, the profile is substantially more cluttered with quantization-related operations. Many NVTX ranges explicitly include QuantizeLinear/DequantizeLinear stages together with clone, copy, and tensor-reformat operations. This confirms that a significant portion of the runtime is spent not on useful convolution or detection work, but on managing low-precision data transformations between operators. In a model with many convolutional blocks and concatenation-heavy structures such as YOLOv8n, these additional graph boundaries accumulate quickly and can dominate the practical cost of the supposed INT8 path.

These results show that quantization cannot be evaluated only by checking whether a graph nominally uses INT8 values. What matters is whether the compiler and runtime can preserve low-precision execution through fused kernels and avoid repeated format conversions. In the present study, ONNX Runtime QDQ quantization produced limited or even negative efficiency benefits while still causing noticeable accuracy degradation. TensorRT QDQ improved latency relative to the PyTorch baseline, but the loss in accuracy was severe and the profile showed substantial conversion overhead. TensorRT implicit INT8 calibration produced the best speed among the tested INT8 approaches, but still degraded accuracy by roughly 10% in mAP50 and mAP50–95 relative to the FP32 baseline.

Overall, the results suggest that quantization for YOLOv8n on Jetson Orin Nano is highly deployment-path dependent. A naive or graph-heavy INT8 representation does not guarantee a practically useful result. In this work, the most effective TensorRT deployment path remained the standard optimized TensorRT engine in higher precision, while INT8 became attractive only when the hardware-native implicit TensorRT calibration path was used. Even then, the speed gain came with a clear drop in predictive quality. Quantization therefore appears here as a deployment trade-off rather than as a universally beneficial optimization.

Statistical significance was evaluated with a paired bootstrap test on validation images using 1000 resamples, and confidence intervals were computed with the percentile method. The bootstrap analysis confirmed that the evaluated INT8 variants produced a statistically significant decrease in detection accuracy relative to the FP32 baseline, with the strongest degradation observed for the TensorRT QDQ path.

5.6.2 INT8 Inference on Raspberry Pi 5 CPU

To evaluate the practical impact of integer quantization on an embedded CPU platform, additional benchmarks were performed on the Raspberry Pi 5 using INT8 YOLOv8n models. The comparison focused on two deployment frameworks that provided a usable CPU INT8 path in the present study: TFLite and ONNX Runtime. The experiments were executed with the same rigorous benchmarking procedure used in the previous sections: 5 isolated repetitions, 200 timed runs per repetition, 30 warm-up iterations, performance governor enabled, and active cooldown control between repetitions.

The resulting latency and power measurements are summarized in Table 5.22. Accuracy was evaluated on a 1000-image subset of the COCO validation split and is summarized in Table 5.23. The PyTorch FP32 model is used as the accuracy baseline.

Table 5.22: End-to-end INT8 YOLOv8n inference results on Raspberry Pi 5 CPU. Values are reported as mean latency ± half-width of the 95% confidence interval across 5 isolated repetitions, with 200 timed runs per repetition after 30 warm-up iterations.

Backend	Latency (ms)	FPS mean	Power (mW)	FPS/W
TFLite INT8	31.05 ± 0.17	32.20	2340	13.76
ONNX Runtime INT8	133.81 ± 2.31	7.47	4394	1.70

The corresponding relative accuracy changes are shown in Table 5.24.

A striking result is the large performance advantage of TFLite INT8. With a mean latency of only 31.05 ms, it is more than four times faster than ONNX Runtime INT8 and substantially faster than all previously evaluated FP32 and FP16 CPU configurations.

Table 5.23: Accuracy of INT8 YOLOv8n variants on Raspberry Pi 5 CPU. Relative changes are computed with respect to the PyTorch FP32 baseline on the same 1000-image validation subset.

Backend	Precision	Recall	mAP50	mAP50–95
PyTorch FP32 (baseline)	0.6291	0.4884	0.5384	0.3898
ONNX Runtime INT8 QDQ	0.5729	0.4801	0.5052	0.3557
TFLite INT8	0.6172	0.4450	0.5042	0.3486

Table 5.24: Relative accuracy change of INT8 backends on Raspberry Pi 5 CPU with respect to the PyTorch FP32 baseline.

Backend	Δ Prec. (%)	Δ Rec. (%)	Δ mAP50 (%)	Δ mAP50–95 (%)
ONNX Runtime INT8 QDQ	-8.93	-1.70	-6.16	-8.74
TFLite INT8	-1.89	-8.88	-6.35	-10.56

Compared with the earlier TFLite FP32/FP16 results (approximately 154 ms), the INT8 TFLite model achieves roughly a 5× speedup. At the same time, it also delivers by far the best energy efficiency, reaching 13.76 FPS/W.

This behavior is plausible on the Raspberry Pi 5 CPU. TFLite is strongly optimized for mobile and embedded processors and is able to execute quantized models using an efficient integer arithmetic path for a large portion of the network. In this case, both weights and activations are represented with lower precision, reducing memory traffic and improving cache utilization. On a CPU-bound platform such as Raspberry Pi 5, this can produce a much larger practical gain than would be expected from arithmetic reduction alone.

By contrast, ONNX Runtime uses a QDQ representation in which explicit `QuantizeLinear` and `DequantizeLinear` operators are inserted into the graph. If these conversions are not fully eliminated or fused during execution, a significant part of the runtime may be spent on tensor conversions rather than on useful convolution work. In a model such as YOLOv8n, which contains many convolution blocks, concatenation operations, and detection-head transformations, these graph boundaries can accumulate and reduce the practical benefit of quantization. This explains why ONNX Runtime INT8 still improves over its FP32 CPU path, but only modestly compared with TFLite.

From the accuracy perspective, both INT8 variants introduce a noticeable degradation relative to the PyTorch FP32 baseline. The mAP50 drop is similar for the two backends, around 6%, while the mAP50–95 reduction reaches 8.74% for ONNX Runtime and 10.56% for TFLite. The error pattern is not identical: ONNX Runtime loses more precision, whereas TFLite preserves precision better but shows a larger drop in recall. This suggests that the two deployment paths affect confidence scores and box regression

somewhat differently, even though both are based on INT8 inference.

On Raspberry Pi 5 CPU, Linux perf profiling showed that TFLite INT8 inference spent most active execution time inside clearly identifiable XNNPACK quantized microkernels, such as IGEMM/GEMM and LUT-based kernels. In contrast, ONNX Runtime INT8 required substantially more total cycles and instructions, achieved a lower IPC, and exhibited a higher LLC miss rate. This indicates that, for this model and deployment path, TFLite provided a more efficient CPU execution path, likely due to better-optimized low-level kernels and/or lower runtime overhead.

Overall, the Raspberry Pi 5 experiments show that quantization can deliver very large practical benefits on CPU, but only when the runtime can preserve a genuinely efficient integer execution path. In the evaluated setup, TFLite INT8 was the most effective deployment option by a large margin in both latency and energy efficiency. However, this gain came with a measurable reduction in detection accuracy, which means that INT8 deployment remains a trade-off between efficiency and predictive quality rather than a universally beneficial optimization.

5.7 Evaluation of YOLOv8n Pruning

Structured pruning was evaluated as a strategy to reduce the computational cost of YOLOv8n on edge hardware without requiring a change of inference framework or precision format. The motivation was to investigate whether selectively removing output channels from convolutional layers could yield measurable latency reductions while preserving acceptable detection accuracy. The investigation was split into two stages: a systematic per-layer sensitivity sweep to identify which layers are most and least sensitive to channel reduction, followed by a full-model pruning experiment with fine-tuning.

5.7.1 Structural Constraints of YOLOv8n

Applying structured pruning to YOLOv8n requires handling several architectural constraints. When an output channel is removed from a convolutional layer, the corresponding input channels of all downstream layers that consume that feature map must be removed as well. This dependency chain must be resolved correctly for every targeted layer.

The most complex case is the `C2f` block illustrated in Figure 5.8 (used in `model.2`, `model.4`, `model.6`, `model.8`, and the detection neck). Each `C2f` block contains two boundary convolutions (`cv1`, `cv2`) and n bottleneck units whose outputs are concatenated along the channel dimension before being passed to `cv2`. Pruning the output channels of `cv1` affects

the inputs of every bottleneck; pruning a bottleneck's output affects the concatenated tensor feeding into `cv2`. This means that channel reduction in any `C2f` constituent requires consistent propagation through all branches of the block simultaneously. The shortcut connections within the bottleneck units impose a further constraint: the two branches of each residual addition must maintain equal channel counts, which limits the granularity at which individual components can be pruned independently.

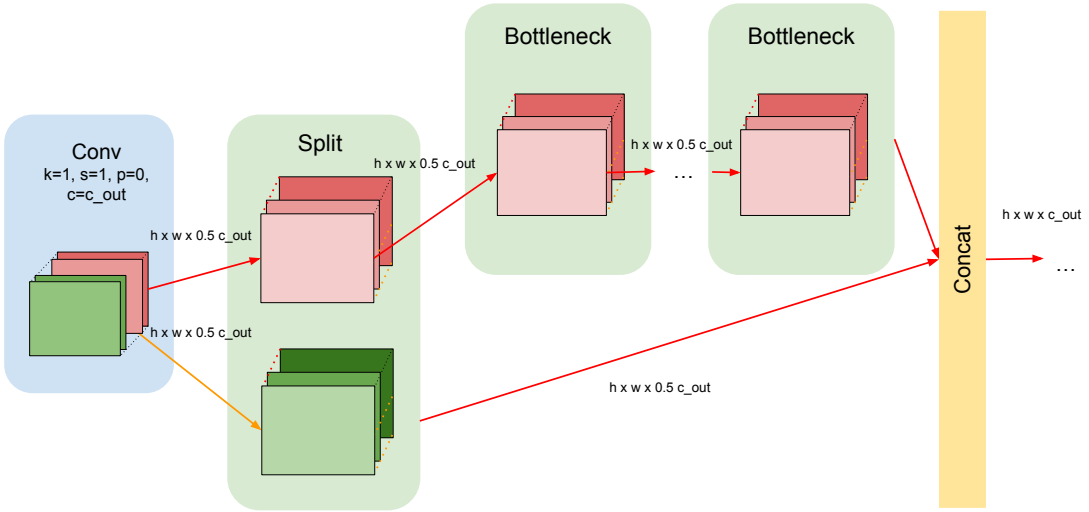

Figure 5.8: Data flow and channel dependencies within the C2f block.

Handling these dependencies required a custom propagation pass that, for each targeted layer, identified all dependent downstream layers and adjusted their weight tensors accordingly.

5.7.2 Experimental Setup

Pruning experiments were split across two machines depending on whether retraining was required. The per-layer sensitivity sweep, which involves no gradient computation, was run on the Jetson Orin Nano using PyTorch FP32 with the CUDA backend. The full-model pruning with fine-tuning and the final benchmarks were run on a server equipped with an NVIDIA GeForce RTX 3070 GPU (`cuda:0`). All inference measurements used PyTorch FP32, batch size 1. The target model in all cases was the standard YOLOv8n checkpoint trained on COCO, with 3 157 200 parameters and 8.7 GFLOPs. Detection accuracy was evaluated on the COCO 2017 validation set; the unmodified baseline achieved mAP50-95 of 0.3715 and mAP50 of 0.5211.

Per-Layer Sensitivity Sweep. A dedicated script (`layer\_speed\_sensitivity.py`) swept 38 convolutional layers at sparsity levels from 10 % to 99 % in roughly 10 % steps, covering every `cv1`/`cv2` boundary and all bottleneck internals across the backbone and detection neck. The sweep was executed on the Jetson Orin Nano using PyTorch FP32 with the CUDA backend and `--no-preload` to avoid saturating the device's unified CPU/GPU memory pool. The baseline inference latency on this platform was 25.1 ms (batch size 1, 300 timed runs after 100 warm-up iterations), consistent with the PyTorch FP32 baseline reported in the backend comparison (Table 5.8). Accuracy was evaluated on up to 1 000 COCO validation images per configuration. For each configuration, output channels were selected for removal by ranking them according to their L2 weight norm (magnitude-based criterion); channels with the smallest norms were discarded first. Only the `any` parity mode (nearest integer to the target) was used, yielding approximately 418 configurations in total. Each configuration was evaluated without any subsequent fine-tuning so that the latency change and accuracy drop could be attributed purely to the structural modification rather than to retraining.

Full-Model Pruning with Fine-Tuning. A separate experiment, conducted on the RTX 3070 server, applied magnitude-based channel pruning across multiple layers simultaneously, selecting the combination of layers that appeared most resilient in the sensitivity sweep. The pruned model was then fine-tuned for 50 epochs on a 1 % subset of the COCO training set using the standard Ultralytics training pipeline with half-precision gradients (`amp=true`). The fine-tuned checkpoint was benchmarked against both the same model before pruning and the stock YOLOv8n baseline, with 200 timed runs after 30 warm-up iterations.

5.7.3 Per-Layer Sensitivity Results

Table 5.25 summarises the speedup and accuracy impact for selected sparsity levels on three representative layers from the Jetson Orin Nano sweep. The full sweep covered 418 configurations; only the most informative points are shown.

The results reveal a consistent pattern across all 38 targeted layers: channel reduction provides essentially no latency benefit. Across all 418 configurations, the speedup ratio ranged from 0.975 to 1.021, with a mean of approximately 1.005. No configuration exceeded a 1.05× speedup, and the distribution was centred near 1.0, indicating that latency is statistically unchanged by channel reduction.

The accuracy side of the tradeoff was more consistent: pruning aggressively degrades detection quality. Removing 50 % of the channels from `model.18.cv2` caused a 0.114 point drop in mAP50-95 (29 % relative degradation) while yielding only a 1.5 % latency

Table 5.25: Per-layer pruning sensitivity on Jetson Orin Nano (PyTorch FP32, CUDA, batch 1, no fine-tuning). Speedup is relative to the unpruned baseline (25.1 ms). Accuracy drop is absolute change in mAP50-95 relative to baseline (0.392).

Layer	Sparsity	Kept ch	Speedup	Latency (ms)	ΔmAP50-95
`model.21.cv2` (256 ch)	0 %	256	1.016×	24.66	+0.00
	20 %	205	1.010×	24.75	−0.023
	50 %	128	1.011×	24.74	−0.057
	70 %	77	1.010×	24.77	−0.125
	90 %	26	1.013×	24.77	−0.176
`model.18.cv2` (128 ch)	0 %	128	1.011×	24.74	+0.00
	20 %	102	1.016×	24.60	−0.012
	50 %	64	1.015×	24.64	−0.114
	70 %	38	1.002×	24.96	−0.197
	90 %	13	1.015×	24.65	−0.260
`model.2.cv1` (16 ch)	12.5 %	14	1.005×	25.00	−0.362
	50.0 %	8	1.009×	24.93	−0.389
	68.75 %	5	1.011×	24.87	−0.392

improvement. Removing 70 % of channels from the same layer reduced mAP50-95 by 0.197 points (50 % relative) with essentially no latency gain. The earliest backbone layer (`model.2.cv1`) proved particularly fragile: even removing just 12.5 % of its channels caused a 0.362 point mAP50-95 collapse, consistent with the role of early layers in forming the low-level feature representations that all subsequent computations depend upon.

Most Resilient Layers. The layers with the smallest accuracy impact at moderate sparsity were concentrated in the detection neck and later backbone stages. Table 5.26 lists the ten most resilient layers identified in the Orin Nano sweep.

These are predominantly the bottleneck internals and upsample-path convolutions of the detection neck. By contrast, the backbone's early layers (`model.2.cv1`, `model.4.cv1`) incurred accuracy drops exceeding 90 % of baseline mAP at comparable sparsity levels, confirming that sensitivity decreases with network depth.

One notable observation on the Orin is the exceptional tolerance of `model.21.cv1` (128 ch): even at 99 % channel removal (keeping a single output channel), mAP50-95 settled at 0.211 - a 46 % relative drop, but far less than the near-total collapse seen in backbone layers at comparable sparsity. This layer is a `cv1` projection inside the last C2f block of the FPN and contributes less directly to the final feature representation than earlier stages; its redundancy is unusually high. Despite this, the corresponding latency speedup at 99 % sparsity was only 1.014× (24.67 ms vs. 25.1 ms baseline), confirming that even an extreme structural reduction does not translate to a meaningful runtime

Table 5.26: Ten most accuracy-resilient layers in YOLOv8n under single-layer magnitude pruning without fine-tuning (Jetson Orin Nano, COCO val 1 000-image subset, mAP50-95 baseline 0.3921, evaluated at the closest available sparsity step to 12.5 %).

Layer	Pruned mAP50-95	ΔmAP50-95
`model.12.m.0.cv1`	0.3915	−0.0006 (−0.14 %)
`model.15.m.0.cv2`	0.3906	−0.0015 (−0.38 %)
`model.12.m.0.cv2`	0.3905	−0.0015 (−0.39 %)
`model.16`	0.3902	−0.0018 (−0.47 %)
`model.21.cv2`	0.3901	−0.0020 (−0.51 %)
`model.15.m.0.cv1`	0.3901	−0.0020 (−0.51 %)
`model.19`	0.3892	−0.0028 (−0.72 %)
`model.18.m.0.cv2`	0.3892	−0.0028 (−0.73 %)
`model.6.m.1.cv1`	0.3891	−0.0029 (−0.75 %)
`model.21.m.0.cv2`	0.3889	−0.0032 (−0.82 %)

improvement.

5.7.4 Full-Model Pruning Results

The multi-layer pruned and fine-tuned model was benchmarked against both its pre-pruning counterpart and the stock YOLOv8n baseline. Table 5.27 presents the three-way comparison.

Table 5.27: Full-model pruning results on NVIDIA GeForce RTX 3070 (PyTorch FP32, batch 1). Benchmark: 200 timed runs, 30 warm-up iterations. Accuracy: COCO 2017 val. *Stock baseline*: official Ultralytics YOLOv8n pretrained weights. *Fine-tune base (unpruned)*: full-width YOLOv8n fine-tuned for 50 epochs on 1 % of COCO — same training procedure as the pruned model but without channel removal; used as a fair paired comparison. *Pruned + fine-tuned*: structurally pruned model (selected channels removed) then fine-tuned for 50 epochs under the same schedule.

Model	mAP50-95	mAP50	Mean lat. (ms)	FPS
Stock YOLOv8n (pretrained)	0.3714	0.5212	6.18	161.9
Fine-tune base (unpruned, 50 ep)	0.3588	0.5067	6.00	166.6
Pruned + fine-tuned (50 ep)	0.3446	0.4897	5.85	170.9
Δ pruned vs. stock pretrained	−0.027 (−7.2 %)	−0.032	+0.28 ms (+4.6 %)	−7.1
Δ pruned vs. fine-tune base	−0.014 (−4.0 %)	−0.017	−0.15 ms (−2.3 %)	+4.3

Compared to its direct paired counterpart - the same fine-tuned model without channel removal - the pruned model was 2.3 % faster (5.85 ms vs. 6.00 ms) and lost 0.014 mAP50-95 (4.0 % relative). Compared to the stock YOLOv8n baseline, the pruned model was slower by 4.6 % (5.85 ms vs. 6.18 ms) while losing 7.2 % of mAP50-95. The higher absolute latency of the stock baseline in this benchmark (6.18 ms) compared to the Orin

Nano sensitivity sweep (25.1 ms, different device) reflects both the different hardware target and the benchmarking methodology: the RTX 3070 comparison used 200 timed runs with 30 warm-up iterations under higher scheduling variance, whereas the Orin sweep used 300 runs after 100 warm-up iterations in a more controlled per-layer environment.

5.7.5 Discussion

The pruning experiments produced a clear and consistent negative result: structured channel pruning of YOLOv8n does not yield meaningful latency reductions, whether measured on the Jetson Orin Nano (CUDA sensitivity sweep) or the RTX 3070 (fine-tuned model benchmarks), when operating in PyTorch FP32. The multi-platform agreement rules out the possibility that the null result is an artefact of a particular runtime or instruction-set alignment, and instead points to a hardware-agnostic root cause.

The fundamental reason lies in how cuDNN dispatches convolution kernels. As shown by the Roofline analysis in Section 5.2, the majority of convolutional layers on the Orin Nano GPU operate in the compute-bound regime, achieving several hundred GFLOP/s with high arithmetic intensity. In principle, reducing the channel count lowers the FLOP count and should bring a compute-bound layer closer to its throughput ceiling. In practice, however, cuDNN selects tiled GEMM or Winograd kernels based on the original tensor dimensions, and those kernels use fixed warp-level tiling strategies that do not shrink proportionally with modest channel reductions. A layer with 10 % fewer channels launches the same kernel, with the same grid geometry and shared-memory footprint, but simply leaves a fraction of the warp lanes idle. The actual reduction in execution time only manifests when the channel count drops far enough to trigger a different kernel configuration — a threshold that the sparsity levels explored here (up to $\sim$90 %) rarely cross cleanly. This is confirmed by the sweep data: speedups cluster at 1.00 ± 0.02 across all 418 configurations, regardless of sparsity level.

A secondary issue is that YOLOv8n is already a compact model (3.15 M parameters, 8.7 GFLOPs). Compact models leave less redundancy to remove, and the accuracy-speed tradeoff degrades rapidly. The most resilient layers (the detection-neck bottlenecks) offer the smallest accuracy drop at moderate sparsity, but their absolute channel counts are already modest and the channel reductions applied here fall below the threshold at which cuDNN would select a meaningfully different kernel, so removing channels from them provides essentially no latency benefit.

In summary, structured channel pruning applied to the PyTorch FP32 representation of YOLOv8n is not an effective standalone optimization on either hardware target evaluated. The per-layer sweep on the Jetson Orin Nano confirmed a speedup ceiling of approximately 1.02× across 418 configurations. The full-model pruning on the RTX 3070

achieved at most 2.3 % relative speedup against its paired fine-tuned counterpart, and that gain came at a disproportionate accuracy cost (4.0 % mAP50-95 drop). The comparison against the stock baseline shows a net regression in both metrics. The quantization results reported in Section 5.6.2 - where LiteRT INT8 on the Raspberry Pi 5 achieved a 9.4× speedup over the PyTorch eager baseline - illustrate that precision reduction is a more effective lever than structural pruning for this class of model and hardware.

It should be noted that all pruning experiments reported here used PyTorch FP32 inference. It is possible that TensorRT's layer-fusion and kernel-selection passes could extract a modest additional benefit from a reduced channel count, since the engine builder operates on the pruned graph directly rather than relying on the runtime to skip channels. However, given the cuDNN kernel-granularity limitation and the already negligible speedups observed in PyTorch, a substantial gain from that path is not expected.

5.8 GPU Kernel-Level Performance Analysis

To better understand the performance characteristics of the optimized YOLOv8n inference pipeline, kernel-level profiling was performed using NVIDIA Nsight Systems and NVIDIA Nsight Compute on the Jetson Orin Nano platform. The profiling compared TensorRT engines executed in FP32 and INT8 precision modes.

The results provide insight into the underlying execution behaviour of the GPU kernels and reveal several key performance bottlenecks and optimization opportunities.

5.8.1 Kernel Structure of YOLOv8n Inference

Nsight Compute profiling revealed that YOLOv8n inference consists of approximately 35–40 unique GPU kernels executed repeatedly during inference. These kernels can be categorized into several functional groups:

- Tensor Core convolution kernels
- Tensor format conversion kernels
- Tensor permutation kernels
- Pointwise elementwise kernels
- Auxiliary operations such as pooling and softmax

The dominant compute kernels correspond to Tensor Core convolution implementations generated by TensorRT, typically with names such as

```
sm80_xmma_fprop_implicit_gemm_interleaved_i8i8_i8i32_f32
```

which represent implicit GEMM implementations of convolution layers optimized for Ampere Tensor Cores.

INT8 inference uses kernels such as:

```
trt_ampere_int8x4_icudnn_int8x4
```

which exploit INT8 Tensor Core instructions.

5.8.2 Observed Performance Characteristics

The benchmark results, summarized from the measurements reported in Tables 5.8 and 5.20, show the following end-to-end latency for TensorRT-compiled YOLOv8n on the Jetson Orin Nano:

Precision	Latency (ms)	FPS
TensorRT FP32	11.38	87.87
TensorRT INT8	5.34	187.23

This corresponds to a speedup of approximately 2.13×, when switching from TensorRT FP32 to TensorRT INT8 implicit calibration. The PyTorch FP32 eager baseline reached 25.02 ms (39.96 FPS), so the TensorRT INT8 engine provides a roughly 4.7× improvement over the uncompiled reference, while the TensorRT FP32 engine alone contributes a 2.2× uplift over PyTorch.

Despite this improvement, the profiling results show that GPU hardware utilization is still far from optimal.

5.8.3 Occupancy Limitations

Many Tensor Core convolution kernels exhibit very low theoretical occupancy, often in the range of $16\% - 33\%$.

This behaviour is caused primarily by high register pressure and large shared memory requirements. For example, several kernels required more than 240 registers per thread and over 80 kB of shared memory per block.

High register pressure and large shared memory requirements reduce the number of thread blocks that can be scheduled simultaneously on a Streaming Multiprocessor (SM).

Since registers and shared memory are limited hardware resources, each thread block must reserve the required amount before execution can begin.

For example, if a kernel requires a large number of registers per thread or a substantial amount of shared memory per block, only a small number of blocks can reside on the SM at the same time. This limits the number of active warps and therefore reduces the theoretical occupancy of the kernel.

Low occupancy can negatively affect GPU performance because GPUs rely on warp-level parallelism to hide memory and instruction latency. When one warp stalls due to memory access or pipeline dependencies, the scheduler switches execution to another ready warp. However, if only a small number of warps are active due to resource constraints, the scheduler has fewer opportunities to hide these latencies. As a result, the compute units remain idle for longer periods, leading to reduced hardware utilization.

5.8.4 Kernel Launch Granularity

Another recurring issue observed in the profiling results is the small grid size of many kernels. Several kernels were launched with grid sizes smaller than the number of available Streaming Multiprocessors.

For example, kernels with grid sizes of 7 blocks were observed on a GPU with 8 SMs. That means 12.5% of the GPU is unused for the entire kernel. In such cases the GPU cannot fully utilize all available compute resources.

This effect is commonly referred to as the tail effect and is particularly visible when performing inference with batch size 1.

5.8.5 Memory Layout Transformation Overhead

A substantial portion of execution time is spent in kernels responsible for tensor layout transformations and format conversions. Examples include:

```
1 genericReformat::copyVectorizedKernel = layout/type conversion copy
2 CUTENSOR_NAMESPACE::permutationKernel = tensor transpose / dimension reorder /
     permutation
3 cuInt8::nc32hw32ToNc32hw32 = INT8 packed-layout repacking inside TensorRT's
     channel-blocked formats
```

These kernels do not perform useful neural network computation but instead move or reorder tensor data between different memory layouts required by TensorRT kernels.

Several of these kernels exhibit high memory throughput utilization (often above 70%),

indicating that they are primarily memory-bandwidth bound operations.

```
genericReformat::copyVectorizedKernel
```

5.8.6 Pointwise Kernel Fragmentation

Another common pattern is the presence of many small pointwise kernels such as:

```
generatedNativePointwise
```

These kernels implement operations such as activation functions, elementwise additions, or scaling operations.

While each kernel individually executes quickly, the large number of such kernels introduces kernel launch overhead and reduces overall GPU efficiency.

5.8.7 Overall Performance Bottlenecks

The profiling results suggest that YOLOv8n inference performance is limited by a combination of factors rather than a single dominant bottleneck.

The most significant contributors are:

- Tensor layout conversion overhead
- High register pressure in Tensor Core kernels
- Small kernel grid sizes due to batch size 1
- Fragmentation into many small pointwise kernels
- Moderate compute utilization across many kernels

These observations indicate that the GPU is neither purely compute-bound nor purely memory-bound. Instead, inference performance is limited by kernel launch granularity and resource-heavy kernel implementations.

5.8.8 Future Optimization Opportunities

Based on the kernel-level profiling analysis presented above, several optimization directions can be identified for further improving YOLO inference performance on the Jetson Orin Nano.

Operator Fusion

A large number of small pointwise kernels suggests that additional operator fusion could significantly reduce kernel launch overhead. Fusing activation, normalization, and elementwise operations into larger kernels can reduce both kernel count and memory traffic.

Compiler frameworks such as TVM or TensorRT graph optimizations may enable such transformations.

Reducing Tensor Layout Conversions

Memory layout transformation kernels represent a non-negligible portion of total inference time. Reducing the number of tensor format conversions can therefore improve performance.

Possible approaches include:

- ensuring consistent tensor layouts across the network
- minimizing precision conversions
- optimizing tensor format selection during engine compilation

Batch Size Optimization

Small kernel grid sizes observed during profiling indicate that batch size 1 inference does not fully utilize GPU resources.

Increasing the batch size can improve SM utilization and reduce the tail effect. However, this may not be acceptable for latency-sensitive real-time applications.

Model Architecture Optimization

Model architecture also influences GPU efficiency. Potential improvements include:

- reducing small tensor operations
- increasing arithmetic intensity
- restructuring layers to improve GPU utilization

Architectural modifications specifically designed for edge GPUs could further improve throughput.

Compiler-Level Optimization

Given the hardware-aware optimization focus of this work, compiler frameworks such as TVM provide an opportunity for further performance improvements.

Potential directions include:

- auto-tuned convolution schedules
- optimized memory layouts
- kernel fusion
- reduced kernel launch overhead

Compiler-generated kernels may reduce the register pressure and shared memory usage observed in TensorRT kernels.

5.8.9 Summary

The kernel-level profiling results confirm that TensorRT compilation combined with INT8 quantization provides substantial inference improvements for YOLOv8n on the Jetson Orin Nano. TensorRT FP32 reduces mean latency from 25.02 ms (PyTorch baseline) to 11.38 ms (2.2× speedup), while TensorRT INT8 implicit calibration achieves 5.34 ms and 187.23 FPS (4.7× over the PyTorch baseline, 2.13× over TensorRT FP32). FP16 inference, evaluated separately, reached 6.31 ms, positioning INT8 as the fastest measured path despite a roughly 10% reduction in mAP50 relative to FP32.

However, substantial optimization potential remains. The profiling shows that even the fastest path is affected by tensor layout conversion overhead, small kernel grid sizes at batch size 1, and high register pressure in Tensor Core kernels. Future work should focus on reducing format-reformat overhead through consistent layout selection during engine compilation, improving operator fusion to consolidate small pointwise kernels, and exploring compiler-driven scheduling (e.g. TVM auto-tuning) to lower register pressure and better exploit the eight Streaming Multiprocessors of the Orin Nano GPU.

5.9 Design and Implementation of a TensorRT Plugin for the C2f Block

The implementation is complete; results are undergoing final validation and will be reported here.

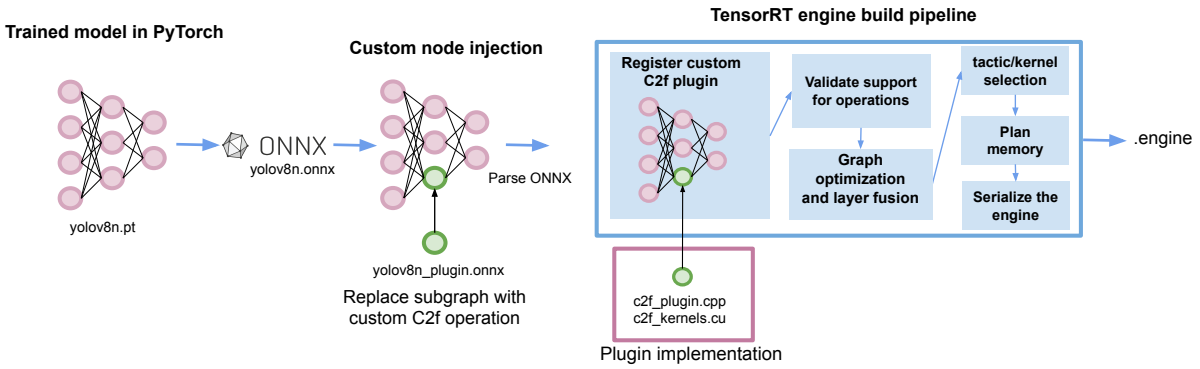

Figure 5.9: TensorRT engine build workflow with custom plugin injection. A trained YOLOv8 model is exported from PyTorch to ONNX, a selected C2f subgraph is replaced by a custom plugin node, and TensorRT builds a target-specific engine by parsing the modified graph, validating supported operators and registered plugins, applying graph optimizations, selecting tactics, planning memory, and serializing the final engine

5.10 Evaluation of Activation Function Simplification: SiLU vs. ReLU

The implementation is complete; results are undergoing final validation and will be reported here.

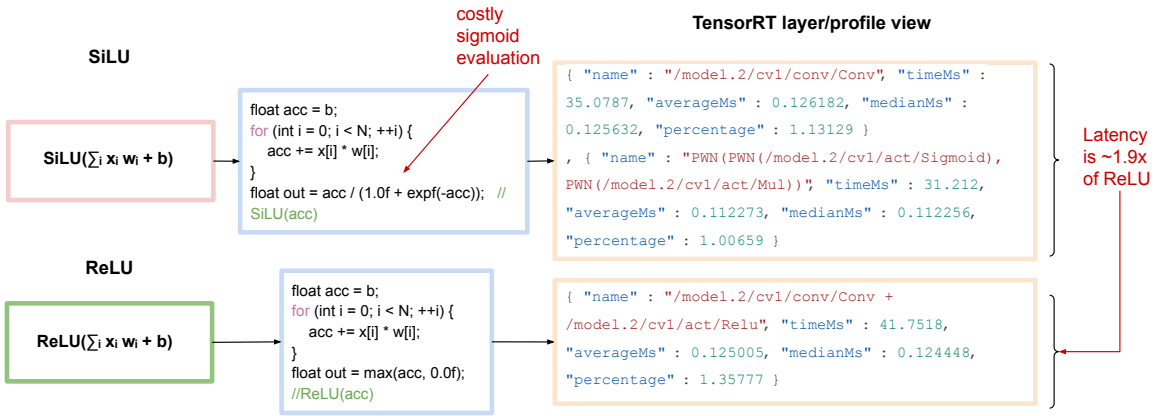

Figure 5.10: Comparison of TensorRT per-layer profiling for convolution followed by SiLU and ReLU. In the ReLU case, TensorRT reports a fused Conv + Relu region, whereas in the SiLU case the activation appears as additional pointwise Sigmoid and Mul operations. For the profiled example, the convolution time remains similar, but the extra SiLU activation work increases the total layer cost to approximately 1.9× that of the fused ReLU case.

6 Discussion

6.1 Discussion Points

6.1.1 Quantization and accelerator deployment

6.1.2 Unexplored hardware configurations and deployment possibilities

7 Conclusion

8 Future Work

9 Description of Use of Artificial Intelligence

In working on this Master's Thesis, I have used artificial intelligence (AI) tools in the following limited ways. I used ChatGPT (OpenAI) to assist with idea development, clarifying concepts, and improving the structure and language of the text (for example, by suggesting alternative formulations, checking grammar, and helping to simplify explanations). AI was also used to get help with technical formatting issues in LaTeX and to debug small code examples by explaining error messages and proposing corrections.

All scientific content, including the problem formulation, choice of methods, implementation, experiments, analysis, and conclusions, has been developed, reviewed, and academically assessed by me. AI tools were not used to generate entire sections of the report, to perform literature searches independently, or to answer the assignment in its entirety. I have not uploaded sensitive or confidential information to the AI tools. I am solely responsible for the academic content of this assignment.

References

[1] Martín Abadi et al. "{TensorFlow}: a system for {Large-Scale} machine learning". In: *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 2016, pp. 265–283.

[2] Apache TVM. *Apache TVM*. Accessed: 2026-04-12. 2026. URL: `https://tvm.apache.org/`.

[3] Amir H Ashouri et al. "A survey on compiler autotuning using machine learning". In: *ACM Computing Surveys (CSUR)* 51.5 (2018), pp. 1–42.

[4] Dimitris Bertsimas and John Tsitsiklis. "Simulated annealing". In: *Statistical science* 8.1 (1993), pp. 10–15.

[5] Tianqi Chen et al. "{TVM}: An automated {End-to-End} optimizing compiler for deep learning". In: *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 2018, pp. 578–594.

[6] Jonathan Frankle and Michael Carbin. "The lottery ticket hypothesis: Finding sparse, trainable neural networks". In: *arXiv preprint arXiv:1803.03635* (2018).

[7] Jonathan Frankle et al. "Stabilizing the lottery ticket hypothesis". In: *arXiv preprint arXiv:1903.01611* (2019).

[8] Roy Frostig, Matthew James Johnson, and Chris Leary. "Compiling machine learning programs via high-level tracing". In: *SysML conference 2018*. 2019.

[9] David E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley Professional, 1989. URL: `https://www2.fiit.stuba.sk/~kvasnicka/Free%20books/Goldberg_Genetic_Algorithms_in_Search.pdf`.

[10] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: `https://www.deeplearningbook.org/`.

[11] Ariel Gordon et al. "Morphnet: Fast & simple resource-constrained structure learning of deep networks". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 1586–1595.

[12] Yiwen Guo, Anbang Yao, and Yurong Chen. "Dynamic network surgery for efficient dnns". In: *Advances in neural information processing systems* 29 (2016).

[13] Song Han et al. "Learning both weights and connections for efficient neural network". In: *Advances in neural information processing systems* 28 (2015).

[14] Yang He et al. “Filter pruning via geometric median for deep convolutional neural networks acceleration”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 4340–4349.

[15] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. “Distilling the knowledge in a neural network”. In: *arXiv preprint arXiv:1503.02531* (2015).

[16] Hengyuan Hu et al. “Network trimming: A data-driven neuron pruning approach towards efficient deep architectures”. In: *arXiv preprint arXiv:1607.03250* (2016).

[17] Nathan Hubens et al. “An experimental study of the impact of pre-training on the pruning of a convolutional neural network”. In: *Proceedings of the 3rd International Conference on Applications of Intelligent Systems*. 2020, pp. 1–6.

[18] IREE. *IREE: Intermediate Representation Execution Environment*. Accessed: 2026-04-12. 2026. URL: `https://iree.dev/`.

[19] Benoit Jacob et al. “Quantization and training of neural networks for efficient integer-arithmetic-only inference”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 2704–2713.

[20] Zhe Jia et al. “Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking”. In: *arXiv preprint arXiv:1804.06826* (2018). URL: `https://arxiv.org/abs/1804.06826`.

[21] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. Version 8.0.0. 2023. URL: `https://github.com/ultralytics/ultralytics`.

[22] Chris Lattner and Vikram Adve. “LLVM: A compilation framework for lifelong program analysis & transformation”. In: *International symposium on code generation and optimization, 2004. CGO 2004.* IEEE. 2004, pp. 75–86.

[23] Chris Lattner et al. “MLIR: Scaling compiler infrastructure for domain specific computation”. In: *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE. 2021, pp. 2–14.

[24] Namhoon Lee, Thalaiyasingam Ajanthan, and Philip HS Torr. “Snip: Single-shot network pruning based on connection sensitivity”. In: *arXiv preprint arXiv:1810.02340* (2018).

[25] Hao Li et al. “Pruning filters for efficient convnets”. In: *arXiv preprint arXiv:1608.08710* (2016).

[26] Mingzhen Li et al. “The deep learning compiler: A comprehensive survey”. In: *IEEE Transactions on Parallel and Distributed Systems* 32.3 (2020), pp. 708–727.

[27] Mingbao Lin et al. “Filter sketch for network pruning”. In: *IEEE Transactions on Neural Networks and Learning Systems* 33.12 (2021), pp. 7091–7100.

[28] Ning Liu et al. “Autocompress: An automatic dnn structured pruning framework for ultra-high compression rates”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34. 04. 2020, pp. 4876–4883.

[29] LLVM Project. *Multi-Level Intermediate Representation (MLIR) Overview*. Accessed: 2026-04-12. 2026. URL: https://mlir.llvm.org/.

[30] Jian-Hao Luo, Jianxin Wu, and Weiyao Lin. “Thinet: A filter level pruning method for deep neural network compression”. In: *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 5058–5066.

[31] Stefano Markidis et al. “NVIDIA Tensor Core Programmability, Performance & Precision”. In: *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 2018, pp. 522–531. DOI: 10.1109/IPDPSW.2018.00091.

[32] Microsoft. *ONNX GitHub repository*. https://github.com/onnx/onnx. Accessed: 2026-04-26. 2017.

[33] Iman Mirzadeh et al. “Relu strikes back: Exploiting activation sparsity in large language models”. In: *arXiv preprint arXiv:2310.04564* (2023).

[34] Volodymyr Mnih et al. “Human-level control through deep reinforcement learning”. In: *Nature* 518.7540 (2015), pp. 529–533. DOI: 10.1038/nature14236.

[35] Pavlo Molchanov et al. “Pruning convolutional neural networks for resource efficient inference”. In: *arXiv preprint arXiv:1611.06440* (2016).

[36] Vinod Nair and Geoffrey E. Hinton. “Rectified Linear Units Improve Restricted Boltzmann Machines”. In: *Proceedings of the 27th International Conference on Machine Learning (ICML)* (2010).

[37] NVIDIA. *Jetson Orin Nano Developer Kit datasheet*. NVIDIA Datasheet. Accessed: 2026-02-16. URL: https://nvdam.widen.net/s/zkfqjmtds2/jetson-orin-datasheet-nano-developer-kit-3575392-r2.

[38] NVIDIA. *NVIDIA TensorRT Documentation*. 2026. URL: https://docs.nvidia.com/deeplearning/tensorrt/latest/index.html (visited on 04/23/2026).

[39] NVIDIA. *Optimizing TensorRT Performance*. 2026. URL: https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/optimization.html (visited on 04/23/2026).

[40] ONNX Runtime. *ONNX Runtime Documentation*. Accessed: 2026-04-12. 2026. URL: https://onnxruntime.ai/docs/.

[41] ONNX Runtime. *ONNX Runtime Execution Providers*. Accessed: 2026-04-12. 2026. URL: https://onnxruntime.ai/docs/execution-providers/.

[42] OpenXLA Project. *HLO*. https://openxla.org/xla/hlo_to_thunks. Accessed: 2026-04-26. 2026.

[43] OpenXLA Project. *StableHLO*. https://openxla.org/stablehlo. Accessed: 2026-04-26. 2024.

[44] OpenXLA Project. *XLA Architecture*. Accessed: 2026-04-12. 2024. URL: https://openxla.org/xla/architecture.

[45] OpenXLA Project. *XLA Tooling*. Accessed: 2026-04-12. 2026. URL: https://openxla.org/xla/tools.

[46] Adam Paszke et al. "Pytorch: An imperative style, high-performance deep learning library". In: *Advances in neural information processing systems* 32 (2019).

[47] Adam Polyak and Lior Wolf. "Channel-level acceleration of deep face representations". In: *IEEe Access* 3 (2015), pp. 2163–2175.

[48] Python Software Foundation. *Thread State and the Global Interpreter Lock*. Python/C API Reference Manual, accessed 2026-04-24. Python Software Foundation. 2026. URL: https://docs.python.org/3/c-api/threads.html.

[49] PyTorch Foundation. *PyTorch 2.x*. Accessed: 2026-04-12. 2025. URL: https://pytorch.org/get-started/pytorch-2-x/.

[50] PyTorch Foundation. *torch.compiler*. Accessed: 2026-04-12. 2025. URL: https://docs.pytorch.org/docs/stable/user_guide/torch_compiler/torch.compiler.html.

[51] PyTorch Foundation. *torch.compiler FAQ*. Accessed: 2026-04-12. 2025. URL: https://docs.pytorch.org/docs/stable/user_guide/torch_compiler/torch.compiler_faq.html.

[52] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. "Searching for Activation Functions". In: *arXiv preprint arXiv:1710.05941* (2017).

[53] RangeKing. *Brief summary of YOLOv8 model structure (Issue #189)*. GitHub. Ultralytics/ultralytics repository. 2023. URL: https://github.com/ultralytics/ultralytics/issues/189.

[54] Raspberry Pi Foundation. *Raspberry Pi processors*. Raspberry Pi Documentation. Accessed: 2026-02-16. URL: https://www.raspberrypi.com/documentation/computers/processors.html.

[55] Mohammad Rastegari et al. "Xnor-net: Imagenet classification using binary convolutional neural networks". In: *European conference on computer vision*. Springer. 2016, pp. 525–542.

[56] James Reed et al. "torch. fx: Practical program capture and transformation for deep learning in python". In: *Proceedings of Machine Learning and Systems* 4 (2022), pp. 638–651.

[57] Nadav Rotem et al. “Glow: Graph lowering compiler techniques for neural networks”. In: *arXiv preprint arXiv:1805.00907* (2018).

[58] TensorFlow. *TensorFlow MLIR Overview*. Accessed: 2026-04-12. 2021. URL: https://www.tensorflow.org/mlir/overview.

[59] Arun Thangamani, Vincent Loechner, and Stéphane Genaud. “A survey of general-purpose polyhedral compilers”. In: *ACM Transactions on Architecture and Code Optimization* 21.4 (2024), pp. 1–26.

[60] Ultralytics. *YOLOv8 Performance Metrics*. Ultralytics YOLOv8 documentation. Accessed 2026. URL: https://docs.ultralytics.com/models/yolov8/#performance-metrics.

[61] Sunil Vadera and Salem Ameen. “Methods for pruning deep neural networks”. In: *Ieee Access* 10 (2022), pp. 63280–63300.

[62] Chaoqi Wang, Guodong Zhang, and Roger Grosse. “Picking winning tickets before training by preserving gradient flow”. In: *arXiv preprint arXiv:2002.07376* (2020).

[63] Samuel Williams, Andrew Waterman, and David Patterson. “Roofline: An Insightful Visual Performance Model for Multicore Architectures”. In: *Communications of the ACM* 52.4 (2009), pp. 65–76.

[64] Hongbin Zhang et al. “Compiler technologies in deep learning co-design: A survey”. In: *Intelligent Computing* 2 (2023), p. 0040.

[65] Shuoming Zhang et al. “The new compiler stack: a survey on the synergy of LLMs and compilers”. In: *CCF Transactions on High Performance Computing* (2026), pp. 1–32.

[66] Lianmin Zheng et al. “Ansor: Generating {High-Performance} tensor programs for deep learning”. In: *14th USENIX symposium on operating systems design and implementation (OSDI 20)*. 2020, pp. 863–879.

[67] Zhuangwei Zhuang et al. “Discrimination-aware channel pruning for deep neural networks”. In: *Advances in neural information processing systems* 31 (2018).

A Demonstration (Appendix A)

Table A.1: Per-layer performance statistics of YOLOv8n on Jetson Orin Nano (GPU, FP32). Regime is derived from the roofline knee $AI_{knee} \approx 14.56$ FLOPs/byte (using measured BW=55.68 GB/s and peak=810.68 GFLOP/s): Memory-bound if AI$<$ 13.10, Knee/transition if 13.10 $\leq$AI$\leq$ 16.02, Compute-bound if AI$>$ 16.02.

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.0.conv	18.39	7.713	4.8	Memory-bound
model.1.conv	12.74	23.955	18.5	Compute-bound
model.2.cv1.conv	8.84	7.995	5.9	Memory-bound
model.2.m.0.cv1.conv	4.81	35.899	24.5	Compute-bound
model.2.m.0.cv2.conv	4.72	35.899	25.0	Compute-bound
model.2.cv2.conv	9.84	9.593	8.0	Memory-bound
model.3.conv	6.93	47.291	34.1	Compute-bound
model.4.cv1.conv	3.94	15.920	13.3	Knee / transition
model.4.m.0.cv1.conv	2.93	70.416	40.2	Compute-bound
model.4.m.0.cv2.conv	2.74	70.416	43.1	Compute-bound
model.4.m.1.cv1.conv	2.77	70.416	42.6	Compute-bound
model.4.m.1.cv2.conv	2.69	70.416	43.8	Compute-bound
model.4.cv2.conv	5.36	21.192	19.6	Compute-bound
model.5.conv	5.37	85.714	43.9	Compute-bound
model.6.cv1.conv	2.13	30.769	24.6	Compute-bound
model.6.m.0.cv1.conv	2.15	122.034	54.9	Compute-bound
model.6.m.0.cv2.conv	2.00	122.034	58.9	Compute-bound
model.6.m.1.cv1.conv	2.01	122.034	58.6	Compute-bound
model.6.m.1.cv2.conv	1.93	122.034	61.1	Compute-bound
model.6.cv2.conv	3.48	40.506	30.1	Compute-bound
model.7.conv	4.60	97.959	51.3	Compute-bound
model.8.cv1.conv	1.46	48.485	35.9	Compute-bound
model.8.m.0.cv1.conv	2.38	118.033	49.5	Compute-bound
model.8.m.0.cv2.conv	2.32	118.033	50.7	Compute-bound
model.8.cv2.conv	1.92	55.491	41.1	Compute-bound
model.9.cv1.conv	1.07	35.165	24.5	Compute-bound
model.9.cv2.conv	2.66	59.813	39.4	Compute-bound
model.12.cv1.conv	4.62	45.283	34.1	Compute-bound
model.12.m.0.cv1.conv	2.13	122.034	55.5	Compute-bound
model.12.m.0.cv2.conv	2.00	122.034	59.1	Compute-bound
model.12.cv2.conv	3.01	36.641	26.1	Compute-bound
model.15.cv1.conv	6.76	23.821	23.3	Compute-bound
model.15.m.0.cv1.conv	2.93	70.416	40.3	Compute-bound

Continued on next page

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.15.m.0.cv2.conv	2.69	70.416	43.8	Compute-bound
model.15.cv2.conv	4.67	19.085	16.8	Compute-bound
model.16.conv	2.87	53.731	41.1	Compute-bound
model.18.cv1.conv	3.01	36.641	26.1	Compute-bound
model.18.m.0.cv1.conv	2.18	122.034	54.2	Compute-bound
model.18.m.0.cv2.conv	2.02	122.034	58.5	Compute-bound
model.18.cv2.conv	2.98	36.641	26.4	Compute-bound
model.19.conv	2.75	73.096	42.9	Compute-bound
model.21.cv1.conv	1.95	55.491	40.4	Compute-bound
model.21.m.0.cv1.conv	2.42	118.033	48.8	Compute-bound
model.21.m.0.cv2.conv	2.27	118.033	51.9	Compute-bound
model.21.cv2.conv	1.94	55.491	40.5	Compute-bound
model.22.cv2.0.0.conv	8.39	137.799	56.2	Compute-bound
model.22.cv2.0.1.conv	8.27	137.799	57.1	Compute-bound
model.22.cv2.0.2	3.95	15.919	13.3	Knee / transition
model.22.cv2.1.0.conv	3.94	154.839	59.9	Compute-bound
model.22.cv2.1.1.conv	2.06	122.034	57.4	Compute-bound
model.22.cv2.1.2	0.84	15.681	15.6	Compute-bound
model.22.cv2.2.0.conv	2.30	107.063	51.2	Compute-bound
model.22.cv2.2.1.conv	0.92	83.721	32.2	Compute-bound
model.22.cv2.2.2	0.51	14.798	6.4	Compute-bound
model.22.cv3.0.0.conv	11.51	152.381	51.3	Compute-bound
model.22.cv3.0.1.conv	13.53	170.414	54.5	Compute-bound
model.22.cv3.0.2	4.97	19.874	16.5	Compute-bound
model.22.cv3.1.0.conv	5.35	173.494	55.1	Compute-bound
model.22.cv3.1.1.conv	3.30	146.939	55.8	Compute-bound
model.22.cv3.1.2	1.09	19.506	18.9	Compute-bound
model.22.cv3.2.0.conv	2.83	115.663	52.0	Compute-bound
model.22.cv3.2.1.conv	1.25	94.737	37.0	Compute-bound
model.22.cv3.2.2	0.56	18.161	9.1	Compute-bound
model.22.dfl.conv	1.89	0.471	0.6	Strongly memory-bound
Full Model (THOP)	327.00	157.765	13.54	Mixed / Overhead-dominated

Table A.2: Per-layer performance statistics of YOLOv8n on Jetson Orin Nano (GPU, FP32). Regime is derived from the roofline knee $AI_{knee} \approx 14.56$ FLOPs/byte (using measured BW=55.68 GB/s and peak=810.68 GFLOP/s): Memory-bound if AI$<$ 13.10, Knee/transition if 13.10 $\leq$AI$\leq$ 16.02, Compute-bound if AI$>$ 16.02.

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.0.conv	0.78	7.713	114.1	Memory-bound
model.1.conv	0.81	23.955	292.6	Compute-bound
model.2.cv1.conv	0.26	7.995	204.2	Memory-bound
model.2.m.0.cv1.conv	0.45	35.899	259.8	Compute-bound
model.2.m.0.cv2.conv	0.44	35.899	267.2	Compute-bound
model.2.cv2.conv	0.30	9.593	263.5	Memory-bound
model.3.conv	0.46	47.291	508.0	Compute-bound
model.4.cv1.conv	0.20	15.920	262.0	Knee / transition
model.4.m.0.cv1.conv	0.57	70.416	208.7	Compute-bound
model.4.m.0.cv2.conv	0.56	70.416	211.8	Compute-bound
model.4.m.1.cv1.conv	0.56	70.416	211.6	Compute-bound
model.4.m.1.cv2.conv	0.55	70.416	213.3	Compute-bound
model.4.cv2.conv	0.23	21.192	446.4	Compute-bound
model.5.conv	0.40	85.714	593.7	Compute-bound
model.6.cv1.conv	0.18	30.769	296.6	Compute-bound
model.6.m.0.cv1.conv	0.24	122.034	494.4	Compute-bound
model.6.m.0.cv2.conv	0.23	122.034	502.4	Compute-bound
model.6.m.1.cv1.conv	0.24	122.034	501.2	Compute-bound
model.6.m.1.cv2.conv	0.23	122.034	502.4	Compute-bound
model.6.cv2.conv	0.23	40.506	462.6	Compute-bound
model.7.conv	0.52	97.959	455.0	Compute-bound
model.8.cv1.conv	0.18	48.485	284.6	Compute-bound
model.8.m.0.cv1.conv	0.34	118.033	351.1	Compute-bound
model.8.m.0.cv2.conv	0.33	118.033	358.1	Compute-bound
model.8.cv2.conv	0.19	55.491	424.5	Compute-bound
model.9.cv1.conv	0.19	35.165	136.6	Compute-bound
model.9.cv2.conv	0.18	59.813	577.3	Compute-bound
model.12.cv1.conv	0.24	45.283	647.4	Compute-bound
model.12.m.0.cv1.conv	0.25	122.034	480.6	Compute-bound
model.12.m.0.cv2.conv	0.24	122.034	499.3	Compute-bound
model.12.cv2.conv	0.20	36.641	396.8	Compute-bound
model.15.cv1.conv	0.26	23.821	601.7	Compute-bound
model.15.m.0.cv1.conv	0.56	70.416	209.2	Compute-bound
model.15.m.0.cv2.conv	0.55	70.416	213.2	Compute-bound
model.15.cv2.conv	0.21	19.085	372.1	Compute-bound
model.16.conv	0.28	53.731	417.4	Compute-bound
model.18.cv1.conv	0.20	36.641	402.2	Compute-bound
model.18.m.0.cv1.conv	0.24	122.034	488.6	Compute-bound
model.18.m.0.cv2.conv	0.24	122.034	498.5	Compute-bound

Continued on next page

Layer	Time (ms)	AI (FLOPs/byte)	Perf (GFLOP/s)	Regime
model.18.cv2.conv	0.20	36.641	402.5	Compute-bound
model.19.conv	0.36	73.096	327.2	Compute-bound
model.21.cv1.conv	0.18	55.491	426.3	Compute-bound
model.21.m.0.cv1.conv	0.34	118.033	351.7	Compute-bound
model.21.m.0.cv2.conv	0.33	118.033	361.8	Compute-bound
model.21.cv2.conv	0.18	55.491	429.7	Compute-bound
model.22.cv2.0.0.conv	0.57	137.799	830.1	Compute-bound
model.22.cv2.0.1.conv	0.56	137.799	842.6	Compute-bound
model.22.cv2.0.2	0.26	15.919	205.0	Knee / transition
model.22.cv2.1.0.conv	0.45	154.839	524.0	Compute-bound
model.22.cv2.1.1.conv	0.24	122.034	493.1	Compute-bound
model.22.cv2.1.2	0.22	15.681	58.7	Knee / transition
model.22.cv2.2.0.conv	0.41	107.063	288.2	Compute-bound
model.22.cv2.2.1.conv	0.21	83.721	139.2	Compute-bound
model.22.cv2.2.2	0.22	14.798	14.9	Knee / transition
model.22.cv3.0.0.conv	0.66	152.381	891.4	Compute-bound
model.22.cv3.0.1.conv	0.83	170.414	892.5	Compute-bound
model.22.cv3.0.2	0.29	19.874	284.8	Compute-bound
model.22.cv3.1.0.conv	0.42	173.494	707.2	Compute-bound
model.22.cv3.1.1.conv	0.39	146.939	471.0	Compute-bound
model.22.cv3.1.2	0.22	19.506	91.9	Compute-bound
model.22.cv3.2.0.conv	0.38	115.663	385.9	Compute-bound
model.22.cv3.2.1.conv	0.25	94.737	184.1	Compute-bound
model.22.cv3.2.2	0.22	18.161	22.8	Compute-bound
model.22.dfl.conv	0.27	0.471	3.9	Strongly memory-bound
Full Model (THOP)	26.02	157.765	170.22	Mixed / Overhead-dominated

B Demonstration (Appendix B)

YOLOv8n summary: 129 layers, 3,157,200 parameters, 0 gradients, 8.9 GFLOPs.

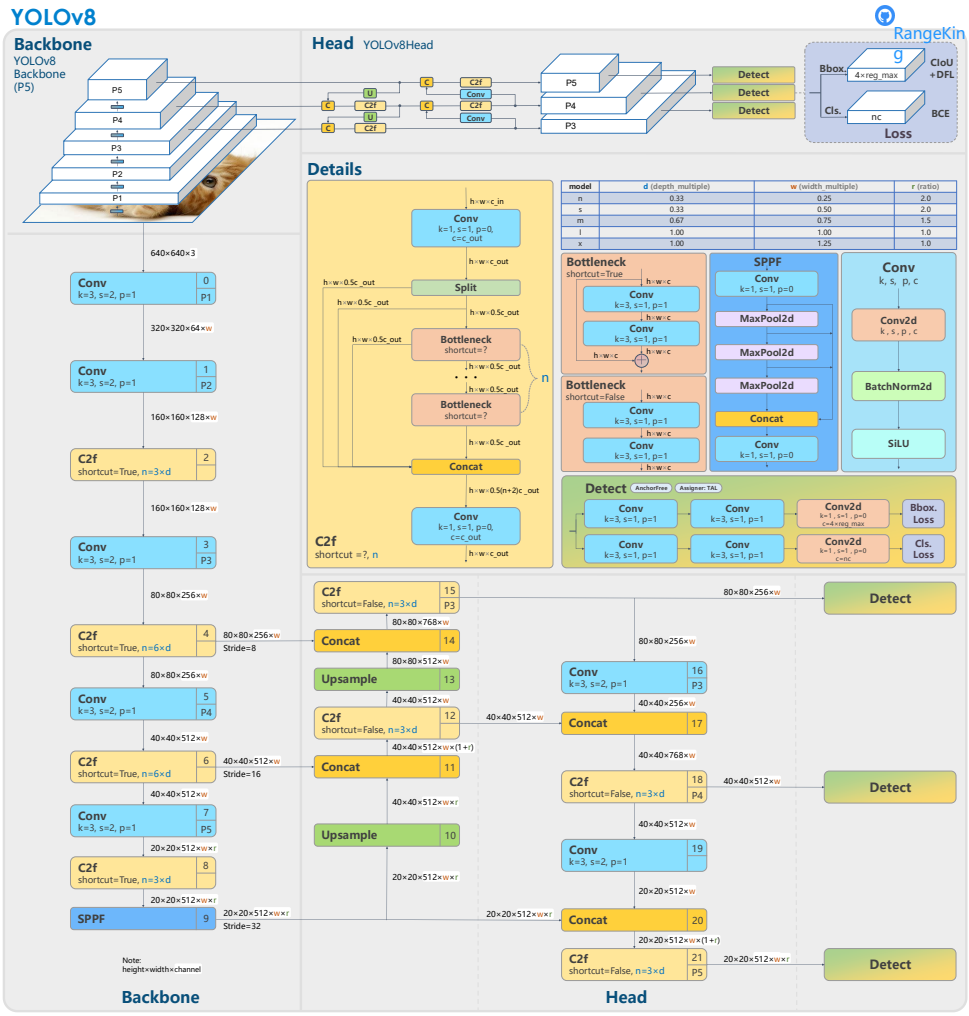

Figure B.1: YOLOv8 structure [53]